



Алгоритмическая алхимия ускорение Python с C и Rust

Модуль 1 - Врата Академии магия Python

Урок 1 - Лаборатория скорости почему Python медленный

Шаг 01:

Миссия 1

Почему Python часто медленный по сравнению с C?

Подсказка: Подумайте о том, как интерпретатор выполняет код.

Ответ:

- **Python интерпретируемый, выполняется построчно**
- Python всегда компилируется в машинный код
- Python использует низкоуровневые системные вызовы по умолчанию
- Python автоматически параллельный и быстрее C

Шаг 02:

Миссия 2

Какой фактор сильно влияет на производительность Python?

Подсказка: Подумайте о глобальной блокировке потоков.

Ответ:

- **Global Interpreter Lock (GIL)**
- Автоматическая компиляция в Rust

- Отсутствие сборщика мусора
- Неограниченная многопоточность

🔮 Шаг 03:

Миссия 3

Что помогает ускорить Python без переписывания на C?

Подсказка: Думайте о библиотеках, которые используют C внутри.

Ответ:

- **NumPy и Cython**
- Чистый Python без библиотек
- Только threading модуль
- Использование input() вместо функций

🔮 Шаг 04:

Миссия 4

Сопоставьте причину медленной работы Python с её объяснением.

Подсказка: Подумайте, что влияет на скорость интерпретатора.

Ответ:

- **GIL** — Блокировка потоков, ограничивающая параллельность
- **Интерпретация** — Код выполняется построчно без компиляции в машинный код
- **Динамическая типизация** — Типы определяются во время выполнения, что замедляет операции
- **Сборка мусора** — Управление памятью требует дополнительного времени

🔮 Шаг 05:

Миссия 5

Сопоставьте подход к ускорению Python с его описанием.

Подсказка: Подумайте, какой инструмент использует C под капотом.

Ответ:

- **Cython** — Компилирует Python в C для ускорения циклов
- **NumPy** — Библиотека для быстрого вычисления массивов
- **PyPy** — JIT-компилятор для ускорения Python-кода
- **multiprocessing** — Использует процессы для обхода GIL

🔮 Шаг 06:

Миссия 6

Сопоставьте ситуацию с выбором оптимизации Python.

Подсказка: Подумайте, что лучше всего подходит для числовых массивов, а что для многопоточности.

Ответ:

- **Ускорение матричных вычислений** — NumPy
- **Параллельное выполнение независимых задач** — multiprocessing
- **Оптимизация горячих циклов** — Cython
- **Обход GIL и ускорение всего интерпретатора** — PyPy

Шаг 07:

Миссия 7

Расположите шаги ускорения Python кода в правильном порядке:

Подсказка: Сначала определите узкие места, потом применяйте оптимизацию и проверяйте результат.

Ответ:

- Профилирование и определение узких мест
- Выбор подходящего инструмента ускорения
- Применение выбранной оптимизации
- Тестирование и проверка корректности

Шаг 08:

Миссия 8

Расположите последовательность ускорения алгоритма с использованием C и Rust:

Подсказка: Сначала выделяем узкие места, потом решаем, какой язык использовать, а в конце проверяем результат.

Ответ:

- Профилирование кода Python
- Определение горячих функций
- Переписывание функций на C или Rust
- Подключение расширений к Python
- Тестирование и проверка ускорения

Шаг 09:

Миссия 9

Какой подход помогает снизить накладные расходы Python на вызов функций?

Подсказка: Подумайте о циклах и повторяющихся вызовах.

Ответ:

- **Вынесение часто вызываемых функций в C-расширения**
- Использование глобальных переменных для всех функций
- Добавление print() в каждый вызов функции
- Создание функций внутри функций для каждого вызова

Шаг 10:

Миссия 10

Что из перечисленного поможет ускорить численные расчёты в Python?

Подсказка: Думайте о библиотеках, которые используют векторизацию.

Ответ:

- **Использование NumPy вместо обычных списков**
- Использование обычных Python списков
- Циклы for вместо функций NumPy
- Прямое использование print() для вычислений

Шаг 11:

Миссия 11

Сопоставьте термин оптимизации с его объяснением.

Подсказка: Подумайте, какие методы помогают ускорять Python на уровне кода и интерпретатора.

Ответ:

- **Vectorization** — Использование операций над массивами вместо циклов
- **JIT-компиляция** — Компиляция кода в машинный код во время выполнения
- **C-расширения** — Переписывание узких мест на C для ускорения
- **Multiprocessing** — Использование процессов для обхода GIL и параллельности

Шаг 12:

Миссия 12

Сопоставьте инструмент с его назначением для ускорения Python.

Подсказка: Подумайте, какой инструмент лучше подходит для массивов, а какой для JIT.

Ответ:

- **NumPy** — Быстрая работа с многомерными массивами
- **Cython** — Компиляция Python-кода в C для ускорения циклов
- **PyPy** — JIT-компилятор, ускоряющий Python во время выполнения
- **multiprocessing** — Параллельное выполнение задач через процессы

Шаг 13:

Миссия 13

Сопоставьте ситуацию с подходящей оптимизацией.

Подсказка: Размышляйте, что лучше подходит для чисел, а что для параллельных задач.

Ответ:

- **Обработка больших массивов чисел** — NumPy
- **Параллельная обработка независимых задач** — multiprocessing
- **Горячие циклы с повторяющимися вычислениями** — Cython
- **Ускорение всего интерпретатора** — PyPy

Шаг 14:

Миссия 14

Расположите шаги оптимизации Python-кода с использованием Cython:

Подсказка: Сначала пишем чистый Python, потом переносим узкие места в Cython и тестируем.

Ответ:

- Написание Python-функций и тестов
- Профилирование и поиск горячих мест
- Переписывание узких мест на Cython
- Компиляция и подключение к Python
- Тестирование корректности и измерение ускорения

Шаг 15:

Миссия 15

Расположите шаги ускорения Python-кода с использованием PyPy:

Подсказка: Подумайте, что нужно сделать, чтобы JIT-компиляция заработала правильно.

Ответ:

- Выбор Python-кода для ускорения
- Запуск кода на PyPy
- Наблюдение за JIT-компиляцией
- Тестирование ускорения и корректности

Шаг 16:

Миссия 16

Расположите шаги комплексного ускорения Python с использованием C, Rust и библиотек:

Подсказка: Сначала выявляем узкие места, потом решаем, что переписывать, и проверяем результат.

Ответ:

- Профилирование и выявление горячих функций
- Выбор метода ускорения (C, Rust, Cython, NumPy)
- Переписывание или создание расширений
- Подключение расширений к Python
- Тестирование и замер ускорения

Урок 2 - Заклинания анализа профилирование кода

Шаг 01:

Миссия 1

Что такое профилирование кода в Python?

Подсказка: Подумайте, как определить узкие места в программе.

Ответ:

- **Измерение времени выполнения разных частей кода**
- Добавление логов для каждой функции
- Автоматическая оптимизация кода интерпретатором
- Использование только встроенных функций Python

Шаг 02:

Миссия 2

Какой инструмент используется для профилирования Python кода?

Подсказка: Подумайте о встроенных модулях стандартной библиотеки.

Ответ:

- **cProfile**
- time.sleep
- random
- math

Шаг 03:

Миссия 3

Почему важно профилировать код перед оптимизацией?

Подсказка: Думайте, какие части кода реально замедляют программу.

Ответ:

- **Чтобы сосредоточиться на узких местах, а не на всей программе**
- Чтобы сразу переписать весь код на С
- Чтобы добавить больше print()
- Чтобы убрать комментарии

🔮 Шаг 04:

Миссия 4

Какой метод профилирования полезен для замеров небольших фрагментов кода?

Подсказка: Думайте о контексте выполнения и точных измерениях.

Ответ:

- **timeit**
- cProfile
- logging
- random

🔮 Шаг 05:

Миссия 5

Сопоставьте тип профилирования с его особенностью.

Подсказка: Думайте, какой инструмент лучше подходит для функций, а какой для всего скрипта.

Ответ:

- **cProfile** — Профилирование всего Python-скрипта с подробной статистикой
- **timeit** — Измерение времени выполнения небольших фрагментов кода
- **line_profiler** — Показывает время на каждой строке функции
- **memory_profiler** — Показывает использование памяти каждой строкой

🔮 Шаг 06:

Миссия 6

Сопоставьте результат профилирования с его применением.

Подсказка: Думайте о том, как интерпретировать данные о времени и ресурсах.

Ответ:

- **Длительные функции** — Переписать на С или использовать Cython
- **Часто вызываемые функции** — Оптимизировать алгоритмы или добавить кеширование
- **Малые фрагменты кода** — Использовать timeit для замеров
- **Потребление памяти** — Использовать memory_profiler для анализа

🔮 Шаг 07:

Миссия 7

Сопоставьте сценарий с правильным инструментом профилирования.

Подсказка: Каждый инструмент подходит для своей задачи.

Ответ:

- **Нужно узнать, какая строка функции медленная** — line_profiler

- **Измерить общее время работы скрипта** — cProfile
- **Тестирование маленькой функции несколько раз** — timeit
- **Проверка потребления памяти скрипта** — memory_profiler

🔗 Шаг 08:

Миссия 8

Сопоставьте термины профилирования с их особенностями.

Подсказка: Подумайте, что измеряет время, а что память.

Ответ:

- **Hot spot** — Часть кода, где программа проводит больше всего времени
- **Call count** — Количество вызовов функции за выполнение программы
- **Execution time** — Время выполнения функции или блока
- **Memory usage** — Объём памяти, занимаемый функцией или скриптом

🔗 Шаг 09:

Миссия 9

Расположите шаги профилирования функции в Python в правильном порядке:

Подсказка: Сначала выбираем функцию, потом замеряем, потом анализируем результаты.

Ответ:

- Выбор функции или блока кода для анализа
- Применение инструмента профилирования
- Сбор данных о времени и вызовах
- Анализ и интерпретация результатов

🔗 Шаг 10:

Миссия 10

Расположите шаги профилирования всего скрипта на Python:

Подсказка: Сначала запускаем скрипт под профайлером, потом анализируем узкие места и оптимизируем.

Ответ:

- Запуск скрипта через cProfile
- Сбор статистики вызовов функций
- Выявление горячих функций
- Оптимизация кода и повторная проверка

🔗 Шаг 11:

Миссия 11

Расположите шаги анализа потребления памяти Python-программы:

Подсказка: Сначала подключаем profiler, потом замеряем, потом анализируем.

Ответ:

- Импорт memory_profiler
- Запуск скрипта с декоратором @profile
- Сбор информации о потреблении памяти
- Анализ результатов и оптимизация

Шаг 12:

Миссия 12

Расположите шаги комплексного профилирования для проекта на Python:

Подсказка: Сначала измеряем, потом анализируем, затем оптимизируем и проверяем.

Ответ:

- Профилирование кода с cProfile и timeit
- Сбор данных по памяти и времени
- Выявление узких мест и горячих функций
- Оптимизация и проверка ускорения

Урок 3 - Первые амулеты ускорения встроенные инструменты

Шаг 01:

Миссия 1

Какой встроенный инструмент Python помогает измерять время выполнения кода?

Подсказка: Подумайте о модуле, который возвращает точное время выполнения функции.

Ответ:

- **timeit**
- random
- logging
- math

Шаг 02:

Миссия 2

Что делает встроенный модуль cProfile?

Подсказка: Думайте о том, какие метрики помогают понять, где код медленный.

Ответ:

- **Собирает статистику вызовов функций и времени выполнения**
- Измеряет потребление памяти всего скрипта
- Генерирует случайные числа
- Создаёт графический интерфейс

Шаг 03:

Миссия 3

Для чего используется встроенный модуль functools?

Подсказка: Подумайте, как можно ускорить повторяющиеся вызовы функций.

Ответ:

- **Для кэширования результатов функций и ускорения повторных вызовов**
- Для генерации случайных чисел
- Для измерения времени выполнения кода
- Для анализа памяти функций

Шаг 04:

Миссия 4

Что делает встроенный модуль `itertools` в Python?

Подсказка: Подумайте, как ускорить работу с последовательностями без создания новых списков.

Ответ:

- **Создаёт эффективные итераторы для последовательностей**
- Измеряет время выполнения функций
- Профилирует функции на память
- Генерирует случайные последовательности

Шаг 05:

Миссия 5

Сопоставьте инструмент Python с его основной функцией ускорения кода.

Подсказка: Каждый инструмент имеет своё назначение для оптимизации.

Ответ:

- **`timeit`** — Измеряет время выполнения отдельных блоков кода
- **`cProfile`** — Собирает статистику вызовов функций и их времени
- **`functools.lru_cache`** — Кэширует результаты функций для ускорения повторных вызовов
- **`itertools`** — Создаёт эффективные итераторы без дополнительных списков

Шаг 06:

Миссия 6

Сопоставьте сценарий с подходящим встроенным инструментом Python.

Подсказка: Думайте о том, что ускоряет память, что — время, а что — итерации.

Ответ:

- **Многократные вызовы функции с одинаковыми параметрами** — `functools.lru_cache`
- **Измерение скорости выполнения небольшого блока кода** — `timeit`
- **Анализ производительности всего скрипта** — `cProfile`
- **Эффективная обработка больших последовательностей** — `itertools`

Шаг 07:

Миссия 7

Сопоставьте функциональность с модулем Python.

Подсказка: Подумайте, какой инструмент решает какие задачи.

Ответ:

- **Собрать статистику по вызовам функций** — `cProfile`
- **Измерять время выполнения кода в миллисекундах** — `timeit`
- **Создавать ленивые итераторы** — `itertools`
- **Автоматически кэшировать результаты функций** — `functools.lru_cache`

Шаг 08:

Миссия 8

Сопоставьте тип задачи с инструментом Python для ускорения.

Подсказка: Думайте, какой инструмент лучше подходит для функций, итераторов или измерений времени.

Ответ:

- **Часто вызываемые функции** — `functools.lru_cache`
- **Обработка больших списков** — `itertools`
- **Замер производительности блока кода** — `timeit`
- **Определение узких мест скрипта** — `cProfile`

Шаг 09:

Миссия 9

Расположите шаги ускорения функции с помощью `lru_cache`:

Подсказка: Сначала подключаем декоратор, потом вызываем функцию, затем анализируем ускорение.

Ответ:

- Импортировать `functools` и подключить `@lru_cache`
- Определить функцию, которую нужно ускорить
- Вызывать функцию с разными аргументами
- Сравнить время выполнения с кэшированием и без него

Шаг 10:

Миссия 10

Расположите шаги измерения времени блока кода с помощью `timeit`:

Подсказка: Сначала импортируем модуль, потом создаём замер, затем интерпретируем результат.

Ответ:

- Импортировать `timeit`
- Создать строку кода или функцию для измерения
- Вызвать `timeit.timeit()` с указанной функцией
- Проанализировать результат и оптимизировать код

Шаг 11:

Миссия 11

Расположите шаги использования `itertools` для обработки последовательностей:

Подсказка: Думайте о том, как создать итератор и пройти по нему.

Ответ:

- Импортировать `itertools`
- Выбрать нужную функцию-итератор
- Применить итератор к последовательности
- Обработать элементы в цикле или через генератор

Шаг 12:

Миссия 12

Расположите шаги профилирования всего скрипта с `cProfile`:

Подсказка: Сначала запускаем скрипт под профайлером, потом анализируем статистику, затем оптимизируем код.

Ответ:

- Импортировать cProfile и запустить скрипт
- Собрать статистику вызовов функций
- Выявить горячие функции
- Оптимизировать код и проверить ускорение

 Шаг 13:

Миссия 13

Расположите шаги комплексного ускорения Python-кода с встроенными инструментами:

Подсказка: Сначала измеряем, потом кэшируем, потом оптимизируем итерации.

Ответ:

- Профилирование функций с timeit и cProfile
- Кэширование повторных вызовов через functools.lru_cache
- Оптимизация последовательностей с itertools
- Повторная проверка скорости выполнения и корректности

 Шаг 14:

Миссия 14

Расположите шаги для анализа горячих функций и ускорения скрипта:

Подсказка: Сначала выявляем узкие места, потом применяем инструменты ускорения.

Ответ:

- Профилировать скрипт с cProfile
- Выявить горячие функции по времени выполнения
- Применить lru_cache или itertools при необходимости
- Перепроверить ускорение и корректность работы

 Урок 4 - Практикум находи «узкие горлышки» в коде алхимика.

 Шаг 01:

Миссия 1

Что является первым шагом в поиске «узких горлышек» в коде алхимика?

Подсказка: Сначала нужно понять, где код тормозит.

Ответ:

- **Профилирование кода с помощью cProfile или timeit**
- Добавление print() в каждую функцию
- Удаление комментариев
- Сразу переписывать код на C

 Шаг 02:

Миссия 2

Какой инструмент Python лучше использовать для измерения времени выполнения отдельной функции?

Подсказка: Этот инструмент позволяет запускать функцию многократно и замерять среднее время.

Ответ:

- **timeit**
- random
- logging
- os

🔮 Шаг 03:

Миссия 3

Что помогает ускорить часто вызываемые функции без изменения их логики?

Подсказка: Думайте о кэше для функции.

Ответ:

- **functools.lru_cache**
- print()
- del
- sys.exit()

🔮 Шаг 04:

Миссия 4

Какой подход помогает обработать большие последовательности без создания новых списков?

Подсказка: Думайте о ленивых итераторах.

Ответ:

- **itertools**
- list()
- dict()
- set()

🔮 Шаг 05:

Миссия 5

Сопоставьте инструмент Python с его функцией ускорения кода:

Подсказка: Подумайте, что измеряет время, что кэширует, что создаёт итераторы.

Ответ:

- **timeit** — Измеряет время выполнения функции или блока кода
- **cProfile** — Собирает статистику вызовов функций всего скрипта
- **functools.lru_cache** — Кэширует результаты функций для ускорения повторных вызовов
- **itertools** — Создаёт ленивые итераторы для последовательностей

🔮 Шаг 06:

Миссия 6

Сопоставьте сценарий с подходящим инструментом Python:

Подсказка: Каждый инструмент имеет своё применение.

Ответ:

- **Измерение скорости маленькой функции** — timeit
- **Профилирование всего скрипта** — cProfile
- **Обработка больших списков без лишней памяти** — itertools

- **Ускорение часто вызываемой функции** — `functools.lru_cache`

🔗 Шаг 07:

Миссия 7

Сопоставьте тип проблемы с инструментом Python:

Подсказка: Думайте, что решает кэш, что профайлер, а что итератор.

Ответ:

- **Медленная функция, вызываемая часто** — `functools.lru_cache`
- **Большой массив данных** — `itertools`
- **Найти узкие места скрипта** — `cProfile`
- **Замерить время блока кода** — `timeit`

🔗 Шаг 08:

Миссия 8

Расположите шаги ускорения функции с помощью `lru_cache`:

Подсказка: Сначала подключаем декоратор, потом вызываем функцию, затем анализируем ускорение.

Ответ:

- Импортировать `functools` и подключить `@lru_cache`
- Определить функцию, которую нужно ускорить
- Вызывать функцию с разными аргументами
- Сравнить время выполнения с кэшированием и без него

🔗 Шаг 09:

Миссия 9

Расположите шаги измерения времени блока кода с `timeit`:

Подсказка: Сначала импортируем модуль, потом создаём замер, затем интерпретируем результат.

Ответ:

- Импортировать `timeit`
- Создать строку кода или функцию для измерения
- Вызвать `timeit.timeit()` с указанной функцией
- Проанализировать результат и оптимизировать код

🔗 Шаг 10:

Миссия 10

Расположите шаги профилирования скрипта с `cProfile`:

Подсказка: Сначала запускаем скрипт под профайлером, потом анализируем статистику, затем оптимизируем код.

Ответ:

- Импортировать `cProfile` и запустить скрипт
- Собрать статистику вызовов функций
- Выявить горячие функции
- Оптимизировать код и проверить ускорение

Шаг 11:

Миссия 11

Расположите шаги измерения времени блока кода с `timeit`:

Подсказка: Сначала импортируем модуль, потом создаём замер, затем интерпретируем результат.

Ответ:

- Импортировать `timeit`
- Создать строку кода или функцию для измерения
- Вызвать `timeit.timeit()` с указанной функцией
- Проанализировать результат и оптимизировать код

Шаг 12:

Миссия 12

Расположите шаги профилирования скрипта с `cProfile`:

Подсказка: Сначала запускаем скрипт под профайлером, потом анализируем статистику, затем оптимизируем код.

Ответ:

- Импортировать `cProfile` и запустить скрипт
- Собрать статистику вызовов функций
- Выявить горячие функции
- Оптимизировать код и проверить ускорение

Шаг 13:

Миссия 13

Какой метод помогает уменьшить количество аллокаций памяти при обработке больших данных?

Подсказка: Подумайте о ленивой генерации элементов.

Ответ:

- **Использование генераторов и `itertools`**
- Создание списка через `list()` с каждым элементом
- Копирование всех элементов вручную
- Сразу импортировать все данные в память

Шаг 14:

Миссия 14

Какой подход позволяет ускорить вычисления при повторных вызовах одной функции с одинаковыми аргументами?

Подсказка: Декораторы могут хранить результаты предыдущих вызовов.

Ответ:

- **Декоратор `@functools.lru_cache`**
- Вызов функции через `exec()`
- Каждый раз пересоздавать данные
- Сортировка списка внутри функции

🔮 Шаг 15:

Миссия 15

Сопоставьте инструмент с типом оптимизации:

Подсказка: Думайте о памяти, времени и повторных вызовах.

Ответ:

- **timeit** — Замер времени выполнения отдельных функций
- **cProfile** — Анализ производительности всего скрипта
- **itertools** — Ленивая генерация последовательностей
- **functools.lru_cache** — Кэширование результатов функций для ускорения повторных вызовов

🔮 Шаг 16:

Миссия 16

Сопоставьте проблему с оптимизацией:

Подсказка: Каждое решение направлено на конкретный тип узкого места.

Ответ:

- **Медленная функция, вызываемая часто** — lru_cache
- **Большие списки и массивы** — itertools
- **Поиск горячих функций** — cProfile
- **Нужна точная оценка времени блока** — timeit

📖 Модуль 2 - Руны Си — язык древних магов ✂

📖 Урок 1 - Знакомство с магией Python.h — как Python разговаривает с C.

🔮 Шаг 01:

Миссия 1

Что нужно подключить, чтобы Python мог вызывать функции C?

Подсказка: Ищите заголовочный файл, который связывает Python и C.

Ответ:

- **#include**
- #include
- #include
- #include

🔮 Шаг 02:

Миссия 2

Какой тип используется для представления Python-объекта в C?

Подсказка: В Python.h почти всё оперирует этим универсальным типом.

Ответ:

- **PyObject ***
- int
- double
- char *

Шаг 03:

Миссия 3

Сопоставьте функции Python.h с их назначением:

Подсказка: Каждая функция решает отдельную задачу при работе с PyObject.

Ответ:

- **Py_Initialize()** — Инициализация интерпретатора Python
- **Py_Finalize()** — Завершение работы интерпретатора Python
- **PyRun_SimpleString()** — Выполнение Python-кода из C
- **PyImport_ImportModule()** — Импорт Python-модуля из C

Шаг 04:

Миссия 4

Что нужно сделать, чтобы вернуть значение из C-функции обратно в Python?

Подсказка: Подумайте о создании Python-объекта для возврата.

Ответ:

- **Использовать Py_BuildValue()**
- Использовать printf()
- Просто вернуть int
- Использовать malloc()

Шаг 05:

Миссия 5

Сопоставьте макросы Python.h с их функцией:

Подсказка: Некоторые макросы упрощают проверку типов и управление ссылками.

Ответ:

- **Py_INCREF()** — Увеличение счетчика ссылок объекта
- **Py_DECREF()** — Уменьшение счетчика ссылок объекта
- **Py_RETURN_NONE** — Возврат None из функции C
- **PyErr_SetString()** — Установка ошибки Python

Шаг 06:

Миссия 6

Какой шаг обязателен перед использованием Python-объектов в C?

Подсказка: Без этого интерпретатор Python работать не будет.

Ответ:

- **Инициализация интерпретатора с Py_Initialize()**
- Вызов malloc()
- Создание файла .c
- Использование printf()

Шаг 07:

Миссия 7

Расположите шаги создания расширения C для Python:

Подсказка: Сначала пишем код, потом компилируем и подключаем в Python.

Ответ:

- Написать C-файл с функциями Python.h
- Создать метод таблицу PyMethodDef
- Определить PyModuleDef
- Скомпилировать в shared library
- Импортировать в Python

Шаг 08:

Миссия 8

Какой компилятор чаще всего используют для сборки Python C-расширений на Windows?

Подсказка: Он совместим с компиляцией официального CPython.

Ответ:

- **MSVC (Microsoft Visual C++)**
- GCC
- Clang
- Borland C

Шаг 09:

Миссия 9

Сопоставьте типы Python с C-конвертацией:

Подсказка: Каждое преобразование имеет свою функцию.

Ответ:

- **int** — PyLong_AsLong()
- **float** — PyFloat_AsDouble()
- **str** — PyUnicode_AsUTF8()
- **list** — PyList_GetItem()

Шаг 10:

Миссия 10

Что нужно делать после окончания работы с Python-объектом в C, если вы его создали вручную?

Подсказка: Подумайте о памяти и ссылках.

Ответ:

- **Вызывать Py_DECREF()**
- Ничего не делать
- printf()
- Использовать malloc()

Шаг 11:

Миссия 11

Расположите шаги для вызова функции C из Python:

Подсказка: Сначала расширение подключается, потом вызывается метод.

Ответ:

- Импортировать модуль C в Python

- Вызвать функцию из Python
- Функция C использует PyObject* для аргументов
- Возврат значения через Py_BuildValue()

🔧 Шаг 12:

Миссия 12

Какая функция позволяет вызвать строку Python-кода из C?

Подсказка: Это самая простая функция для выполнения Python-кода без создания объектов.

Ответ:

- **PyRun_SimpleString()**
- PyImport_ImportModule()
- PyEval_EvalCode()
- Py_CompileString()

🔧 Шаг 13:

Миссия 13

Сопоставьте ошибки Python.h с их причиной:

Подсказка: Обратите внимание на частые проблемы новичков.

Ответ:

- **Segmentation fault** — Неправильная работа с PyObject* или памятью
- **Reference leak** — Не уменьшен счетчик ссылок Py_DECREF
- **ImportError** — Модуль не найден или не скомпилирован

📖 Урок 2 - Создание простого модуля на C (hello world, factorial).

🔧 Шаг 01:

Миссия 1

Какой заголовочный файл нужно подключить, чтобы создать Python-модуль на C?

Подсказка: Это основной заголовочный файл Python для C API.

Ответ:

- **#include**
- #include
- #include
- #include

🔧 Шаг 02:

Миссия 2

Какая функция инициализирует интерпретатор Python в C-модуле?

Подсказка: Без этой функции PyObject* работать не будет.

Ответ:

- **Py_Initialize()**
- Py_Finalize()
- PyRun_SimpleString()
- PyImport_ImportModule()

Шаг 03:

Миссия 3

Сопоставьте функции Python C API с их назначением:

Подсказка: Каждая функция выполняет уникальную задачу при работе с Python-объектами.

Ответ:

- **PyModule_Create** — Создает новый Python-модуль
- **Py_BuildValue** — Преобразует C-значения в PyObject* для возврата
- **PyImport_ImportModule** — Импортирует Python-модуль в C
- **Py_DECREF** — Уменьшает счетчик ссылок объекта

Шаг 04:

Миссия 4

Какую функцию использовать для возврата None из C-функции в Python?

Подсказка: Это макрос, упрощающий возврат Py_None.

Ответ:

- **Py_RETURN_NONE**
- return NULL
- return 0
- Py_Finalize

Шаг 05:

Миссия 5

Сопоставьте макросы Python C API с их функцией:

Подсказка: Эти макросы помогают управлять ссылками и ошибками.

Ответ:

- **Py_INCREF** — Увеличивает счетчик ссылок объекта
- **Py_DECREF** — Уменьшает счетчик ссылок объекта
- **PyErr_SetString** — Устанавливает ошибку Python
- **Py_RETURN_NONE** — Возвращает None

Шаг 06:

Миссия 6

Какая последовательность действий нужна для создания простого C-модуля с функцией hello_world?

Подсказка: Сначала пишем функцию, затем таблицу методов и модуль.

Ответ:

- **Написать функцию -> Таблица методов -> PyModuleDef -> Скомпилировать -> Импортировать**
- Импортировать -> Скомпилировать -> Таблица методов
- Скомпилировать -> Написать функцию -> PyModuleDef
- Написать функцию -> Py_Initialize -> Таблица методов

Шаг 07:

Миссия 7

Расположите шаги создания функции factorial на C для Python:

Подсказка: Сначала пишем функцию на C, затем подключаем к Python.

Ответ:

- Написать рекурсивную функцию factorial
- Создать таблицу методов PyMethodDef
- Определить структуру модуля PyModuleDef
- Скомпилировать C-файл в shared library
- Импортировать модуль в Python и вызвать функцию

🔗 Шаг 08:

Миссия 8

Какой тип данных используется для передачи аргументов из Python в C?

Подсказка: В C API все Python-объекты имеют один универсальный тип.

Ответ:

- **PyObject***
- int
- float
- char*

🔗 Шаг 09:

Миссия 9

Сопоставьте функции для конвертации между C и Python:

Подсказка: Каждая функция отвечает за свой тип данных.

Ответ:

- **int -> PyObject*** — Py_BuildValue("i", value)
- **double -> PyObject*** — Py_BuildValue("d", value)
- **PyObject* -> int** — PyLong_AsLong(obj)
- **PyObject* -> double** — PyFloat_AsDouble(obj)

🔗 Шаг 10:

Миссия 10

Что произойдет, если забыть вызывать Py_DECREF для созданного PyObject*?

Подсказка: Подумайте о памяти.

Ответ:

- **Произойдет утечка памяти**
- Ничего не произойдет
- Python завершится с ошибкой
- Функция вернет NULL автоматически

🔗 Шаг 11:

Миссия 11

Расположите шаги от написания hello_world на C до вызова из Python:

Подсказка: Сначала создаем функцию, затем подключаем модуль.

Ответ:

- Написать C-функцию `hello_world`
- Создать `PyMethodDef`
- Определить `PyModuleDef`
- Скомпилировать в `.so/.pyd`
- Импортировать в Python
- Вызвать функцию из Python

 Шаг 12:

Миссия 12

Как проверить, что C-модуль правильно подключился в Python?

Подсказка: Используйте стандартные функции Python для импорта.

Ответ:

- **Попытаться импортировать модуль и вызвать функцию**
- Запустить gcc без компиляции
- Использовать `Py_Initialize` без импорта
- Проверить только наличие `.c` файла

 Шаг 13:

Миссия 13

Сопоставьте ошибки компиляции C-модуля с причиной:

Подсказка: Ошибки часто связаны с `Python.h` или `PyObject`.

Ответ:

- **undefined reference to `Py_Initialize`** — `Python.h` не подключен или `Python dev` не установлен
- **error: expected ';' before '}'** — Синтаксическая ошибка в C-коде
- **'PyObject' undeclared** — `Python.h` не подключен

 Шаг 14:

Миссия 14

Что делает `PyModuleDef`?

Подсказка: Это структура, описывающая модуль для Python.

Ответ:

- **Описывает модуль и его методы для Python**
- Вызывает Python-функции
- Определяет глобальные переменные C
- Компилирует модуль автоматически

 Шаг 15:

Миссия 15

Расположите шаги вызова функции `factorial` с Python в правильном порядке:

Подсказка: Сначала импорт, потом передаем аргументы и получаем результат.

Ответ:

- Импортировать модуль

- Вызвать функцию с аргументом
- Python передает аргумент как PyObject*
- С функция вычисляет факториал
- Возврат результата через Py_BuildValue
- Python получает результат

Урок 3 - Сборка заклинаний distutils, setuptools.

Шаг 01:

Миссия 1

Какой файл обычно создают для сборки Python-модуля с использованием distutils?

Подсказка: Это конфигурационный скрипт с инструкциями по сборке.

Ответ:

- **setup.py**
- requirements.txt
- pyproject.toml
- MANIFEST.in

Шаг 02:

Миссия 2

Какой модуль Python рекомендуют использовать вместо устаревшего distutils?

Подсказка: Это современный инструмент для сборки пакетов и управления зависимостями.

Ответ:

- **setuptools**
- os
- sys
- shutil

Шаг 03:

Миссия 3

Сопоставьте ключевые параметры setup.py с их назначением:

Подсказка: Каждый параметр определяет важную часть пакета.

Ответ:

- **name** — Название вашего пакета
- **version** — Версия сборки пакета
- **packages** — Список подпакетов, включаемых в сборку
- **scripts** — Исполняемые скрипты, добавляемые к пакету

Шаг 04:

Миссия 4

Какой командой собирают пакет с использованием setuptools?

Подсказка: Эта команда запускается в терминале и читает setup.py.

Ответ:

- **python setup.py sdist**
- **pip install setup.py**

- `python build.py`
- `python setup.py test`

🔧 Шаг 05:

Миссия 5

Сопоставьте форматы дистрибутива Python с их описанием:

Подсказка: Один архив для исходников, другой — уже готовый пакет для установки.

Ответ:

- **`sdist`** — Source distribution — архив исходников
- **`bdist_wheel`** — Binary distribution — готовый wheel пакет

🔧 Шаг 06:

Миссия 6

Что нужно добавить в `setup.py` для автоматического включения всех подпакетов?

Подсказка: Список пакетов можно генерировать автоматически.

Ответ:

- **`packages=find_packages()`**
- `packages=[]`
- `include_package_data=True`
- `zip_safe=False`

🔧 Шаг 07:

Миссия 7

Расположите шаги сборки Python-пакета в правильном порядке:

Подсказка: Сначала создаем `setup.py`, потом собираем и устанавливаем.

Ответ:

- Создать `setup.py`
- Запустить сборку `python setup.py sdist`
- Создать wheel файл с `bdist_wheel`
- Установить пакет с `pip install`

🔧 Шаг 08:

Миссия 8

Как включить дополнительные файлы (например, данные) в пакет через `setuptools`?

Подсказка: Нужно настроить `MANIFEST.in` и опцию в `setup.py`.

Ответ:

- **`include_package_data=True`**
- `packages=[]`
- `zip_safe=False`
- `install_requires=[]`

🔧 Шаг 09:

Миссия 9

Сопоставьте команды сборки `setuptools` с их результатом:

Подсказка: Один создает исходники, другой — wheel.

Ответ:

- **python setup.py sdist** — Создает архив исходников
- **python setup.py bdist_wheel** — Создает wheel пакет
- **python setup.py install** — Устанавливает пакет локально

🔮 Шаг 10:

Миссия 10

Как указать зависимости пакета в setup.py?

Подсказка: Есть специальный параметр, где перечисляются сторонние пакеты.

Ответ:

- **install_requires=['requests','numpy']**
- **requires=[]**
- **packages=[]**
- **setup_requires=[]**

🔮 Шаг 11:

Миссия 11

Расположите действия при публикации пакета на PyPI:

Подсказка: Сначала создаем дистрибутив, потом загружаем на сервер.

Ответ:

- Создать source и wheel дистрибутив
- Установить twine
- Загрузить пакеты с twine upload dist/*
- Проверить установку через pip install

🔮 Шаг 12:

Миссия 12

Как сделать пакет совместимым с zip-install через setuptools?

Подсказка: Есть параметр, который отвечает за безопасность архива.

Ответ:

- **zip_safe=True**
- **zip_safe=False**
- **include_package_data=False**
- **packages=[]**

🔮 Шаг 13:

Миссия 13

Какую команду использовать для локального тестирования установки пакета?

Подсказка: pip может устанавливать пакеты из текущей директории.

Ответ:

- **pip install .**
- **python setup.py test**
- **pip install setup.py**

- `python -m build`

Шаг 14:

Миссия 14

Расположите шаги подготовки пакета к публикации в правильном порядке:

Подсказка: Сначала проверяем файлы, потом создаем дистрибутив, затем загружаем.

Ответ:

- Создать/проверить `setup.py` и `MANIFEST.in`
- Создать source и wheel дистрибутив
- Проверить локальную установку через `pip install .`
- Загрузить пакет на PyPI с помощью `twine`

Шаг 15:

Миссия 15

Что укажет `pip` при успешной установке вашего пакета?

Подсказка: Это вывод в терминале после завершения установки.

Ответ:

- **Successfully installed**
- Build failed
- Module not found
- Error: missing setup.py

Шаг 16:

Миссия 16

Сопоставьте файлы пакета Python с их ролью:

Подсказка: Один файл описывает сборку, другой — данные пакета.

Ответ:

- **setup.py** — Скрипт для сборки и установки пакета
- **MANIFEST.in** — Содержит дополнительные файлы для включения в пакет
- **README.md** — Документация пакета для PyPI
- **LICENSE** — Лицензионные условия

Урок 4 - Работа с массивами и числами в C.

Шаг 01:

Миссия 1

Как в C объявить массив из 10 целых чисел?

Подсказка: Используйте квадратные скобки и указание размера массива.

Ответ:

- `int arr[10];`
- `int arr;`
- `array arr(10);`
- `int arr();`

🔮 Шаг 02:

Миссия 2

Как получить длину массива в C, если известен тип и имя массива?

Подсказка: Используйте `sizeof` для вычисления количества элементов.

Ответ:

- `sizeof(arr)/sizeof(arr[0])`
- `length(arr)`
- `arr.size()`
- `count(arr)`

🔮 Шаг 03:

Миссия 3

Сопоставьте типы чисел в C с их диапазоном:

Подсказка: Помните о размерах типов и знаковости.

Ответ:

- **int** — От -2,147,483,648 до 2,147,483,647
- **unsigned int** — От 0 до 4,294,967,295
- **short** — От -32,768 до 32,767
- **long long** — От -9,223,372,036,854,775,808 до 9,223,372,036,854,775,807

🔮 Шаг 04:

Миссия 4

Как правильно инициализировать массив из 5 элементов числами 1,2,3,4,5?

Подсказка: Используйте фигурные скобки и перечисление значений.

Ответ:

- `int arr[5] = {1,2,3,4,5};`
- `int arr = {1,2,3,4,5};`
- `arr = [1,2,3,4,5];`
- `int arr(5) = {1,2,3,4,5};`

🔮 Шаг 05:

Миссия 5

Сопоставьте арифметические операции с результатом при $a=5$, $b=2$:

Подсказка: Учтите правила деления целых чисел.

Ответ:

- $a + b$ — 7
- $a - b$ — 3
- $a * b$ — 10
- a / b — 2

🔮 Шаг 06:

Миссия 6

Как в C пройти цикл по всем элементам массива из 10 элементов?

Подсказка: Используйте цикл for и индекс элемента.

Ответ:

- **for(int i=0;i<10;i++)**
- for i in range(10)
- while(i<=10)
- foreach(int i in arr)

Шаг 07:

Миссия 7

Расположите шаги вычисления суммы элементов массива:

Подсказка: Сначала инициализируем переменную, потом проходим массив и накапливаем сумму.

Ответ:

- Объявить переменную sum=0
- Проходить циклом по всем элементам массива
- Добавлять каждый элемент к sum
- Вывести результат

Шаг 08:

Миссия 8

Как определить максимальный элемент массива в C?

Подсказка: Сравниваем каждый элемент с текущим максимумом.

Ответ:

- **Использовать цикл и переменную max**
- Использовать функция max(arr)
- Присвоить arr[0] и done
- Нельзя определить

Шаг 09:

Миссия 9

Сопоставьте функции стандартной библиотеки C с их действием:

Подсказка: Некоторые работают с числами, другие с массивами.

Ответ:

- **memset** — Заполняет блок памяти указанным байтом
- **memcpy** — Копирует блок памяти
- **abs** — Возвращает модуль числа
- **pow** — Возводит число в степень

Шаг 10:

Миссия 10

Что происходит при выходе за пределы массива в C?

Подсказка: Компилятор не проверяет индексы, будьте осторожны.

Ответ:

- **Поведение неопределено (undefined behavior)**

- Автоматическая ошибка компиляции
- Возврат нуля
- Выдача предупреждения и продолжение работы

🔗 Шаг 11:

Миссия 11

Расположите шаги поиска минимального элемента в массиве:

Подсказка: Сначала задаем начальный минимум, потом сравниваем с каждым элементом.

Ответ:

- Инициализировать `min=arr[0]`
- Пройти циклом по массиву
- Если элемент `< min`, обновить `min`
- Вывести `min`

🔗 Шаг 12:

Миссия 12

Как отсортировать массив чисел в C без использования библиотек?

Подсказка: Реализуйте простой алгоритм сортировки, например, пузырьком.

Ответ:

- **Реализовать цикл с обменом элементов по условию**
- Вызвать `sort(arr)`
- Использовать `reverse(arr)`
- Присвоить `arr=sorted(arr)`

📖 Урок 5 - Практикум ускорить сортировку или поиск чисел.

🔗 Шаг 01:

Миссия 1

Какой метод сортировки в C обычно быстрее для больших массивов?

Подсказка: Учтите сложность алгоритмов и эффективность на больших данных.

Ответ:

- **Quick Sort**
- Bubble Sort
- Insertion Sort
- Selection Sort

🔗 Шаг 02:

Миссия 2

Что лучше использовать для поиска числа в отсортированном массиве?

Подсказка: Эффективнее сравнивать элементы по принципу деления пополам.

Ответ:

- **Binary Search**
- Linear Search
- Bubble Search
- Exponential Search

🔮 Шаг 03:

Миссия 3

Сопоставьте алгоритмы поиска и сортировки с их характеристиками:

Подсказка: Обратите внимание на сложность и способ работы с массивом.

Ответ:

- **Bubble Sort** — Простой обмен соседних элементов, медленный для больших массивов
- **Quick Sort** — Разделение массива и рекурсивная сортировка, быстрая средняя сложность
- **Linear Search** — Проверка каждого элемента по очереди
- **Binary Search** — Деление отсортированного массива пополам для поиска

🔮 Шаг 04:

Миссия 4

Какой подход ускоряет сортировку почти отсортированного массива?

Подсказка: Некоторые алгоритмы эффективны, если массив уже частично отсортирован.

Ответ:

- **Insertion Sort**
- Quick Sort
- Bubble Sort
- Selection Sort

🔮 Шаг 05:

Миссия 5

Сопоставьте типы массивов и поисковых алгоритмов с оптимальной ситуацией:

Подсказка: Учтите, отсортирован массив или нет, размер и частота поиска.

Ответ:

- **Отсортированный массив** — Binary Search подходит для быстрого поиска
- **Неотсортированный массив** — Linear Search подходит для простого перебора
- **Маленький массив** — Insertion Sort может быть быстрее, чем Quick Sort
- **Большой массив** — Quick Sort эффективен благодаря логарифмическому разбиению

🔮 Шаг 06:

Миссия 6

Как улучшить производительность поиска числа в очень большом массиве?

Подсказка: Подумайте о предварительной сортировке для ускорения поиска.

Ответ:

- **Отсортировать массив и использовать Binary Search**
- Использовать Linear Search без сортировки
- Применить Bubble Sort после каждого поиска
- Ничего не менять

🔮 Шаг 07:

Миссия 7

Расположите шаги для ускоренной сортировки массива с помощью Quick Sort:

Подсказка: Сначала выбираем опорный элемент, затем делим массив и рекурсивно сортируем.

Ответ:

- Выбрать опорный элемент (pivot)
- Разделить массив на элементы меньше и больше pivot
- Рекурсивно отсортировать подмассивы
- Объединить результаты

Шаг 08:

Миссия 8

Какой метод сортировки стабильный и подходит для маленьких массивов?

Подсказка: Сохраняет порядок одинаковых элементов.

Ответ:

- **Insertion Sort**
- Quick Sort
- Heap Sort
- Selection Sort

Шаг 09:

Миссия 9

Какой метод сортировки в C обычно быстрее для больших массивов?

Подсказка: Учтите сложность алгоритмов и эффективность на больших данных.

Ответ:

- **Quick Sort**
- Bubble Sort
- Insertion Sort
- Selection Sort

Шаг 10:

Миссия 10

Что лучше использовать для поиска числа в отсортированном массиве?

Подсказка: Эффективнее сравнивать элементы по принципу деления пополам.

Ответ:

- **Binary Search**
- Linear Search
- Bubble Search
- Exponential Search

Шаг 11:

Миссия 11

Сопоставьте алгоритмы поиска и сортировки с их характеристиками:

Подсказка: Обратите внимание на сложность и способ работы с массивом.

Ответ:

- **Bubble Sort** — Простой обмен соседних элементов, медленный для больших массивов
- **Quick Sort** — Разделение массива и рекурсивная сортировка, быстрая средняя сложность

- **Linear Search** — Проверка каждого элемента по очереди
- **Binary Search** — Деление отсортированного массива пополам для поиска

🔮 Шаг 12:

Миссия 12

Какой подход ускоряет сортировку почти отсортированного массива?

Подсказка: Некоторые алгоритмы эффективны, если массив уже частично отсортирован.

Ответ:

- **Insertion Sort**
- Quick Sort
- Bubble Sort
- Selection Sort

🔮 Шаг 13:

Миссия 13

Сопоставьте типы массивов и поисковых алгоритмов с оптимальной ситуацией:

Подсказка: Учтите, отсортирован массив или нет, размер и частота поиска.

Ответ:

- **Отсортированный массив** — Binary Search подходит для быстрого поиска
- **Неотсортированный массив** — Linear Search подходит для простого перебора
- **Маленький массив** — Insertion Sort может быть быстрее, чем Quick Sort
- **Большой массив** — Quick Sort эффективен благодаря логарифмическому разбиению

🔮 Шаг 14:

Миссия 14

Как улучшить производительность поиска числа в очень большом массиве?

Подсказка: Подумайте о предварительной сортировке для ускорения поиска.

Ответ:

- **Отсортировать массив и использовать Binary Search**
- Использовать Linear Search без сортировки
- Применить Bubble Sort после каждого поиска
- Ничего не менять

🔮 Шаг 15:

Миссия 15

Расположите шаги для ускоренной сортировки массива с помощью Quick Sort:

Подсказка: Сначала выбираем опорный элемент, затем делим массив и рекурсивно сортируем.

Ответ:

- Выбрать опорный элемент (pivot)
- Разделить массив на элементы меньше и больше pivot
- Рекурсивно отсортировать подмассивы
- Объединить результаты

Шаг 16:

Миссия 16

Какой метод сортировки стабильный и подходит для маленьких массивов?

Подсказка: Сохраняет порядок одинаковых элементов.

Ответ:

- **Insertion Sort**
- Quick Sort
- Heap Sort
- Selection Sort

Модуль 3 - Зельеварение Cython

Урок 1 - Что такое Cython и зачем алхимики его используют.

Шаг 01:

Миссия 1

Алиса пишет числовые вычисления на Python, и они работают медленно. Что может помочь ускорить код?

Подсказка: Иногда Python слишком медленный для тяжёлых циклов.

Ответ:

- **Использовать Cython для компиляции кода**
- Переписать на JavaScript
- Сжать код в zip-файл
- Удалить комментарии

Шаг 02:

Миссия 2

Боб хочет ускорить обработку больших массивов чисел. Что из нижеперечисленного правильно?

Подсказка: Смотрите на библиотеки, которые компилируются в машинный код.

Ответ:

- **Использовать Cython и static typing для чисел**
- Писать все циклы в чистом Python
- Добавить больше print() для отладки
- Перевести код в текстовый файл

Шаг 03:

Миссия 3

Чарли заметил, что функция работает быстро на маленьких данных, но медленно на больших. Что стоит сделать первым?

Подсказка: Сначала найдите узкое место, а не переписывайте весь код.

Ответ:

- **Профилировать функцию, чтобы понять, где теряется время**
- Добавить больше функций print
- Удалить все комментарии

- Сразу переписать на C

Шаг 04:

Миссия 4

 Задача: Сопоставление понятий Cython

Понятия:

Ответ:

- Python
- Cython
- Статическая типизация
- Компиляция в C

Определения:

1. Язык высокого уровня для разработки приложений.
2. Надстройка над Python, позволяющая ускорять код.
3. Определение типов переменных заранее для повышения скорости.
4. Преобразование кода в C для быстрой работы.

Подсказка: Обратите внимание на различие между Python и его ускоренной версией.

- **Python** — Язык высокого уровня для разработки приложений.
- **Cython** — Надстройка над Python, позволяющая ускорять код.
- **Статическая типизация** — Определение типов переменных заранее для повышения скорости.
- **Компиляция в C** — Преобразование кода в C для быстрой работы.

Шаг 05:

Миссия 5

 Сопоставьте ошибки Python и их последствия при работе с Cython

Понятия:

Ответ:

- Неправильный тип переменной
- Неоптимальный цикл
- Отсутствие cdef
- Импорт стандартного модуля

Определения:

1. Замедление выполнения, потеря преимуществ Cython
2. Ошибка компиляции или неожиданный результат
3. Низкая скорость из-за работы Python-интерпретатора
4. Проблемы не возникают, если модуль корректно используется

Подсказка: Cython сильно выигрывает на статических типах и cdef-функциях.

- **Неправильный тип переменной** — Ошибка компиляции или неожиданный результат
- **Неоптимальный цикл** — Низкая скорость из-за работы Python-интерпретатора
- **Отсутствие cdef** — Замедление выполнения, потеря преимуществ Cython
- **Импорт стандартного модуля** — Проблемы не возникают, если модуль корректно используется

Шаг 06:

Миссия 6

 Сопоставьте термины: профилирование и оптимизация

Понятия:

Ответ:

- Профилирование
- Оптимизация
- Hotspot
- Cythonize

Определения:

1. Поиск самых медленных частей программы
2. Изменение кода для ускорения
3. Участок кода, который занимает больше всего времени
4. Процесс компиляции Python в C с помощью Cython

Подсказка: Найдите медленные участки и решите, как их ускорить.

- **Профилирование** — Поиск самых медленных частей программы
- **Оптимизация** — Изменение кода для ускорения
- **Hotspot** — Участок кода, который занимает больше всего времени
- **Cythonize** — Процесс компиляции Python в C с помощью Cython

Шаг 07:

Миссия 7

Расположите шаги ускорения Python кода с помощью Cython в правильном порядке:

1. Писать код в Python
2. Определить узкие места с помощью профилирования
3. Добавить cdef и статические типы
4. Компилировать через cythonize
5. Тестировать и сравнить скорость

Подсказка: Сначала код существует, потом анализ, потом ускорение, затем проверка.

Ответ:

- Писать код в Python
- Определить узкие места с помощью профилирования
- Добавить cdef и статические типы
- Компилировать через cythonize
- Тестировать и сравнить скорость

Шаг 08:

Миссия 8

Расположите шаги от идеи до быстрого модуля на Cython:

1. Написание Python-функции
2. Определение типов и добавление cdef
3. Компиляция через cythonize
4. Замеры производительности

5. Использование ускоренного модуля в проекте

Подсказка: Следуйте естественному порядку от прототипа до внедрения.

Ответ:

- Написание Python-функции
- Определение типов и добавление cdef
- Компиляция через cythonize
- Замеры производительности
- Использование ускоренного модуля в проекте

Шаг 09:

Миссия 9

 Сопоставьте термины: профилирование и оптимизация кода

Понятия:

Ответ:

- Профилирование
- Оптимизация
- Hotspot
- Cythonize

Определения:

1. Поиск самых медленных частей программы
2. Изменение кода для ускорения
3. Участок кода, который занимает больше всего времени
4. Процесс компиляции Python в C с помощью Cython

Подсказка: Сначала находите медленные места, затем ускоряйте их.

- **Профилирование** — Поиск самых медленных частей программы
- **Оптимизация** — Изменение кода для ускорения
- **Hotspot** — Участок кода, который занимает больше всего времени
- **Cythonize** — Процесс компиляции Python в C с помощью Cython

Шаг 10:

Миссия 10

Расположите шаги ускорения Python кода с помощью Cython:

1. Написание исходной Python-функции
2. Определение узких мест через профилирование
3. Добавление cdef и статических типов
4. Компиляция через cythonize
5. Тестирование и сравнение производительности

Подсказка: Сначала анализируем, потом улучшаем и проверяем.

Ответ:

- Написание исходной Python-функции
- Определение узких мест через профилирование
- Добавление cdef и статических типов
- Компиляция через cythonize

- Тестирование и сравнение производительности

Шаг 11:

Миссия 11

Расположите шаги от идеи до быстрого модуля на Cython:

1. Написание Python-функции
2. Определение типов и добавление cdef
3. Компиляция через cythonize
4. Замеры производительности
5. Использование ускоренного модуля в проекте

Подсказка: Следуйте естественному порядку: от прототипа до внедрения.

Ответ:

- Написание Python-функции
- Определение типов и добавление cdef
- Компиляция через cythonize
- Замеры производительности
- Использование ускоренного модуля в проекте

Шаг 12:

Миссия 12

Дейв хочет ускорить многократное вычисление факториала. Что принесёт наибольшую пользу?

Подсказка: Смотрите на типы переменных и циклы.

Ответ:

- **Использовать Cython с cdef и int типами**
- Добавить рекурсивные вызовы без изменений
- Считать факториал вручную на бумаге
- Писать print() на каждом шаге цикла

Шаг 13:

Миссия 13

Эми проверяет производительность кода и видит узкое место в цикле. Что лучше сделать?

Подсказка: Сначала подумайте о типах и компиляции.

Ответ:

- **Переписать цикл с использованием cdef и статических типов**
- Добавить больше print() для отладки
- Удалить цикл и ничего не менять
- Перевести весь проект в JavaScript

Шаг 14:

Миссия 14

 Сопоставьте типы функций Cython с их особенностями

Понятия:

Ответ:

- def

- `cdef`
- `cpdef`
- `inline`

Определения:

1. Обычная Python-функция
2. Функция с C-уровнем, доступная только в Cython
3. Функция доступна и в Python, и в Cython
4. Функция для вставки в другие функции без накладных расходов

Подсказка: Смотрите на то, кто будет вызывать функцию и на каком уровне.

- **`def`** — Обычная Python-функция
- **`cdef`** — Функция с C-уровнем, доступная только в Cython
- **`cpdef`** — Функция доступна и в Python, и в Cython
- **`inline`** — Функция для вставки в другие функции без накладных расходов

Шаг 15:

Миссия 15

 Сопоставьте типы данных Cython с их преимуществами

Понятия:

Ответ:

- `int`
- `float`
- `double`
- `PyObject*`

Определения:

1. Целые числа, быстрое выполнение арифметики
2. Числа с плавающей точкой, умеренная точность
3. Числа с двойной точностью, высокая точность
4. Любой Python-объект, гибкость, но медленно

Подсказка: Типы с указанием C обычно быстрее, но менее гибки.

- **`int`** — Целые числа, быстрое выполнение арифметики
- **`float`** — Числа с плавающей точкой, умеренная точность
- **`double`** — Числа с двойной точностью, высокая точность
- **`PyObject*`** — Любой Python-объект, гибкость, но медленно

Шаг 16:

Миссия 16

Расположите шаги компиляции модуля через Cython:

1. Написание `.pyx` файла
2. Создание `setup.py` с `cythonize`
3. Запуск `python setup.py build_ext --inplace`
4. Импорт ускоренного модуля в проект
5. Тестирование работы функции

Подсказка: Сначала создаём файл, потом компилируем, затем проверяем результат.

Ответ:

- Написание .рух файла
- Создание setup.py с cythonize
- Запуск python setup.py build_ext --inplace
- Импорт ускоренного модуля в проект
- Тестирование работы функции

Урок 2 - Компиляция Python-скриптов в бинарные заклинания.

Шаг 01:

Миссия 1

Алиса хочет превратить свой Python-скрипт в быстрое бинарное заклинание. Что стоит использовать в первую очередь?

Подсказка: Сначала нужно выбрать инструмент компиляции.

Ответ:

- **Cython для компиляции в бинарный модуль**
- Переписать код на Java
- Сжать скрипт в zip-файл
- Использовать только Python без изменений

Шаг 02:

Миссия 2

Боб заметил, что его цикл с миллионами итераций медленно работает. Какой подход ускорит скрипт?

Подсказка: Статическая типизация в Cython может помочь.

Ответ:

- **Использовать cdef для переменных и функций**
- Добавить print() внутри цикла
- Писать код в другом файле без изменений
- Запускать цикл медленнее с time.sleep

Шаг 03:

Миссия 3

Чарли хочет скомпилировать модуль и использовать его в другом проекте. С чего начать?

Подсказка: Сначала создайте файл с кодом для компиляции.

Ответ:

- **Создать .рух файл с функциями**
- Сразу писать setup.py без кода
- Переносить код в текстовый документ
- Использовать print() для компиляции

Шаг 04:

Миссия 4

 Сопоставьте понятия и их определения, связанные с компиляцией Python:

Понятия:

Ответ:

- `pyx` файл
- `cdef`
- `cythonize`
- `setup.py`

Определения:

1. Файл с Python-кодом для компиляции в C
2. Объявление переменных и функций с C-типами для ускорения
3. Команда для компиляции `.pyx` в C-модуль
4. Скрипт для настройки сборки модуля

Подсказка: Подумайте, кто что делает в процессе сборки.

- **`pyx` файл** — Файл с Python-кодом для компиляции в C
- **`cdef`** — Объявление переменных и функций с C-типами для ускорения
- **`cythonize`** — Команда для компиляции `.pyx` в C-модуль
- **`setup.py`** — Скрипт для настройки сборки модуля

 Шаг 05:

Миссия 5

 Сопоставьте термины с их особенностями:

Понятия:

Ответ:

- `def`
- `cpdef`
- `inline`
- `cdef class`

Определения:

1. Обычная Python-функция
2. Функция доступна и в Python, и в Cython
3. Функция для вставки в другие функции без накладных расходов
4. Класс с C-уровнем для ускорения методов

Подсказка: Различайте доступность функции и уровень её скорости.

- **`def`** — Обычная Python-функция
- **`cpdef`** — Функция доступна и в Python, и в Cython
- **`inline`** — Функция для вставки в другие функции без накладных расходов
- **`cdef class`** — Класс с C-уровнем для ускорения методов

 Шаг 06:

Миссия 6

 Сопоставьте типы данных в Cython с их преимуществами:

Понятия:

Ответ:

- `int`

- float
- double
- PyObject*

Определения:

1. Целые числа, быстрое выполнение арифметики
2. Числа с плавающей точкой, умеренная точность
3. Числа с двойной точностью, высокая точность
4. Любой Python-объект, гибкость, но медленнее

Подсказка: C-тип быстрее, Python-объект универсальнее.

- **int** — Целые числа, быстрое выполнение арифметики
- **float** — Числа с плавающей точкой, умеренная точность
- **double** — Числа с двойной точностью, высокая точность
- **PyObject*** — Любой Python-объект, гибкость, но медленнее

Шаг 07:

Миссия 7

Расположите шаги компиляции Python-скрипта через Cython:

1. Создание .pyx файла
2. Написание setup.py с cythonize
3. Запуск python setup.py build_ext --inplace
4. Импорт ускоренного модуля в проект
5. Тестирование функций

Подсказка: Сначала создаём файл, потом компилируем, затем проверяем работу.

Ответ:

- Создание .pyx файла
- Написание setup.py с cythonize
- Запуск python setup.py build_ext --inplace
- Импорт ускоренного модуля в проект
- Тестирование функций

Шаг 08:

Миссия 8

Дейв хочет ускорить обработку больших списков чисел. Что принесёт наибольшую пользу?

Подсказка: Смотрите на типы переменных и использование Cython.

Ответ:

- **Использовать cdef и статические типы для чисел**
- Добавить print() в каждом цикле
- Переписать код на Python без изменений
- Сжать скрипт в архив

Шаг 09:

Миссия 9

Эми проверяет производительность своего скрипта. Цикл с миллионами итераций медленно выполняется. Что стоит сделать?

Подсказка: Сначала улучшите типизацию и компиляцию, а не переписывайте всё заново.

Ответ:

- **Применить `cdef` и типы для переменных цикла**
- Добавить `print()` для отладки
- Удалить цикл и ничего не менять
- Переписать весь скрипт на другой язык

Шаг 10:

Миссия 10

 Сопоставьте действия с результатами при компиляции через Cython:

Понятия:

Ответ:

- Добавление `cdef`
- Использование `cpdef`
- Компиляция через `cythonize`
- Импорт модуля

Определения:

1. Ускорение функции за счёт C-типов
2. Функция доступна и в Python, и в Cython
3. Создание бинарного модуля из `.pyx`
4. Использование ускоренной функции в проекте

Подсказка: Разделите изменения кода и процесс компиляции.

- **Добавление `cdef`** — Ускорение функции за счёт C-типов
- **Использование `cpdef`** — Функция доступна и в Python, и в Cython
- **Компиляция через `cythonize`** — Создание бинарного модуля из `.pyx`
- **Импорт модуля** — Использование ускоренной функции в проекте

Шаг 11:

Миссия 11

 Сопоставьте файлы и их роль в Cython-проекте:

Понятия:

Ответ:

- `.pyx`
- `setup.py`
- `.c` файл
- Бинарный модуль (`.so/.pyd`)

Определения:

1. Исходный Python-код для компиляции
2. Скрипт для сборки модуля
3. Промежуточный файл после `cythonize`
4. Финальный ускоренный модуль для импорта

Подсказка: Следуйте цепочке от кода до рабочего бинарника.

- **.pyx** — Исходный Python-код для компиляции
- **setup.py** — Скрипт для сборки модуля
- **.c файл** — Промежуточный файл после cythonize
- **Бинарный модуль (.so/.pyd)** — Финальный ускоренный модуль для импорта

Шаг 12:

Миссия 12

 Сопоставьте ошибки при компиляции и их последствия:

Понятия:

Ответ:

- Неправильный тип переменной
- Отсутствие cdef
- Синтаксическая ошибка в .pyx
- Импорт модуля до компиляции

Определения:

1. Ошибка компиляции или неожиданный результат
2. Замедление выполнения, теряются преимущества Cython
3. Сборка модуля невозможна
4. Python выдаст ImportError

Подсказка: Разделяйте логические ошибки и синтаксис.

- **Неправильный тип переменной** — Ошибка компиляции или неожиданный результат
- **Отсутствие cdef** — Замедление выполнения, теряются преимущества Cython
- **Синтаксическая ошибка в .pyx** — Сборка модуля невозможна
- **Импорт модуля до компиляции** — Python выдаст ImportError

Шаг 13:

Миссия 13

Расположите шаги оптимизации Python-скрипта через Cython:

1. Написание Python-кода
2. Определение узких мест через профилирование
3. Добавление cdef и статических типов
4. Компиляция через cythonize
5. Тестирование и сравнение скорости

Подсказка: Сначала анализируем, потом ускоряем, затем проверяем.

Ответ:

- Написание Python-кода
- Определение узких мест через профилирование
- Добавление cdef и статических типов
- Компиляция через cythonize
- Тестирование и сравнение скорости

Шаг 14:

Миссия 14

Расположите процесс создания бинарного модуля:

1. Создание .pyx файла
2. Добавление cdef/статических типов
3. Запуск setup.py с cythonize
4. Появление .c файла
5. Получение бинарного модуля (.so/.pyd)

Подсказка: Следуйте цепочке от исходного кода до бинарника.

Ответ:

- Создание .pyx файла
- Добавление cdef/статических типов
- Запуск setup.py с cythonize
- Появление .c файла
- Получение бинарного модуля (.so/.pyd)

 **Шаг 15:**

Миссия 15

Расположите шаги интеграции бинарного модуля в проект:

1. Компиляция модуля через cythonize
2. Импорт модуля в проект
3. Использование функций модуля
4. Проверка результатов на тестах

Подсказка: Сначала создаём модуль, затем используем и проверяем его.

Ответ:

- Компиляция модуля через cythonize
- Импорт модуля в проект
- Использование функций модуля
- Проверка результатов на тестах

 **Урок 3 - Аннотации типов превращение переменных в «твёрдые кристаллы».**

 **Шаг 01:**

Миссия 1

Алиса хочет ускорить свой Python-скрипт и превратить переменные в «твёрдые кристаллы». С чего начать?

Подсказка: Сначала выберите способ задания типов переменных.

Ответ:

- **Использовать cdef и статические типы**
- Добавить комментарии к переменным
- Переписать код на другой язык
- Не менять код, оставить всё как есть

 **Шаг 02:**

Миссия 2

Боб заметил, что цикл с миллионом итераций выполняется медленно. Что ускорит его работу?

Подсказка: Тип переменной и её использование критичны.

Ответ:

- **Объявить переменные через `cdef` с конкретным типом**
- Добавить `print()` внутри цикла
- Переписать цикл на обычный Python без изменений
- Разбить цикл на несколько функций без типизации

Шаг 03:

Миссия 3

Чарли хочет, чтобы функции с типизированными переменными были быстрее. Какой способ наиболее эффективен?

Подсказка: Разделяйте Python и Cython-уровень функций.

Ответ:

- **Использовать `cdef` для функций и переменных**
- Добавить типы в `docstring`
- Объявить переменные через глобальные константы
- Оставить `def` без изменений

Шаг 04:

Миссия 4

 Сопоставьте термины аннотаций типов с их значением:

Понятия:

Ответ:

- `cdef`
- `cpdef`
- `int`
- `float`

Определения:

1. Объявление переменных или функций на C-уровне
2. Функция доступна и в Python, и в Cython
3. Целочисленный тип для быстрой арифметики
4. Числа с плавающей точкой, умеренная точность

Подсказка: Типы влияют на скорость и точность вычислений.

- **`cdef`** — Объявление переменных или функций на C-уровне
- **`cpdef`** — Функция доступна и в Python, и в Cython
- **`int`** — Целочисленный тип для быстрой арифметики
- **`float`** — Числа с плавающей точкой, умеренная точность

Шаг 05:

Миссия 5

 Сопоставьте объявление переменной с её преимуществом:

Понятия:

Ответ:

- `cdef int x`
- `cdef double y`
- `cdef PyObject* z`
- `cdef float w`

Определения:

1. Быстрая работа с целыми числами
2. Высокая точность для чисел с плавающей точкой
3. Гибкость, поддержка любых Python-объектов
4. Средняя точность для чисел с плавающей точкой

Подсказка: Смотрите на тип и его влияние на производительность.

- **`cdef int x`** — Быстрая работа с целыми числами
- **`cdef double y`** — Высокая точность для чисел с плавающей точкой
- **`cdef PyObject* z`** — Гибкость, поддержка любых Python-объектов
- **`cdef float w`** — Средняя точность для чисел с плавающей точкой

Шаг 06:

Миссия 6

 Сопоставьте типы функций с их возможностями:

Понятия:

Ответ:

- `def`
- `cdef`
- `cpdef`
- `inline`

Определения:

1. Обычная Python-функция
2. Функция на C-уровне для быстрого вызова
3. Функция доступна и в Python, и в Cython
4. Функция для вставки без накладных расходов

Подсказка: Различайте доступность и производительность функций.

- **`def`** — Обычная Python-функция
- **`cdef`** — Функция на C-уровне для быстрого вызова
- **`cpdef`** — Функция доступна и в Python, и в Cython
- **`inline`** — Функция для вставки без накладных расходов

Шаг 07:

Миссия 7

Расположите шаги оптимизации Python-скрипта через аннотации типов:

1. Написание Python-кода
2. Выявление узких мест через профилирование
3. Добавление `cdef` и статических типов
4. Компиляция через `cythonize`

5. Тестирование и проверка скорости

Подсказка: Сначала пишем код, потом анализируем, ускоряем и проверяем.

Ответ:

- Написание Python-кода
- Выявление узких мест через профилирование
- Добавление cdef и статических типов
- Компиляция через cythonize
- Тестирование и проверка скорости

Шаг 08:

Миссия 8

 Сопоставьте ошибки типизации с последствиями:

Понятия:

Ответ:

- Неправильный тип переменной
- Отсутствие cdef
- Использование Python-типов вместо C-типов
- Объявление переменной после использования

Определения:

1. Компиляция может завершиться ошибкой
2. Скрипт работает медленно
3. Функция не получает ускорения
4. Python выдаёт NameError

Подсказка: Разделяйте ошибки синтаксиса и типизации.

- **Неправильный тип переменной** — Компиляция может завершиться ошибкой
- **Отсутствие cdef** — Скрипт работает медленно
- **Использование Python-типов вместо C-типов** — Функция не получает ускорения
- **Объявление переменной после использования** — Python выдаёт NameError

Шаг 09:

Миссия 9

 Сопоставьте сценарии с рекомендуемым типом аннотации:

Понятия:

Ответ:

- Счётчик циклов
- Большие числа с высокой точностью
- Списки Python-объектов
- Промежуточные вычисления с плавающей точкой

Определения:

1. cdef int
2. cdef double
3. cdef PyObject*
4. cdef float

Подсказка: Подбирайте тип под задачу: целые, числа с плавающей точкой или объекты Python.

- **Счётчик циклов** — `cdef int`
- **Большие числа с высокой точностью** — `cdef double`
- **Списки Python-объектов** — `cdef PyObject*`
- **Промежуточные вычисления с плавающей точкой** — `cdef float`

🔧 Шаг 10:

Миссия 10

📄 Сопоставьте функции с оптимизацией:

Понятия:

Ответ:

- `def` обычная функция
- `cdef` функция
- `cpdef` функция
- `inline` функция

Определения:

1. Python-доступная функция без ускорения
2. Функция на C-уровне, быстро вызывается внутри Cython
3. Функция доступна и из Python, и из Cython
4. Встраиваемая функция без накладных расходов

Подсказка: Разделяйте уровень доступа и производительность.

- **`def` обычная функция** — Python-доступная функция без ускорения
- **`cdef` функция** — Функция на C-уровне, быстро вызывается внутри Cython
- **`cpdef` функция** — Функция доступна и из Python, и из Cython
- **`inline` функция** — Встраиваемая функция без накладных расходов

🔧 Шаг 11:

Миссия 11

Расположите шаги использования аннотаций типов для ускорения скрипта:

1. Писать код на Python
2. Определить узкие места с профилированием
3. Добавить `cdef`/статические типы переменных
4. Компиляция через `cythonize`
5. Тестирование и проверка ускорения

Подсказка: Сначала анализируем, потом ускоряем, затем проверяем.

Ответ:

- Писать код на Python
- Определить узкие места с профилированием
- Добавить `cdef`/статические типы переменных
- Компиляция через `cythonize`
- Тестирование и проверка ускорения

Шаг 12:

Миссия 12

Расположите последовательность работы с типизированными функциями:

1. Создание Python-функции
2. Добавление cdef или cpdef
3. Компиляция через cythonize
4. Импорт модуля
5. Использование функции и проверка скорости

Подсказка: Сначала пишем, потом ускоряем, затем используем.

Ответ:

- Создание Python-функции
- Добавление cdef или cpdef
- Компиляция через cythonize
- Импорт модуля
- Использование функции и проверка скорости

Шаг 13:

Миссия 13

Расположите шаги интеграции бинарного модуля с аннотациями типов в проект:

1. Компиляция модуля
2. Импорт модуля в проект
3. Вызов ускоренных функций
4. Проверка результатов и тесты

Подсказка: Сначала создаём модуль, затем используем его в проекте.

Ответ:

- Компиляция модуля
- Импорт модуля в проект
- Вызов ускоренных функций
- Проверка результатов и тесты

Шаг 14:

Миссия 14

Эми хочет, чтобы переменные в её проекте были максимально быстрыми. Что выбрать для целочисленных счётчиков?

Подсказка: Подумайте о прямом C-типе.

Ответ:

- **cdef int**
- float
- PyObject*
- str

Шаг 15:

Миссия 15

Фред использует числа с плавающей точкой для вычислений. Что оптимальнее для высокой точности?

Подсказка: Есть тип, который быстрее float и точнее.

Ответ:

- **cdef double**
- cdef float
- int
- PyObject*

Шаг 16:

Миссия 16

 Сопоставьте типы аннотаций с их областью применения:

Понятия:

Ответ:

- cdef int
- cdef float
- cdef double
- cdef PyObject*

Определения:

1. Целочисленные счётчики и индексы
2. Промежуточные вычисления с умеренной точностью
3. Вычисления с высокой точностью
4. Любые Python-объекты, списки, словари

Подсказка: Подбирайте тип под задачу и ожидаемую точность.

- **cdef int** — Целочисленные счётчики и индексы
- **cdef float** — Промежуточные вычисления с умеренной точностью
- **cdef double** — Вычисления с высокой точностью
- **cdef PyObject*** — Любые Python-объекты, списки, словари

Шаг 17:

Миссия 17

Расположите шаги превращения Python-переменной в «твёрдый кристалл»:

1. Определить тип переменной
2. Добавить cdef или cpdef
3. Компиляция через cythonize
4. Использование ускоренной переменной в коде
5. Тестирование производительности

Подсказка: Сначала выбираем тип, потом внедряем и проверяем.

Ответ:

- Определить тип переменной
- Добавить cdef или cpdef
- Компиляция через cythonize
- Использование ускоренной переменной в коде

- Тестирование производительности

Шаг 18:

Миссия 18

Расположите шаги работы с функцией, полностью ускоренной через аннотации типов:

1. Создание Python-функции
2. Добавление cdef/статических типов
3. Компиляция через cythonize
4. Импорт модуля
5. Использование и тестирование функции

Подсказка: Сначала пишем функцию, потом ускоряем, затем используем и проверяем.

Ответ:

- Создание Python-функции
- Добавление cdef/статических типов
- Компиляция через cythonize
- Импорт модуля
- Использование и тестирование функции

Урок 4 - Ускорение циклов и рекурсии.

Шаг 01:

Миссия 1

Анна хочет ускорить простой цикл в Python. Что из нижеперечисленного поможет ей больше всего?

Подсказка: Сначала подумайте о компиляции и типизации.

Ответ:

- **Использовать Cython с cdef переменными**
- Добавить print() в цикл
- Увеличить интервал цикла
- Писать комментарии для каждой строки

Шаг 02:

Миссия 2

Боб использует рекурсивную функцию для вычисления факториала. Как её можно ускорить?

Подсказка: Подумайте о статической типизации и компиляции.

Ответ:

- **Добавить cdef типы и компилировать через Cython**
- Сделать функцию вложенной
- Использовать global переменные
- Удалить return

Шаг 03:

Миссия 3

Чарли хочет ускорить большой цикл с числами с плавающей точкой. Что выбрать?

Подсказка: Подбирайте тип под данные.

Ответ:

- **cdef double**
- int
- str
- list

Шаг 04:

Миссия 4

 Сопоставьте методы ускорения циклов с их эффектом:

Понятия:

Ответ:

- Использовать cdef переменные
- Компиляция через cythonize
- Избегать Python-объектов внутри цикла
- Использовать встроенные функции

Определения:

1. Повышение скорости за счёт статической типизации
2. Создание бинарного модуля для быстрого выполнения
3. Меньше накладных расходов на Python-интерпретатор
4. Оптимизация простых операций через C-функции

Подсказка: Каждый метод ускоряет цикл по-своему.

- **Использовать cdef переменные** — Повышение скорости за счёт статической типизации
- **Компиляция через cythonize** — Создание бинарного модуля для быстрого выполнения
- **Избегать Python-объектов внутри цикла** — Меньше накладных расходов на Python-интерпретатор
- **Использовать встроенные функции** — Оптимизация простых операций через C-функции

Шаг 05:

Миссия 5

 Сопоставьте типы циклов с оптимизацией:

Понятия:

Ответ:

- for i in range(n)
- while условие
- Вложенный цикл
- Цикл с Python-объектами

Определения:

1. Лучше использовать cdef int для счётчика
2. Можно преобразовать в for с типизированным счётчиком
3. Подходит inline функции для ускорения
4. Следует минимизировать и использовать PyObject* с осторожностью

Подсказка: Разные циклы требуют разных подходов к оптимизации.

- **for i in range(n)** — Лучше использовать cdef int для счётчика
- **while условие** — Можно преобразовать в for с типизированным счётчиком
- **Вложенный цикл** — Подходит inline функции для ускорения
- **Цикл с Python-объектами** — Следует минимизировать и использовать PyObject* с осторожностью

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте рекурсивные функции с подходом к ускорению:

Понятия:

Ответ:

- Факториал
- Фибоначчи без мемоизации
- Фибоначчи с мемоизацией
- Глубокая рекурсия

Определения:

1. Можно ускорить через cdef int и cpdef функции
2. Следует добавить мемоизацию для уменьшения количества вызовов
3. Использовать cdef и встроенные типы для повышения скорости
4. Рассмотреть преобразование в итеративный вариант

Подсказка: Рекурсия разная: иногда лучше мемоизация, иногда итерация.

- **Факториал** — Можно ускорить через cdef int и cpdef функции
- **Фибоначчи без мемоизации** — Следует добавить мемоизацию для уменьшения количества вызовов
- **Фибоначчи с мемоизацией** — Использовать cdef и встроенные типы для повышения скорости
- **Глубокая рекурсия** — Рассмотреть преобразование в итеративный вариант

🔧 Шаг 07:

Миссия 7

Расположите шаги ускорения цикла через Cython:

1. Писать цикл на Python
2. Определить узкие места с профилированием
3. Добавить cdef типы переменных
4. Компиляция через cythonize
5. Тестирование и проверка ускорения

Подсказка: Сначала пишем код, потом оптимизируем и проверяем.

Ответ:

- Писать цикл на Python
- Определить узкие места с профилированием
- Добавить cdef типы переменных
- Компиляция через cythonize
- Тестирование и проверка ускорения

Шаг 08:

Миссия 8

Расположите последовательность ускорения рекурсивной функции:

1. Создать функцию на Python
2. Добавить cdef/статические типы
3. Рассмотреть мемоизацию
4. Компиляция через cythonize
5. Использование и тестирование функции

Подсказка: Сначала пишем функцию, потом ускоряем и тестируем.

Ответ:

- Создать функцию на Python
- Добавить cdef/статические типы
- Рассмотреть мемоизацию
- Компиляция через cythonize
- Использование и тестирование функции

Шаг 09:

Миссия 9

Эмма хочет ускорить вложенные циклы. Что будет эффективнее?

Подсказка: Подумайте о типах переменных внутри вложенных циклов.

Ответ:

- **Использовать cdef int для всех счётчиков**
- Оставить Python int
- Использовать str для индексов
- Добавить print() в каждый цикл

Шаг 10:

Миссия 10

Джек хочет ускорить рекурсивное вычисление чисел Фибоначчи. Что ему стоит сделать в первую очередь?

Подсказка: Подумайте, как уменьшить количество повторных вызовов функции.

Ответ:

- **Добавить мемоизацию**
- Удалить return
- Использовать глобальные переменные
- Печатать промежуточные значения

Шаг 11:

Миссия 11

 Сопоставьте типы циклов с оптимизацией в Cython:

Понятия:

Ответ:

- Простой цикл for

- Вложенный цикл `for`
- Цикл `while`
- Цикл с объектами Python

Определения:

1. Использовать `cdef` для счётчиков
2. Типизировать все переменные и уменьшить Python-объекты
3. Преобразовать `for` с типизированным счётчиком
4. Минимизировать использование `PyObject` внутри цикла

Подсказка: Разные циклы требуют разных подходов.

- **Простой цикл `for`** — Использовать `cdef` для счётчиков
- **Вложенный цикл `for`** — Типизировать все переменные и уменьшить Python-объекты
- **Цикл `while`** — Преобразовать `for` с типизированным счётчиком
- **Цикл с объектами Python** — Минимизировать использование `PyObject` внутри цикла

Шаг 12:

Миссия 12

 Сопоставьте подходы к ускорению рекурсивных функций с их эффектом:

Понятия:

Ответ:

- Мемоизация
- Статические типы через `cdef`
- Преобразование в итерацию
- Использование `cpdef` функций

Определения:

1. Сокращает количество повторных вызовов
2. Ускоряет выполнение через статическую типизацию
3. Снижает нагрузку на стек при глубокой рекурсии
4. Обеспечивает быстрый вызов функции из Python и C

Подсказка: Каждая оптимизация решает разные проблемы рекурсии.

- **Мемоизация** — Сокращает количество повторных вызовов
- **Статические типы через `cdef`** — Ускоряет выполнение через статическую типизацию
- **Преобразование в итерацию** — Снижает нагрузку на стек при глубокой рекурсии
- **Использование `cpdef` функций** — Обеспечивает быстрый вызов функции из Python и C

Шаг 13:

Миссия 13

Расположите шаги ускорения вложенных циклов:

1. Писать вложенный цикл на Python
2. Профилировать для нахождения узких мест
3. Типизировать счётчики через `cdef`
4. Минимизировать Python-объекты внутри цикла
5. Компиляция через `cythonize`
6. Тестирование и измерение ускорения

Подсказка: Логика: сначала пишем, потом типизируем и проверяем.

Ответ:

- Писать вложенный цикл на Python
- Профилировать для нахождения узких мест
- Типизировать счётчики через cdef
- Минимизировать Python-объекты внутри цикла
- Компиляция через cythonize
- Тестирование и измерение ускорения

🔮 Шаг 14:

Миссия 14

Расположите последовательность ускорения рекурсивной функции Фибоначчи:

1. Написать рекурсивную функцию на Python
2. Добавить мемоизацию
3. Типизировать переменные через cdef
4. Использовать cpdef функции
5. Компиляция через cythonize
6. Тестирование и замер скорости

Подсказка: Сначала пишем функцию, потом оптимизируем, потом проверяем ускорение.

Ответ:

- Написать рекурсивную функцию на Python
- Добавить мемоизацию
- Типизировать переменные через cdef
- Использовать cpdef функции
- Компиляция через cythonize
- Тестирование и замер скорости

🔮 Шаг 15:

Миссия 15

Сара хочет ускорить сложный цикл с числами с плавающей точкой. Что эффективнее всего?

Подсказка: Подумайте о типах переменных и накладных расходах Python.

Ответ:

- **Использовать cdef double**
- int
- str
- list

🔮 Шаг 16:

Миссия 16

Майк хочет ускорить рекурсивный обход дерева. Какой метод лучше?

Подсказка: Подумайте о сочетании итерации и типизации.

Ответ:

- **Преобразовать рекурсию в итерацию и добавить cdef типы**
- Удалить return

- Использовать str для индексов
- Добавить print() на каждом уровне

Урок 5 - Практикум ускорить обработку числовых рядов

Шаг 01:

Миссия 1

Аня хочет ускорить суммирование длинного числового ряда в Python. Что даст наибольший эффект?

Подсказка: Подумайте о типах данных и Python-объектах.

Ответ:

- **Типизировать числа через cdef int или double**
- Использовать list вместо массива
- Писать цикл на Python без изменений
- Добавлять print() в каждом шаге цикла

Шаг 02:

Миссия 2

Боб хочет ускорить вычисление среднего ряда чисел. Какой метод эффективнее всего?

Подсказка: Подумайте, как уменьшить накладные расходы Python.

Ответ:

- **Использовать массивы с типами Cython**
- Складывать в Python list
- Вычислять через float без cdef
- Печать промежуточных значений

Шаг 03:

Миссия 3

Катя хочет ускорить обработку ряда чисел с плавающей точкой. Какой тип переменной выбрать?

Подсказка: Python float медленнее C double в циклах.

Ответ:

- **cdef double**
- int
- Python float
- str

Шаг 04:

Миссия 4

 Сопоставьте подход к ускорению рядов чисел с его эффектом:

Понятия:

Ответ:

- Использование cdef для переменных
- Массивы Cython вместо list
- Компиляция через cythonize

- Минимизация Python-объектов в цикле

Эффекты:

1. Сокращает накладные расходы на Python
2. Позволяет циклам работать с типизированными числами
3. Ускоряет выполнение кода на уровне C
4. Уменьшает количество операций с объектами Python

Подсказка: Каждое улучшение решает разные узкие места.

- **Использование cdef для переменных** — Позволяет циклам работать с типизированными числами
- **Массивы Cython вместо list** — Уменьшает количество операций с объектами Python
- **Компиляция через cythonize** — Ускоряет выполнение кода на уровне C
- **Минимизация Python-объектов в цикле** — Сокращает накладные расходы на Python

💡 Шаг 05:

Миссия 5

📄 Сопоставьте оптимизацию циклов с её результатом:

Понятия:

Ответ:

- Цикл for с cdef int
- Цикл while с PyObject
- Вложенный цикл с cdef double
- Суммирование через Python list

Эффекты:

1. Быстро выполняется с типизированными счётчиками
2. Медленно, много Python-объектов
3. Быстро для чисел с плавающей точкой
4. Медленно, накладные расходы Python

Подсказка: Типизация и минимизация Python-объектов ускоряет циклы.

- **Цикл for с cdef int** — Быстро выполняется с типизированными счётчиками
- **Цикл while с PyObject** — Медленно, много Python-объектов
- **Вложенный цикл с cdef double** — Быстро для чисел с плавающей точкой
- **Суммирование через Python list** — Медленно, накладные расходы Python

💡 Шаг 06:

Миссия 6

Игорь хочет ускорить вычисление среднего квадрата числового ряда. Что выбрать?

Подсказка: Подумайте о типах и цикле.

Ответ:

- **cdef double для переменных и массивы Cython**
- Python float в list
- Использовать строку для чисел
- Добавлять print() в цикле

Шаг 07:

Миссия 7

Мария хочет ускорить суммирование 1 миллиона чисел. Какой подход наиболее эффективен?

Подсказка: Подумайте о том, как Python обрабатывает объекты.

Ответ:

- **Использовать типизированный массив Cython и cdef int**
- Python list и for loop
- Суммирование через str
- Добавлять print на каждом шаге

Шаг 08:

Миссия 8

 Сопоставьте методы ускорения циклов с их преимуществами:

Ответ:

- Использование cdef int для счётчиков
 - Использование cdef double для чисел с плавающей точкой
 - Минимизация Python-объектов внутри цикла
 - Компиляция через cythonize
1. Быстро выполняется с типизированными целыми числами
 2. Быстро выполняется с числами с плавающей точкой
 3. Сокращает накладные расходы Python
 4. Код выполняется почти на уровне C

Подсказка: Каждое улучшение решает отдельную узкую проблему.

- **Использование cdef int для счётчиков** — Быстро выполняется с типизированными целыми числами
- **Использование cdef double для чисел с плавающей точкой** — Быстро выполняется с числами с плавающей точкой
- **Минимизация Python-объектов внутри цикла** — Сокращает накладные расходы Python
- **Компиляция через cythonize** — Код выполняется почти на уровне C

Шаг 09:

Миссия 9

Сергей хочет ускорить вычисление факториала большого числа с помощью рекурсии. Что поможет наибольшему ускорению?

Подсказка: Подумайте о типизации переменных и накладных расходах Python.

Ответ:

- **Типизировать аргументы и результат с cdef**
- Оставить Python int
- Писать print в каждой рекурсии
- Использовать str вместо числа

Шаг 10:

Миссия 10

Лена работает с суммой квадратичных рядов. Какой подход ускорит вычисления?

Подсказка: Подумайте о массивах и типах чисел.

Ответ:

- **Использовать массивы Cython с cdef double**
- Python list с float
- Преобразовать числа в строки
- Добавлять print() на каждом шаге

Шаг 11:

Миссия 11

 Сопоставьте тип данных с его особенностью в Cython:

Ответ:

- cdef int
 - cdef double
 - Python int
 - Python float
1. Быстрое выполнение целых чисел, низкие накладные расходы
 2. Быстрое выполнение чисел с плавающей точкой
 3. Медленно, накладные расходы Python
 4. Медленно, тип Python float

Подсказка: Cython ускоряет только cdef-типизированные переменные.

- **cdef int** — Быстрое выполнение целых чисел, низкие накладные расходы
- **cdef double** — Быстрое выполнение чисел с плавающей точкой
- **Python int** — Медленно, накладные расходы Python
- **Python float** — Медленно, тип Python float

Шаг 12:

Миссия 12

Дмитрий оптимизирует сложение элементов массива длиной 10 миллионов. Как ускорить обработку?

Подсказка: Минимизация Python-объектов критична.

Ответ:

- **Использовать cdef массив и цикл for**
- Суммировать через Python list и sum()
- Использовать строковый массив
- Добавлять print в каждом шаге цикла

Шаг 13:

Миссия 13

Наташа хочет ускорить рекурсивное вычисление чисел Фибоначчи. Что сделает её функцию быстрее?

Подсказка: Типизация и уменьшение Python-объектов помогают рекурсии.

Ответ:

- **cdef int аргументы и результат функции**
- Python int для аргументов

- Добавлять `print()` внутри рекурсии
- Использовать `float` вместо `int`

🔗 Шаг 14:

Миссия 14

Расположите шаги ускорения числового ряда в порядке выполнения:

1. Типизация переменных с `cdef`
2. Использование массивов Cython вместо Python list
3. Минимизация Python-объектов внутри цикла
4. Компиляция через `cythonize`

Подсказка: Сначала ускоряем вычисления, потом компилируем.

Ответ:

- Типизация переменных с `cdef`
- Использование массивов Cython вместо Python list
- Минимизация Python-объектов внутри цикла
- Компиляция через `cythonize`

🔗 Шаг 15:

Миссия 15

📄 Сопоставьте проблему с решением в ускорении числовых рядов:

Ответ:

- Медленное суммирование Python list
 - Рекурсия с Python int
 - Числа с плавающей точкой медленно
 - Частое создание Python-объектов
1. Использовать `cdef` массив и цикл
 2. Типизировать аргументы с `cdef int`
 3. Использовать `cdef double`
 4. Минимизировать Python-объекты

Подсказка: Каждая проблема требует своего подхода.

- **Медленное суммирование Python list** — Использовать `cdef` массив и цикл
- **Рекурсия с Python int** — Типизировать аргументы с `cdef int`
- **Числа с плавающей точкой медленно** — Использовать `cdef double`
- **Частое создание Python-объектов** — Минимизировать Python-объекты

🔗 Шаг 16:

Миссия 16

Алексей хочет ускорить суммирование больших рядов с циклами `for`. Что наиболее критично?

Подсказка: Минимизация Python-объектов и типизация ключевые для Cython.

Ответ:

- **Использовать `cdef` массив и типизированные переменные**
- Python list с `float`
- Писать `print()` внутри цикла

- Использовать строки вместо чисел

Модуль 4 - Алхимические инструменты Cython

Урок 1 - Векторизация работа с NumPy через Cython.

Шаг 01:

Миссия 1

Анна хочет ускорить умножение двух массивов NumPy. Что даст наибольший прирост скорости?
Подсказка: Старайтесь использовать преимущества Cython и векторизации.

Ответ:

- **Использовать typed memoryviews в Cython**
- Перебирать элементы через Python for
- Преобразовать массивы в списки Python
- Добавлять print() внутри цикла

Шаг 02:

Миссия 2

Иван работает с большим числовым массивом. Какой способ ускорит суммирование всех элементов?

Подсказка: Минимизируйте Python-объекты и используйте Cython типизацию.

Ответ:

- **Использовать cdef double[:] и цикл for**
- Python list и sum()
- Перебор через while с Python int
- Использовать строки вместо чисел

Шаг 03:

Миссия 3

 Сопоставьте термины и их описание в контексте Cython и NumPy:

Ответ:

- Typed memoryviews
 - cdef double
 - Python float
 - Vectorized operation
1. Типизированный доступ к массиву без накладных расходов Python
 2. Быстрые числа с плавающей точкой в Cython
 3. Медленно, накладные расходы Python
 4. Применение операций ко всему массиву сразу

Подсказка: Cython ускоряет только правильно типизированные объекты и операции.

- **Typed memoryviews** — Типизированный доступ к массиву без накладных расходов Python
- **cdef double** — Быстрые числа с плавающей точкой в Cython
- **Python float** — Медленно, накладные расходы Python
- **Vectorized operation** — Применение операций ко всему массиву сразу

🔦 Шаг 04:

Миссия 4

Мария хочет ускорить умножение матриц с NumPy через Cython. Что важно сделать?

Подсказка: Смотрите на типы и доступ к элементам массивов.

Ответ:

- **Использовать typed memoryviews и cdef для индексов**
- Перебирать Python list с float
- Писать print() внутри циклов
- Использовать str вместо чисел

🔦 Шаг 05:

Миссия 5

📖 Сопоставьте термины и преимущества:

Ответ:

- Vectorized addition
 - Typed memoryviews
 - Cython loops
 - NumPy broadcasting
1. Применяет операцию ко всему массиву быстро
 2. Позволяет работать с массивами как с C массивами
 3. Быстро выполняется при правильной типизации
 4. Автоматически расширяет массивы по размеру для операций

Подсказка: Подумайте, что ускоряет вычисления и что облегчает работу с размерами массивов.

- **Vectorized addition** — Применяет операцию ко всему массиву быстро
- **Typed memoryviews** — Позволяет работать с массивами как с C массивами
- **Cython loops** — Быстро выполняется при правильной типизации
- **NumPy broadcasting** — Автоматически расширяет массивы по размеру для операций

🔦 Шаг 06:

Миссия 6

Павел работает с двумерными массивами и хочет ускорить доступ к элементам. Что лучше сделать?

Подсказка: Типизация и прямой доступ через memoryview ускоряют операции.

Ответ:

- **Использовать typed memoryviews и cdef int для индексов**
- Перебирать Python list с float
- Добавлять print() внутри циклов
- Использовать str вместо чисел

🔦 Шаг 07:

Миссия 7

Расположите шаги ускорения операции над массивом NumPy через Cython в правильном порядке:

1. Типизировать переменные с cdef
2. Использовать typed memoryviews
3. Минимизировать Python-объекты внутри цикла
4. Компилировать через cythonize

Подсказка: Сначала ускоряем вычисления, потом компилируем код.

Ответ:

- Типизировать переменные с cdef
- Использовать typed memoryviews
- Минимизировать Python-объекты внутри цикла
- Компилировать через cythonize

Шаг 08:

Миссия 8

Екатерина хочет применять векторизованные функции к массивам больших размеров. Что важно помнить?

Подсказка: Используйте преимущества NumPy и Cython без Python overhead.

Ответ:

- **Применять операции через typed memoryviews и vectorized functions**
- Перебирать элементы через Python for
- Использовать списки Python
- Добавлять print() на каждом шаге

Шаг 09:

Миссия 9

Анна хочет ускорить умножение двух массивов NumPy. Что даст наибольший прирост скорости?

Подсказка: Старайтесь использовать преимущества Cython и векторизации.

Ответ:

- **Использовать typed memoryviews в Cython**
- Перебирать элементы через Python for
- Преобразовать массивы в списки Python
- Добавлять print() внутри цикла

Шаг 10:

Миссия 10

Иван работает с большим числовым массивом. Какой способ ускорит суммирование всех элементов?

Подсказка: Минимизируйте Python-объекты и используйте Cython типизацию.

Ответ:

- **Использовать cdef double[:] и цикл for**
- Python list и sum()
- Перебор через while с Python int
- Использовать строки вместо чисел

Шаг 11:

Миссия 11

 Сопоставьте термины и их описание в контексте Cython и NumPy:

Ответ:

- Typed memoryviews
- cdef double
- Python float
- Vectorized operation

1. Типизированный доступ к массиву без накладных расходов Python
2. Быстрые числа с плавающей точкой в Cython
3. Медленно, накладные расходы Python
4. Применение операций ко всему массиву сразу

Подсказка: Cython ускоряет только правильно типизированные объекты и операции.

- **Typed memoryviews** — Типизированный доступ к массиву без накладных расходов Python
- **cdef double** — Быстрые числа с плавающей точкой в Cython
- **Python float** — Медленно, накладные расходы Python
- **Vectorized operation** — Применение операций ко всему массиву сразу

Шаг 12:

Миссия 12

Мария хочет ускорить умножение матриц с NumPy через Cython. Что важно сделать?

Подсказка: Смотрите на типы и доступ к элементам массивов.

Ответ:

- **Использовать typed memoryviews и cdef для индексов**
- Перебирать Python list с float
- Писать print() внутри циклов
- Использовать str вместо чисел

Шаг 13:

Миссия 13

 Сопоставьте подходы ускорения с их характеристикой:

Ответ:

- Typed memoryviews
- cdef int
- NumPy broadcasting
- Python loop

1. Типизированный быстрый доступ к массиву
2. Объявление индексов и счетчиков для скорости
3. Операции над массивами без циклов Python
4. Медленный способ, Python интерпретирует каждый шаг

Подсказка: Сочетайте правильную типизацию с векторизацией для максимальной скорости.

- **Typed memoryviews** — Типизированный быстрый доступ к массиву
- **cdef int** — Объявление индексов и счетчиков для скорости

- **NumPy broadcasting** — Операции над массивами без циклов Python
- **Python loop** — Медленный способ, Python интерпретирует каждый шаг

🔮 Шаг 14:

Миссия 14

Какой способ наиболее эффективен для сложения двух больших массивов в Cython?

Подсказка: Минимизируйте Python overhead.

Ответ:

- **Векторизованная операция с typed memoryviews**
- Python for loop с float
- Использовать map() и lambda
- Списки Python и append()

🔮 Шаг 15:

Миссия 15

Расположите шаги оптимизации операций над массивами NumPy через Cython:

1. Использовать typed memoryviews для массивов
2. Объявить индексы и счетчики через cdef
3. Векторизовать операции через NumPy или Cython
4. Минимизировать Python объекты и overhead

Подсказка: Сначала типизация, потом индексы, затем векторизация и минимизация Python overhead.

Ответ:

- Использовать typed memoryviews для массивов
- Объявить индексы и счетчики через cdef
- Векторизовать операции через NumPy или Cython
- Минимизировать Python объекты и overhead

📖 Урок 2 - Встраивание C-функций в Cython.

🔮 Шаг 01:

Миссия 1

Алекс хочет вызвать C-функцию из Cython. Что нужно сделать первым шагом?

Подсказка: Подумайте, как объявить внешнюю функцию.

Ответ:

- **Использовать cdef extern для объявления функции**
- Просто вызвать функцию Python
- Объявить функцию как def в Cython без extern
- Записать функцию в строку и eval

🔮 Шаг 02:

Миссия 2

Какой тип данных стоит использовать для передачи чисел в C-функцию через Cython?

Подсказка: Минимизируйте накладные расходы Python.

Ответ:

- **cdef int или cdef double**
- Python int или float
- str
- list

🔮 **Шаг 03:**

Миссия 3

Сопоставьте понятия и их описание для встраивания C-функций:

Ответ:

- cdef extern
- cdef int
- Python call
- Include directive

1. Объявление внешней C-функции для вызова из Cython
2. Типизированная переменная для передачи в C
3. Медленный вызов Python функции
4. Подключение заголовочного файла C

Подсказка: Обратите внимание на разницу между C и Python контекстом.

- **cdef extern** — Объявление внешней C-функции для вызова из Cython
- **cdef int** — Типизированная переменная для передачи в C
- **Python call** — Медленный вызов Python функции
- **Include directive** — Подключение заголовочного файла C

🔮 **Шаг 04:**

Миссия 4

Екатерина хочет ускорить функцию вычисления факториала через C. Какой подход правильный?

Подсказка: Подумайте, как избежать Python overhead.

Ответ:

- **Написать рекурсивную C-функцию и вызвать её через cdef extern**
- Использовать def и Python рекурсию
- Вычислять через list comprehension
- Использовать eval для строки с формулой

🔮 **Шаг 05:**

Миссия 5

Сопоставьте понятия и их роль в Cython с C-функциями:

Ответ:

- cdef double
- cdef extern from
- Python float
- Header file

1. Типизированная переменная для скорости

2. Объявление внешней функции из С-файла
3. Медленный объект Python
4. Подключение С-файла для использования функций

Подсказка: Минимизируйте использование Python объектов для ускорения.

- **cdef double** — Типизированная переменная для скорости
- **cdef extern from** — Объявление внешней функции из С-файла
- **Python float** — Медленный объект Python
- **Header file** — Подключение С-файла для использования функций

Шаг 06:

Миссия 6

Какой способ передачи массивов в С-функцию через Cython наиболее быстрый?

Подсказка: Используйте нативные С-подходы.

Ответ:

- **Typed memoryviews и cdef массивы**
- Python list
- NumPy объекты без типизации
- Списки строк

Шаг 07:

Миссия 7

Расположите шаги встраивания С-функции в Cython:

1. Подключить заголовочный файл с помощью include
2. Объявить функцию через cdef extern from
3. Определить типизированные переменные для аргументов
4. Вызвать функцию из Cython-кода

Подсказка: Сначала подключение, потом объявление, затем подготовка переменных и вызов.

Ответ:

- Подключить заголовочный файл с помощью include
- Объявить функцию через cdef extern from
- Определить типизированные переменные для аргументов
- Вызвать функцию из Cython-кода

Шаг 08:

Миссия 8

Алексей хочет уменьшить накладные расходы при вызове функции для больших массивов. Что поможет ускорить обработку?

Подсказка: Минимизируйте использование Python объектов.

Ответ:

- **Использовать cdef массивы и typed memoryviews**
- Передавать Python list напрямую
- Применять map() с Python функцией
- Вычислять через eval строки с формулой

🔮 Шаг 09:

Миссия 9

Сопоставьте ключевые концепции Cython для C-функций и их роль:

Ответ:

- `cdef int`
 - `cdef extern from`
 - Python object
 - `Include`
1. Объявление типизированной переменной для скорости
 2. Подключение внешней функции из C
 3. Медленный объект Python
 4. Подключение заголовочного файла

Подсказка: Вспомните разницу между C и Python объектами.

- **`cdef int`** — Объявление типизированной переменной для скорости
- **`cdef extern from`** — Подключение внешней функции из C
- **Python object** — Медленный объект Python
- **`Include`** — Подключение заголовочного файла

🔮 Шаг 10:

Миссия 10

Какая стратегия ускоряет вызовы математических функций в Cython?

Подсказка: Python overhead добавляет замедление при каждой операции.

Ответ:

- **Использовать `cdef` C-функции вместо `def`**
- Вызывать встроенные Python функции через `def`
- Вычислять через `map` и `lambda`
- Использовать Python list comprehensions

🔮 Шаг 11:

Миссия 11

Расположите шаги для вызова внешней C-функции в правильном порядке:

1. Написать C-файл с функцией
2. Подключить заголовочный файл через `include`
3. Объявить функцию в Cython с `cdef extern from`
4. Вызвать функцию через `typed memoryview` или `cdef` переменные

Подсказка: Сначала создаём функцию в C, потом подключаем и вызываем её.

Ответ:

- Написать C-файл с функцией
- Подключить заголовочный файл через `include`
- Объявить функцию в Cython с `cdef extern from`
- Вызвать функцию через `typed memoryview` или `cdef` переменные

🔮 Шаг 12:

Миссия 12

Мария хочет оптимизировать вычисление суммы больших массивов чисел. Что эффективнее всего?

Подсказка: Минимизируйте Python объекты и циклы.

Ответ:

- **Использовать cdef функции и typed memoryviews**
- Использовать Python sum() на list
- Вычислять через for loop на Python list
- Создавать новые Python объекты на каждый шаг

🔮 Шаг 13:

Миссия 13

Сопоставьте инструменты Cython и их назначение:

Ответ:

- cdef float
 - cdef extern from "file.h"
 - Python list
 - Include
1. Типизированная переменная для скорости
 2. Подключение внешней C-функции
 3. Медленный Python объект
 4. Подключение заголовочного файла

Подсказка: Используйте типизацию для ускорения.

- **cdef float** — Типизированная переменная для скорости
- **cdef extern from "file.h"** — Подключение внешней C-функции
- **Python list** — Медленный Python объект
- **Include** — Подключение заголовочного файла

🔮 Шаг 14:

Миссия 14

Что ускорит выполнение функции при большом количестве повторных вызовов?

Подсказка: Python overhead растёт при каждом вызове def.

Ответ:

- **Переписать функцию через cdef и вызвать напрямую**
- Вызывать def функцию через for loop
- Использовать Python lambda
- Использовать eval внутри цикла

🔮 Шаг 15:

Миссия 15

Расположите этапы интеграции C-функции в Cython:

1. Создать C-файл с функцией

2. Добавить заголовочный файл через include
3. Объявить extern функцию в Cython
4. Использовать cdef переменные для передачи данных
5. Вызвать функцию внутри Cython кода

Подсказка: Сначала создаём и подключаем, потом типизируем и вызываем.

Ответ:

- Создать C-файл с функцией
- Добавить заголовочный файл через include
- Объявить extern функцию в Cython
- Использовать cdef переменные для передачи данных
- Вызвать функцию внутри Cython кода

Шаг 16:

Миссия 16

Сопоставьте термины Cython с их эффектом на производительность:

Ответ:

- cdef int array
 - Python list
 - cdef extern from "lib.h"
 - Include
1. Быстрая типизированная переменная
 2. Медленный Python объект
 3. Использование C-функции напрямую
 4. Подключение заголовочного файла для функций

Подсказка: Используйте cdef и extern для ускорения.

- **cdef int array** — Быстрая типизированная переменная
- **Python list** — Медленный Python объект
- **cdef extern from "lib.h"** — Использование C-функции напрямую
- **Include** — Подключение заголовочного файла для функций

Урок 3 - Работа с памятью указатели и массивы.

Шаг 01:

Миссия 1

Алексей хочет работать с массивами чисел максимально быстро. Что поможет ускорить вычисления?

Подсказка: Минимизируйте Python объекты и используйте прямой доступ к памяти.

Ответ:

- **Использовать cdef массивы и typed memoryviews**
- Использовать обычный Python list
- Вычислять через map() и lambda
- Создавать новые Python объекты для каждого элемента

🔮 Шаг 02:

Миссия 2

Мария хочет передавать массив чисел из Python в C-функцию через Cython. Что правильнее всего использовать?

Подсказка: Typed memoryview позволяет безопасно работать с массивами в C.

Ответ:

- **Typed memoryview**
- Python list
- dict
- set

🔮 Шаг 03:

Миссия 3

Сопоставьте термины Cython с их назначением:

Ответ:

- `cdef int*`
- `cdef double[:]`
- Python object
- memoryview

1. Указатель на целые числа в C
2. Типизированный массив с прямым доступом к памяти
3. Медленный Python объект
4. Интерфейс для безопасного доступа к массивам

Подсказка: Указатели дают прямой доступ, typed memoryview — безопасный вариант.

- **`cdef int*`** — Указатель на целые числа в C
- **`cdef double[:]`** — Типизированный массив с прямым доступом к памяти
- **Python object** — Медленный Python объект
- **memoryview** — Интерфейс для безопасного доступа к массивам

🔮 Шаг 04:

Миссия 4

Какой способ выделения памяти для массива в Cython безопаснее всего?

Подсказка: Используйте автоматическое управление памятью, чтобы избежать утечек.

Ответ:

- **Использовать typed memoryview или numpy массивы**
- Выделять через malloc и забыть free
- Создавать Python list и передавать его в C
- Использовать глобальные переменные для хранения массива

🔮 Шаг 05:

Миссия 5

Сопоставьте термины по памяти и их описание:

Ответ:

- Stack memory
 - Heap memory
 - Pointer
 - Memoryview
1. Автоматически выделяемая память для локальных переменных
 2. Динамически выделяемая память через malloc или numpy
 3. Переменная, хранящая адрес в памяти
 4. Безопасный доступ к массивам Python и C

Подсказка: Stack и heap отличаются временем жизни и управлением.

- **Stack memory** — Автоматически выделяемая память для локальных переменных
- **Heap memory** — Динамически выделяемая память через malloc или numpy
- **Pointer** — Переменная, хранящая адрес в памяти
- **Memoryview** — Безопасный доступ к массивам Python и C

 Шаг 06:

Миссия 6

Что помогает избежать ошибок при передаче массивов в C-функции?

Подсказка: Typed memoryviews проверяют размерность и тип данных.

Ответ:

- **Typed memoryview с указанием типа и размерности**
- Передавать Python list без проверки
- Использовать глобальные переменные
- Конвертировать массив в строку

 Шаг 07:

Миссия 7

Расположите шаги безопасной работы с массивами в Cython:

1. Создать typed memoryview для массива
2. Заполнить массив значениями
3. Передать массив в cdef функцию
4. Выполнить вычисления с прямым доступом к памяти
5. Вернуть результаты в Python

Подсказка: Сначала типизируем, затем используем и возвращаем.

Ответ:

- Создать typed memoryview для массива
- Заполнить массив значениями
- Передать массив в cdef функцию
- Выполнить вычисления с прямым доступом к памяти
- Вернуть результаты в Python

 Шаг 08:

Миссия 8

Какой метод работы с массивами быстрее всего при больших объёмах данных?

Подсказка: Используйте C-подобный доступ и избегайте Python циклов.

Ответ:

- **Использовать cdef массивы и указатели**
- Использовать Python list comprehension
- Вычислять через map() и lambda
- Использовать генераторы Python

🔮 Шаг 09:

Миссия 9

Сопоставьте операции с массивами и их характеристики:

Ответ:

- cdef int[:]
 - cdef int*
 - Python list
 - memoryview
1. Типизированный массив для быстрого доступа
 2. Указатель на элементы в памяти
 3. Медленный, гибкий Python объект
 4. Безопасный интерфейс к массивам

Подсказка: Подумайте о скорости и безопасности доступа.

- **cdef int[:]** — Типизированный массив для быстрого доступа
- **cdef int*** — Указатель на элементы в памяти
- **Python list** — Медленный, гибкий Python объект
- **memoryview** — Безопасный интерфейс к массивам

🔮 Шаг 10:

Миссия 10

Что помогает избежать ошибок при передаче массивов в C-функции?

Подсказка: Typed memoryviews проверяют размерность и тип данных.

Ответ:

- **Typed memoryview с указанием типа и размерности**
- Передавать Python list без проверки
- Использовать глобальные переменные
- Конвертировать массив в строку

🔮 Шаг 11:

Миссия 11

Расположите шаги безопасной работы с массивами в Cython:

1. Создать typed memoryview для массива
2. Заполнить массив значениями
3. Передать массив в cdef функцию
4. Выполнить вычисления с прямым доступом к памяти
5. Вернуть результаты в Python

Подсказка: Сначала типизируем, затем используем и возвращаем.

Ответ:

- Создать typed memoryview для массива
- Заполнить массив значениями
- Передать массив в cdef функцию
- Выполнить вычисления с прямым доступом к памяти
- Вернуть результаты в Python

🔮 Шаг 12:

Миссия 12

Какой метод работы с массивами быстрее всего при больших объёмах данных?

Подсказка: Используйте C-подобный доступ и избегайте Python циклов.

Ответ:

- **Использовать cdef массивы и указатели**
- Использовать Python list comprehension
- Вычислять через map() и lambda
- Использовать генераторы Python

🔮 Шаг 13:

Миссия 13

Сопоставьте операции с массивами и их характеристики:

Ответ:

- cdef int[:]
 - cdef int*
 - Python list
 - memoryview
1. Типизированный массив для быстрого доступа
 2. Указатель на элементы в памяти
 3. Медленный, гибкий Python объект
 4. Безопасный интерфейс к массивам

Подсказка: Подумайте о скорости и безопасности доступа.

- **cdef int[:]** — Типизированный массив для быстрого доступа
- **cdef int*** — Указатель на элементы в памяти
- **Python list** — Медленный, гибкий Python объект
- **memoryview** — Безопасный интерфейс к массивам

🔮 Шаг 14:

Миссия 14

Что произойдет, если использовать malloc без free в Cython?

Подсказка: Любая память, не освобождённая вручную, остается занятой.

Ответ:

- **Возникнет утечка памяти**
- Python автоматически очистит память
- Ничего страшного не произойдет

- Программа перестанет компилироваться

Шаг 15:

Миссия 15

Расположите шаги работы с указателями в правильном порядке:

1. Объявить `cdef` указатель
2. Выделить память для элементов
3. Инициализировать значения массива
4. Передать указатель в функцию
5. Освободить память

Подсказка: Сначала создаем указатель и память, затем используем, в конце освобождаем.

Ответ:

- Объявить `cdef` указатель
- Выделить память для элементов
- Инициализировать значения массива
- Передать указатель в функцию
- Освободить память

Урок 4 - Практикум написать быструю матричную операцию.

Шаг 01:

Миссия 1

Какой способ ускоряет умножение больших матриц в Cython?

Подсказка: Используйте `cdef` функции и `typed memoryviews` для прямого доступа к элементам.

Ответ:

- **Использовать `cdef` функции и `typed memoryviews`**
- Использовать Python list comprehension
- Вычислять через `map()` и `lambda`
- Использовать обычный Python цикл без типизации

Шаг 02:

Миссия 2

Сопоставьте методы работы с матрицами и их особенности:

Ответ:

- Python list
 - NumPy array
 - Typed memoryview
 - `cdef` массив
1. Медленный, но гибкий и знакомый Python объект
 2. Быстрое векторное вычисление, готовый интерфейс
 3. Безопасный доступ к элементам с типизацией
 4. Прямой доступ к памяти для максимальной скорости

Подсказка: Подумайте о скорости и безопасности доступа к элементам.

- **Python list** — Медленный, но гибкий и знакомый Python объект

- **NumPy array** — Быстрое векторное вычисление, готовый интерфейс
- **Typed memoryview** — Безопасный доступ к элементам с типизацией
- **cdef массив** — Прямой доступ к памяти для максимальной скорости

🔧 Шаг 03:

Миссия 3

Что является преимуществом typed memoryviews при работе с матрицами?

Подсказка: Подумайте о безопасности и скорости доступа к элементам.

Ответ:

- **Безопасный и быстрый доступ к элементам с проверкой типов**
- Автоматическое умножение матриц без кода
- Гибкость без типизации
- Использование только Python list

🔧 Шаг 04:

Миссия 4

Расположите шаги реализации умножения матриц через Cython в правильном порядке:

1. Определить cdef функцию для умножения
2. Создать typed memoryviews для обеих матриц
3. Пройтись по элементам с помощью cdef циклов
4. Вычислить произведение и записать в результат
5. Вернуть результат в Python

Подсказка: Сначала типизируем, затем используем циклы и возвращаем результат.

Ответ:

- Определить cdef функцию для умножения
- Создать typed memoryviews для обеих матриц
- Пройтись по элементам с помощью cdef циклов
- Вычислить произведение и записать в результат
- Вернуть результат в Python

🔧 Шаг 05:

Миссия 5

Почему использование cdef циклов быстрее обычных Python циклов при работе с матрицами?

Подсказка: cdef циклы компилируются в C и работают напрямую с памятью.

Ответ:

- **Потому что они компилируются в C и обходят интерпретатор Python**
- Потому что они используют NumPy функции
- Потому что они используют Python list comprehension
- Потому что они автоматически распараллеливаются

🔧 Шаг 06:

Миссия 6

Сопоставьте типы матриц и их характеристики:

Ответ:

- Python list
- NumPy array
- Typed memoryview
- cdef массив

1. Гибкий, но медленный
2. Оптимизирован для векторных операций
3. Типизированный доступ с безопасностью
4. Прямой доступ к памяти, высокая скорость

Подсказка: Подумайте о сочетании скорости и безопасности.

- **Python list** — Гибкий, но медленный
- **NumPy array** — Оптимизирован для векторных операций
- **Typed memoryview** — Типизированный доступ с безопасностью
- **cdef массив** — Прямой доступ к памяти, высокая скорость

Шаг 07:

Миссия 7

Алиса хочет ускорить умножение матриц в Python с помощью Cython. Какой подход будет самым простым для начала?

Подсказка: Начните с того, что легко интегрируется с Python.

Ответ:

- **Использовать типизацию переменных и функции cdef в Cython**
- Переписать весь Python-код на Rust
- Использовать только стандартный Python без Cython
- Запустить Python на GPU без изменений кода

Шаг 08:

Миссия 8

Боб заметил, что его матричная операция в Python очень медленная. Что ему следует сделать в первую очередь?

Подсказка: Иногда проще ускорить узкие места, чем переписывать всё.

Ответ:

- **Профилировать код и найти горячие точки**
- Сразу переписать весь код на C
- Добавить больше циклов в Python
- Игнорировать проблему и запускать как есть

Шаг 09:

Миссия 9

Карина хочет ускорить функцию умножения матриц через Cython. Что ей поможет сильнее всего?

Подсказка: Ускорение происходит, когда Cython знает типы данных заранее.

Ответ:

- **Использование cdef для переменных и функций**

- Оставить все переменные как Python-объекты
- Добавить больше print для отладки
- Применять только list вместо массивов NumPy

🔗 Шаг 10:

Миссия 10

Даниил хочет ускорить сложение больших матриц. Какой метод даст наибольший прирост производительности?

Подсказка: Чистый Python медленный для больших чисел итераций.

Ответ:

- **Перевести вложенные циклы на Cython с cdef типами**
- Использовать обычные списки Python и циклы
- Добавить больше функций Python без типизации
- Просто увеличить размер кэша компьютера

🔗 Шаг 11:

Миссия 11

Эмилия изучает ускорение матричных операций и хочет минимизировать обращение к Python-объектам внутри цикла. Что сделать?

Подсказка: Чем меньше Python-объектов внутри цикла, тем быстрее Cython работает.

Ответ:

- **Использовать массивы NumPy с типами C и cdef**
- Оставить списки Python внутри цикла
- Вызывать Python-функции для каждого элемента
- Не использовать массивы, работать с числами напрямую

🔗 Шаг 12:

Миссия 12

📖 Сопоставьте термин из Cython с его назначением:

Термины:

Ответ:

- cdef
- cpdef
- def
- nogil

Назначение:

1. Объявление функции или переменной с C-типом
2. Функция доступна из Python и C
3. Обычная Python-функция
4. Позволяет выполнять код без GIL для параллельных вычислений

🔗 Подсказка:

Подумайте, какие ключевые слова ускоряют выполнение и какие оставляют доступ из Python.

- **cdef** — Объявление функции или переменной с C-типом
- **cpdef** — Функция доступна из Python и C

- **def** — Обычная Python-функция
- **nogil** — Позволяет выполнять код без GIL для параллельных вычислений

Шаг 13:

Миссия 13

 Сопоставьте преимущества Cython с описанием:

Преимущества:

Ответ:

- Скорость выполнения
- Интеграция с Python
- Статическая типизация
- Параллельные вычисления

Описание:

1. Позволяет ускорить код, преобразуя Python в C
2. Может быть вызван из Python без сложных оберток
3. Позволяет явно задавать типы для переменных и функций
4. Использование `nogil` позволяет вычислять в несколько потоков

Подсказка:

Обратите внимание, что каждое свойство Cython имеет конкретную цель.

- **Скорость выполнения** — Позволяет ускорить код, преобразуя Python в C
- **Интеграция с Python** — Может быть вызван из Python без сложных оберток
- **Статическая типизация** — Позволяет явно задавать типы для переменных и функций
- **Параллельные вычисления** — Использование `nogil` позволяет вычислять в несколько потоков

Шаг 14:

Миссия 14

 Сопоставьте типы массивов с их использованием в ускорении Python:

Типы:

Ответ:

- NumPy `ndarray`
- Typed `memoryview`
- Python `list`
- C `array` (cdef массив)

Использование:

1. Позволяет работать с C-типами и циклическими вычислениями без Python-объектов
2. Основной тип Python, медленный для больших вычислений
3. Предоставляет быстрый доступ к массивам в Cython с проверкой типов
4. Стандартный массив NumPy для удобной работы с Python

Подсказка:

Подумайте, какой массив даёт прямой доступ к C-памяти и ускоряет циклы.

- **NumPy `ndarray`** — Стандартный массив NumPy для удобной работы с Python

- **Typed memoryview** — Предоставляет быстрый доступ к массивам в Cython с проверкой типов
- **Python list** — Основной тип Python, медленный для больших вычислений
- **C array (cdef массив)** — Позволяет работать с C-типами и циклическими вычислениями без Python-объектов

🔮 Шаг 15:

Миссия 15

📖 Сопоставьте типы функций Cython с их особенностями:

Типы:

Ответ:

- cdef функция
- cpdef функция
- Python def функция
- inline функция

Особенности:

1. Можно вызывать только из Cython, быстрее всего
2. Можно вызывать из Python и C, компромисс между доступностью и скоростью
3. Доступна только из Python, медленнее всех
4. Рекомендуется для коротких функций, вставляется в код, ускоряя выполнение

🔮 Подсказка:

Обратите внимание, какие функции доступны из Python и какие дают прирост скорости.

- **cdef функция** — Можно вызывать только из Cython, быстрее всего
- **cpdef функция** — Можно вызывать из Python и C, компромисс между доступностью и скоростью
- **Python def функция** — Доступна только из Python, медленнее всех
- **inline функция** — Рекомендуется для коротких функций, вставляется в код, ускоряя выполнение

🔮 Шаг 16:

Миссия 16

📖 Сопоставьте стратегии оптимизации циклов с их описанием:

Стратегии:

Ответ:

- Использовать cdef переменные
- Минимизировать Python-объекты в цикле
- Использовать typed memoryview
- Удалить лишние вызовы функций

Описание:

1. Объявление переменных с C-типом ускоряет арифметику
2. **Меньше обращения к Python-объектам** — быстрее выполнение
3. Быстрый доступ к элементам массивов с проверкой типа
4. Каждый вызов функции в цикле добавляет накладные расходы

🔮 Подсказка:

Подумайте, какие операции замедляют циклы больше всего.

- **Использовать cdef переменные** — Объявление переменных с C-типом ускоряет арифметику
- **Минимизировать Python-объекты в цикле** — Меньше обращения к Python-объектам — быстрее выполнение
- **Использовать typed memoryview** — Быстрый доступ к элементам массивов с проверкой типа
- **Удалить лишние вызовы функций** — Каждый вызов функции в цикле добавляет накладные расходы

Шаг 17:

Миссия 17

Расположите шаги оптимизации матричной операции в правильном порядке:

1. Переписать все функции на C
2. Профилировать код и найти узкие места
3. Использовать cdef и typed memoryview там, где возможно
4. Тестировать, что результаты совпадают с Python-версией

Подсказка:

Сначала нужно узнать, где узкое место, потом ускорять именно его, а не переписывать всё сразу.

Ответ:

- Профилировать код и найти узкие места
- Использовать cdef и typed memoryview там, где возможно
- Тестировать, что результаты совпадают с Python-версией
- Переписать все функции на C

Модуль 5 - Артефакты Rust — клинок скорости 🦀

Урок 1 - Основы Rust владение, заимствование, безопасность памяти.

Шаг 01:

Миссия 1

Алекс изучает Rust и хочет понять, что такое владение (ownership). Какой вариант объясняет владение?

Подсказка: Подумайте, кто отвечает за освобождение памяти.

Ответ:

- **Каждый объект в Rust имеет единственного владельца, который отвечает за его уничтожение**
- Любой объект может иметь нескольких владельцев без ограничений
- Память управляется вручную программистом, как в C
- Владение отсутствует, Rust использует сборщик мусора

Шаг 02:

Миссия 2

Мария хочет передать значение функции в Rust, чтобы не терять доступ к оригиналу. Какой способ передачи лучше всего подходит?

Подсказка: Подумайте о ссылках и их типах.

Ответ:

- **Передавать значение по ссылке (&) или по изменяемой ссылке (&mut)**
- Передавать значение через клон только
- Всегда передавать по значению, теряя оригинал
- Использовать указатели, как в C, без ссылок Rust

Шаг 03:

Миссия 3

Иван экспериментирует с Rust и пытается изменить значение через неизменяемую ссылку. Что произойдет?

Подсказка: Rust защищает данные от неожиданных изменений.

Ответ:

- **Компилятор выдаст ошибку, изменение через & невозможно**
- Изменение пройдет успешно без ошибок
- Значение скопируется автоматически и изменится локально
- Rust использует сборщик мусора и проигнорирует изменения

Шаг 04:

Миссия 4

Олег хочет передать массив в функцию и разрешить её изменять элементы. Какой способ передачи подойдет?

Подсказка: Для изменения нужен специальный тип ссылки.

Ответ:

- **Использовать изменяемую ссылку (&mut)**
- Использовать обычную ссылку (&)
- Передать через клон (clone)
- Передать по значению, не используя ссылки

Шаг 05:

Миссия 5

Светлана изучает правила заимствования в Rust. Что верно о нескольких ссылках на один объект?

Подсказка: Сколько изменяемых ссылок разрешено одновременно?

Ответ:

- **Можно иметь несколько неизменяемых ссылок или одну изменяемую ссылку**
- Можно иметь несколько изменяемых ссылок одновременно
- Нельзя иметь одновременно ссылки и владение
- Rust не ограничивает количество ссылок

Шаг 06:

Миссия 6

 Сопоставьте термин Rust с его описанием:

Термины:

Ответ:

- Владение (Ownership)
- Заимствование (Borrowing)
- Изменяемая ссылка (&mut)
- Неизменяемая ссылка (&)

Описание:

1. Каждый объект имеет одного владельца, который отвечает за очистку памяти
2. Временное использование объекта без передачи владения
3. Ссылка, которая позволяет изменять данные
4. Ссылка, которая только читает данные

 Подсказка:

Обратите внимание, кто может изменять данные и кто контролирует их жизнь.

- **Владение (Ownership)** — Каждый объект имеет одного владельца, который отвечает за очистку памяти
- **Заимствование (Borrowing)** — Временное использование объекта без передачи владения
- **Изменяемая ссылка (&mut)** — Ссылка, которая позволяет изменять данные
- **Неизменяемая ссылка (&)** — Ссылка, которая только читает данные

 Шаг 07:

Миссия 7

 Сопоставьте ошибки компиляции с причиной:

Ошибки:

Ответ:

- Две изменяемые ссылки на один объект
- Изменение через неизменяемую ссылку
- Использование ссылки после окончания её жизни
- Попытка взять владение у объекта с активными ссылками

Причина:

1. Правило заимствования Rust запрещает это для безопасности
2. Нарушение правила неизменяемости
3. Ссылка становится висячей после окончания области
4. Владение не может быть передано, пока объект используется

 Подсказка:

Думайте о безопасности памяти и правилах Rust для ссылок и владения.

- **Две изменяемые ссылки на один объект** — Правило заимствования Rust запрещает это для безопасности
- **Изменение через неизменяемую ссылку** — Нарушение правила неизменяемости
- **Использование ссылки после окончания её жизни** — Ссылка становится висячей после окончания области
- **Попытка взять владение у объекта с активными ссылками** — Владение не может быть передано, пока объект используется

Шаг 08:

Миссия 8

 Сопоставьте типы ссылок с примером:

Типы:

Ответ:

- Неизменяемая ссылка
- Изменяемая ссылка
- Владелец объекта
- Висячая ссылка

Пример:

1. Чтение массива в функции без изменения
2. Изменение элемента массива внутри функции
3. Переменная, которая уничтожает данные при выходе из области
4. Ссылка на объект, который уже уничтожен

 Подсказка:

Подумайте, что безопасно использовать для чтения и записи и что Rust запрещает.

- **Неизменяемая ссылка** — Чтение массива в функции без изменения
- **Изменяемая ссылка** — Изменение элемента массива внутри функции
- **Владелец объекта** — Переменная, которая уничтожает данные при выходе из области
- **Висячая ссылка** — Ссылка на объект, который уже уничтожен

Шаг 09:

Миссия 9

 Сопоставьте ключевые слова Rust с их назначением:

Ключевые слова:

Ответ:

- let
- mut
- ref
- drop

Назначение:

1. Создание переменной
2. Разрешение изменения переменной
3. Создание ссылки на значение
4. Явное уничтожение значения

 Подсказка:

Обратите внимание на контроль над изменяемостью и жизнью объекта.

- **let** — Создание переменной
- **mut** — Разрешение изменения переменной
- **ref** — Создание ссылки на значение
- **drop** — Явное уничтожение значения

🔗 Шаг 10:

Миссия 10

📖 Сопоставьте принципы безопасности памяти Rust с объяснением:

Принципы:

Ответ:

- Владение
- Заимствование
- Неизменяемость
- Отсутствие висячих ссылок

Объяснение:

1. Каждый объект имеет одного владельца
2. Временное использование без передачи владения
3. Нельзя изменять через неизменяемые ссылки
4. Ссылка не может существовать после уничтожения объекта

🔗 **Подсказка:**

Все эти правила помогают избежать сегфолтов и неопределенного поведения.

- **Владение** — Каждый объект имеет одного владельца
- **Заимствование** — Временное использование без передачи владения
- **Неизменяемость** — Нельзя изменять через неизменяемые ссылки
- **Отсутствие висячих ссылок** — Ссылка не может существовать после уничтожения объекта

🔗 Шаг 11:

Миссия 11

Расположите шаги безопасной передачи объекта в функцию Rust:

1. Определяем, нужен ли объект только для чтения или изменения
2. Выбираем передачу по ссылке или владение
3. Передаем объект или ссылку в функцию
4. Функция выполняет операции, соблюдая правила заимствования
5. Возвращаем значение, если нужно, без нарушения владения

🔗 **Подсказка:**

Сначала определяем цель передачи, потом выбираем способ, затем выполняем безопасно.

Ответ:

- Определяем, нужен ли объект только для чтения или изменения
- Выбираем передачу по ссылке или владение
- Передаем объект или ссылку в функцию
- Функция выполняет операции, соблюдая правила заимствования
- Возвращаем значение, если нужно, без нарушения владения

🔗 Шаг 12:

Миссия 12

Расположите этапы безопасного заимствования в Rust:

1. Создаем ссылку (& или &mut)

2. Используем ссылку внутри области видимости
3. Никакая другая изменяемая ссылка не создается одновременно
4. После выхода области ссылка больше не используется

🔗 Подсказка:

Сначала создаем ссылку, затем используем, соблюдая правила заимствования.

Ответ:

- Создаем ссылку (& или &mut)
- Используем ссылку внутри области видимости
- Никакая другая изменяемая ссылка не создается одновременно
- После выхода области ссылка больше не используется

🔗 Шаг 13:

Миссия 13

Расположите шаги корректного использования изменяемой ссылки:

1. Объявляем переменную с `let mut`
2. Создаем изменяемую ссылку (&mut)
3. Вносим изменения через ссылку внутри области видимости
4. После выхода из области ссылка больше не используется

🔗 Подсказка:

Сначала делаем переменную изменяемой, потом создаем ссылку и изменяем данные безопасно.

Ответ:

- Объявляем переменную с `let mut`
- Создаем изменяемую ссылку (&mut)
- Вносим изменения через ссылку внутри области видимости
- После выхода из области ссылка больше не используется

🔗 Шаг 14:

Миссия 14

Расположите шаги безопасного клонирования объекта в Rust:

1. Определяем, нужен ли отдельный объект или можно использовать ссылку
2. Вызываем метод `clone`, если нужен отдельный объект
3. Используем клонированный объект независимо от оригинала
4. Оригинал остаётся доступен без нарушений владения

🔗 Подсказка:

Клон создаёт независимый объект, но оригинал остаётся под контролем владельца.

Ответ:

- Определяем, нужен ли отдельный объект или можно использовать ссылку
- Вызываем метод `clone`, если нужен отдельный объект
- Используем клонированный объект независимо от оригинала
- Оригинал остаётся доступен без нарушений владения

🔗 Шаг 15:

Миссия 15

Расположите шаги безопасного возврата объекта из функции:

1. Определяем, кто будет владельцем возвращаемого объекта
2. Передаем объект в return, соблюдая владение
3. Получатель получает владение без ошибок компиляции
4. Оригинальный объект в функции больше не используется

 Подсказка:

Возврат объекта передаёт владение безопасно, Rust гарантирует отсутствие висячих ссылок.

Ответ:

- Определяем, кто будет владельцем возвращаемого объекта
- Передаем объект в return, соблюдая владение
- Получатель получает владение без ошибок компиляции
- Оригинальный объект в функции больше не используется

 Шаг 16:

Миссия 16

Расположите шаги работы с временными значениями (temporaries) в Rust:

1. Создаем временное значение внутри выражения
2. Используем ссылку на временное значение только в пределах выражения
3. Не сохраняем ссылку на временное значение после завершения выражения
4. Компилятор проверяет, что нет висячих ссылок

 Подсказка:

Rust запрещает сохранять ссылки на временные объекты за пределами их жизни.

Ответ:

- Создаем временное значение внутри выражения
- Используем ссылку на временное значение только в пределах выражения
- Не сохраняем ссылку на временное значение после завершения выражения
- Компилятор проверяет, что нет висячих ссылок

 Шаг 17:

Миссия 17

Расположите шаги работы с структурой данных в Rust с соблюдением владения:

1. Создаем структуру с полями, владение которых нужно отслеживать
2. Используем методы для безопасного доступа к полям через ссылки
3. Передаем структуру или ссылки в функции, соблюдая правила заимствования
4. После выхода области все ссылки становятся недействительными, структура безопасно уничтожается

 Подсказка:

Следите за тем, кто владеет данными и где используются ссылки.

Ответ:

- Создаем структуру с полями, владение которых нужно отслеживать
- Используем методы для безопасного доступа к полям через ссылки
- Передаем структуру или ссылки в функции, соблюдая правила заимствования

- После выхода области все ссылки становятся недействительными, структура безопасно уничтожается

Урок 2 - Подключение к Python библиотека PyO3.

Шаг 01:

Миссия 1

Алекс хочет вызвать Rust-функцию из Python через PyO3. Что нужно сделать в первую очередь?

Подсказка: Подумайте, что PyO3 предоставляет для взаимодействия между языками.

Ответ:

- **Пометить функцию атрибутом `#[pyfunction]`**
- Сразу импортировать модуль Rust в Python без подготовки
- Собирать Rust как обычный бинарник
- Использовать `ctypes` для вызова Rust без PyO3

Шаг 02:

Миссия 2

Мария создает модуль Rust для Python с PyO3. Какой атрибут нужно использовать для объявления модуля?

Подсказка: Подумайте, как Python узнает о существовании модуля.

Ответ:

- **`#[pymodule]`**
- `#[pyfunction]`
- `#[pyclass]`
- `#[pybind]`

Шаг 03:

Миссия 3

Иван хочет сделать структуру Rust доступной в Python. Какой атрибут нужно использовать?

Подсказка: Подумайте о том, как PyO3 связывает Rust-структуры с Python-объектами.

Ответ:

- **`#[pyclass]`**
- `#[pymodule]`
- `#[pyfunction]`
- `#[pybind]`

Шаг 04:

Миссия 4

Светлана хочет, чтобы метод Rust был доступен в Python через объект. Какой атрибут применить к методу?

Подсказка: Подумайте, какой атрибут позволяет методам класса быть видимыми в Python.

Ответ:

- **`#[pymethods]` внутри `#[pyclass]`**
- `#[pyfunction]` на уровне модуля
- `#[pymodule]`

- #[pybind]

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте элементы PyO3 с их назначением:

Элементы:

Ответ:

- #[pyfunction]
- #[pyclass]
- #[pymodule]
- #[pymethods]

Назначение:

1. Позволяет вызвать Rust-функцию из Python
2. Определяет структуру, доступную как Python-объект
3. Определяет модуль Python на Rust
4. Добавляет методы к pyclass для использования в Python

🔮 Подсказка:

Подумайте, какой атрибут для функции, какой для структуры и для модуля.

- **#[pyfunction]** — Позволяет вызвать Rust-функцию из Python
- **#[pyclass]** — Определяет структуру, доступную как Python-объект
- **#[pymodule]** — Определяет модуль Python на Rust
- **#[pymethods]** — Добавляет методы к pyclass для использования в Python

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте объекты PyO3 с их примерами:

Объекты:

Ответ:

- pyfunction
- pyclass
- pymodule
- pymethods

Примеры:

1. fn add(a: i32, b: i32) -> i32
2. struct Point { x: f64, y: f64 }
3. mod math
4. impl Point { fn distance(&self, other: &Point) -> f64 }

🔮 Подсказка:

Смотрите на функции, структуры, модули и методы внутри структур.

- **pyfunction** — fn add(a: i32, b: i32) -> i32
- **pyclass** — struct Point { x: f64, y: f64 }
- **pymodule** — mod math
- **pymethods** — impl Point { fn distance(&self, other: &Point) -> f64 }

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте типы Python-совместимых данных в PyO3 с Rust-типами:

Типы:

Ответ:

- Py String
- Py Int
- Py List
- Py Dict

Rust-тип:

1. String
2. i32 / i64
3. Vec
4. HashMap

🔧 Подсказка:

Подумайте, какие Rust-типы соответствуют Python-типам.

- **PyString** — String
- **PyInt** — i32 / i64
- **PyList** — Vec
- **PyDict** — HashMap

🔧 Шаг 08:

Миссия 8

Расположите шаги создания Rust-модуля для Python через PyO3:

1. Добавляем зависимость PyO3 в Cargo.toml
2. Создаем функцию с #[pyfunction]
3. Создаем модуль с #[pymodule]
4. Собираем проект с maturin или setuptools-rust
5. Импортируем модуль в Python

🔧 **Подсказка:** Начинаем с настройки проекта, затем пишем код, потом собираем и используем.

Ответ:

- Добавляем зависимость PyO3 в Cargo.toml
- Создаем функцию с #[pyfunction]
- Создаем модуль с #[pymodule]
- Собираем проект с maturin или setuptools-rust
- Импортируем модуль в Python

🔧 Шаг 09:

Миссия 9

Расположите шаги создания Rust-модуля для Python через PyO3:

1. Добавляем зависимость PyO3 в Cargo.toml
2. Создаем функцию с #[pyfunction]

3. Создаем модуль с `#[pymodule]`
4. Собираем проект с `maturin` или `setuptools-rust`
5. Импортируем модуль в Python

 Подсказка:

Начинаем с настройки проекта, затем пишем код, потом собираем и используем.

Ответ:

- Добавляем зависимость PyO3 в `Cargo.toml`
- Создаем функцию с `#[pyfunction]`
- Создаем модуль с `#[pymodule]`
- Собираем проект с `maturin` или `setuptools-rust`
- Импортируем модуль в Python

 Шаг 10:

Миссия 10

Расположите шаги безопасной передачи Python-объекта в Rust:

1. Получаем объект Python (`PyObject`)
2. Сохраняем объект в переменной с GIL
3. Используем Python объект через `PyAny` или `PyRef`
4. Возвращаем результат обратно в Python

 Подсказка:

Обратите внимание на необходимость владения и GIL при работе с Python-объектами.

Ответ:

- Получаем объект Python (`PyObject`)
- Сохраняем объект в переменной с GIL
- Используем Python объект через `PyAny` или `PyRef`
- Возвращаем результат обратно в Python

 Шаг 11:

Миссия 11

Расположите шаги для добавления методов к `PyClass`:

1. Создаем структуру с `#[pyclass]`
2. Внутри `impl` добавляем `#[pymethods]`
3. Определяем методы внутри `impl` блока
4. Собираем и используем методы через объект в Python

 Подсказка:

Методы добавляются к структуре через `impl` и специальный атрибут `#[pymethods]`.

Ответ:

- Создаем структуру с `#[pyclass]`
- Внутри `impl` добавляем `#[pymethods]`
- Определяем методы внутри `impl` блока
- Собираем и используем методы через объект в Python

🔪 Шаг 12:

Миссия 12

Расположите шаги работы с Python-массивами через PyO3:

1. Получаем PyList объект из Python
2. Конвертируем PyList в Vec в Rust
3. Обрабатываем данные в Rust
4. Возвращаем результат обратно как PyObject

🔪 Подсказка:

Сначала получаем объект, потом конвертируем, затем работаем и возвращаем обратно.

Ответ:

- Получаем PyList объект из Python
- Конвертируем PyList в Vec в Rust
- Обрабатываем данные в Rust
- Возвращаем результат обратно как PyObject

🔪 Шаг 13:

Миссия 13

Расположите шаги безопасной работы с GIL в Rust:

1. Входим в GIL через Python::with_gil
2. Используем PyObject или PyRef для операций с Python
3. Выходим из GIL после завершения работы с Python-объектами

🔪 Подсказка:

GIL должен быть захвачен всегда, когда Rust работает с Python-объектами.

Ответ:

- Входим в GIL через Python::with_gil
- Используем PyObject или PyRef для операций с Python
- Выходим из GIL после завершения работы с Python-объектами

🔪 Шаг 14:

Миссия 14

Расположите шаги для конвертации Python строки в Rust String:

1. Получаем PyString объект из Python
2. Используем .to_str()? для проверки и конвертации
3. Конвертируем в Rust String
4. Обрабатываем строку в Rust

🔪 Подсказка:

PyString нужно сначала безопасно проверить, а затем конвертировать.

Ответ:

- Получаем PyString объект из Python
- Используем .to_str()? для проверки и конвертации
- Конвертируем в Rust String
- Обрабатываем строку в Rust

🔮 Шаг 15:

Миссия 15

Расположите шаги для возвращения сложного объекта Rust в Python:

1. Создаем объект Rust
2. Оборачиваем объект в Py через #[pyclass]
3. Возвращаем объект в Python как PyObject
4. Используем объект в Python без нарушения владения

🔮 Подсказка:

Объект Rust нужно обернуть и безопасно вернуть в Python.

Ответ:

- Создаем объект Rust
- Оборачиваем объект в Py через #[pyclass]
- Возвращаем объект в Python как PyObject
- Используем объект в Python без нарушения владения

🔮 Шаг 16:

Миссия 16

Расположите шаги обработки Python словаря в Rust:

1. Получаем PyDict объект
2. Конвертируем в HashMap
3. Обрабатываем данные в Rust
4. Возвращаем результат обратно в Python

🔮 Подсказка:

Python словарь сначала нужно получить, потом конвертировать и работать с ним.

Ответ:

- Получаем PyDict объект
- Конвертируем в HashMap
- Обрабатываем данные в Rust
- Возвращаем результат обратно в Python

🔮 Шаг 17:

Миссия 17

Расположите шаги интеграции Rust и Python через PyO3 для полного модуля:

1. Создаем функции с #[pyfunction] и структуры с #[pyclass]
2. Добавляем методы с #[pymethods]
3. Создаем #[pymodule] и регистрируем все функции и классы
4. Собираем проект с maturin
5. Импортируем и используем модуль в Python

🔮 Подсказка:

Следуем логике: создаем код → регистрируем → собираем → используем.

Ответ:

- Создаем функции с #[pyfunction] и структуры с #[pyclass]

- Добавляем методы с `#[pymethods]`
- Создаем `#[pymodule]` и регистрируем все функции и классы
- Собираем проект с `maturin`
- Импортируем и используем модуль в Python

Урок 3 - Maturin — алхимический котёл для сборки.

Шаг 01:

Миссия 1

Что делает Maturin в проекте Rust + Python?

Подсказка: Подумайте о роли сборочного инструмента.

Ответ:

- **Собирает Rust-код и делает его доступным в Python**
- Компилирует Python в Rust
- Только проверяет синтаксис Rust
- Запускает тесты Python без сборки

Шаг 02:

Миссия 2

Какой командой собирают проект с Maturin для Python?

Подсказка: Обратите внимание на стандартную команду сборки.

Ответ:

- **maturin build**
- cargo run
- python setup.py build
- pip install .

Шаг 03:

Миссия 3

Какой флаг Maturin позволяет сразу установить пакет в текущее окружение Python?

Подсказка: Это обычно ускоряет проверку модуля.

Ответ:

- **--install**
- --develop
- --release
- --check

Шаг 04:

Миссия 4

Для чего нужен флаг `--release` в Maturin?

Подсказка: Подумайте о различии между быстрой и оптимизированной сборкой.

Ответ:

- **Собирает оптимизированную версию для продакшена**
- Собирает версию с отладочной информацией
- Отключает сборку зависимостей

- Запускает тесты без сборки

Шаг 05:

Миссия 5

 Сопоставьте термины Maturin с их описанием

Подсказка: Каждый термин отвечает за отдельную часть процесса сборки.

Ответ:

- build
- develop
- publish
- check

1. Собирает пакет для локального тестирования
2. Создает и устанавливает пакет в окружение Python для разработки
3. Проверяет код на ошибки и совместимость
4. Отправляет пакет в PyPI

- **build** — Собирает пакет для локального тестирования
- **develop** — Создает и устанавливает пакет в окружение Python для разработки
- **check** — Проверяет код на ошибки и совместимость
- **publish** — Отправляет пакет в PyPI

Шаг 06:

Миссия 6

 Сопоставьте команды Maturin с их назначением

Подсказка: Подумайте, какая команда подходит для продакшена, а какая для отладки.

Ответ:

- maturin build
- maturin develop
- maturin publish
- maturin check

1. Создает wheel для PyPI
2. Проверяет синтаксис и сборку без установки
3. Устанавливает пакет в текущее окружение для разработки
4. Собирает пакет для локального использования

- **maturin build** — Собирает пакет для локального использования
- **maturin develop** — Устанавливает пакет в текущее окружение для разработки
- **maturin publish** — Создает wheel для PyPI
- **maturin check** — Проверяет синтаксис и сборку без установки

Шаг 07:

Миссия 7

 Сопоставьте типы сборки с их целью

Подсказка: Подумайте, когда нужна скорость разработки, а когда оптимизация.

Ответ:

- debug

- release
- develop
- wheel

1. Быстрая сборка с отладочной информацией
2. Оптимизированная сборка для продакшена
3. Локальная установка для разработки Python-кода
4. Упакованный пакет для распространения

- **debug** — Быстрая сборка с отладочной информацией
- **release** — Оптимизированная сборка для продакшена
- **develop** — Локальная установка для разработки Python-кода
- **wheel** — Упакованный пакет для распространения

Шаг 08:

Миссия 8

Расположите шаги сборки Python-модуля через Maturin:

1. Пишем Rust-код
2. Добавляем зависимости и настройки в Cargo.toml
3. Выбираем команду сборки (build/develop)
4. Запускаем сборку Maturin
5. Импортируем модуль в Python и тестируем

Подсказка: Следуйте логике: код → настройки → сборка → тестирование.

Ответ:

- Пишем Rust-код
- Добавляем зависимости и настройки в Cargo.toml
- Выбираем команду сборки (build/develop)
- Запускаем сборку Maturin
- Импортируем модуль в Python и тестируем

Шаг 09:

Миссия 9

Расположите шаги публикации пакета в PyPI через Maturin:

1. Создаем wheel через maturin build
2. Проверяем пакет командой maturin check
3. Входим в учетную запись PyPI
4. Выполняем команду maturin publish
5. Проверяем доступность пакета через pip install

Подсказка: Сперва сборка и проверка, потом публикация и проверка.

Ответ:

- Создаем wheel через maturin build
- Проверяем пакет командой maturin check
- Входим в учетную запись PyPI
- Выполняем команду maturin publish
- Проверяем доступность пакета через pip install

Шаг 10:

Миссия 10

Расположите последовательность сборки с флагом --release:

1. Настройка Cargo.toml для оптимизации
2. Запуск maturin build --release
3. Сборка wheel для продакшена
4. Установка пакета в окружение Python
5. Тестирование производительности

Подсказка: Оптимизированная сборка идет через release и последующее тестирование.

Ответ:

- Настройка Cargo.toml для оптимизации
- Запуск maturin build --release
- Сборка wheel для продакшена
- Установка пакета в окружение Python
- Тестирование производительности

Шаг 11:

Миссия 11

Расположите шаги установки пакета в локальное окружение:

1. Создаем wheel с maturin build
2. Выбираем команду --install
3. Устанавливаем пакет в текущее окружение
4. Импортируем модуль в Python и проверяем работу

Подсказка: Сначала сборка, затем установка, потом тестирование.

Ответ:

- Создаем wheel с maturin build
- Выбираем команду --install
- Устанавливаем пакет в текущее окружение
- Импортируем модуль в Python и проверяем работу

Шаг 12:

Миссия 12

Расположите последовательность для сборки dev-версии:

1. Добавляем зависимости в Cargo.toml
2. Запускаем maturin develop
3. Используем модуль в Python для отладки
4. Вносим изменения в Rust-код
5. Повторяем сборку develop

Подсказка: Dev-версия позволяет быстро тестировать изменения.

Ответ:

- Добавляем зависимости в Cargo.toml
- Запускаем maturin develop
- Используем модуль в Python для отладки

- Вносим изменения в Rust-код
- Повторяем сборку develop

Шаг 13:

Миссия 13

Расположите шаги проверки пакета перед публикацией:

1. maturin check для проверки совместимости
2. Запуск локальных тестов Python
3. Исправление ошибок и предупреждений
4. Повторная проверка перед публикацией

Подсказка: Проверка пакета — это последовательность «проверка → исправление → проверка».

Ответ:

- maturin check для проверки совместимости
- Запуск локальных тестов Python
- Исправление ошибок и предупреждений
- Повторная проверка перед публикацией

Шаг 14:

Миссия 14

Расположите шаги для публикации нового релиза:

1. Собираем пакет через maturin build --release
2. Проверяем пакет командой maturin check
3. Увеличиваем версию в Cargo.toml и pyproject.toml
4. Публикуем на PyPI через maturin publish
5. Проверяем установку через pip install

Подсказка: Сначала сборка, потом версия, проверка, публикация и тест.

Ответ:

- Собираем пакет через maturin build --release
- Проверяем пакет командой maturin check
- Увеличиваем версию в Cargo.toml и pyproject.toml
- Публикуем на PyPI через maturin publish
- Проверяем установку через pip install

Урок 4 - Простейший модуль Rust для Python.

Шаг 01:

Миссия 1

Что является первой целью при создании простого модуля Rust для Python?

Подсказка: Сначала нужно сделать модуль доступным для Python.

Ответ:

- **Обеспечить возможность импорта Rust-кода в Python**
- Компилировать Python в Rust
- Создать GUI для модуля
- Написать документацию на Rust

🔮 Шаг 02:

Миссия 2

Какой инструмент позволяет интегрировать Rust и Python?

Подсказка: Это библиотека, которая генерирует обёртки для Python.

Ответ:

- **PyO3**
- NumPy
- Pandas
- Cython

🔮 Шаг 03:

Миссия 3

Какой атрибут нужен, чтобы функция Rust была видна из Python?

Подсказка: Смотрите документацию PyO3.

Ответ:

- **`#[pyfunction]`**
- `#[rustfn]`
- `#[python_export]`
- `#[module]`

🔮 Шаг 04:

Миссия 4

Что делает макрос `#[pymodule]`?

Подсказка: Подумайте о точке входа для Python.

Ответ:

- **Определяет Rust-модуль, доступный в Python**
- Компилирует Python в Rust
- Создает документацию для функций
- Запускает тесты Rust

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте термины Rust + Python с их назначением

Подсказка: Каждый термин связан с интеграцией.

Ответ:

- `#[pyfunction]`
 - `#[pymodule]`
 - `#[pyclass]`
 - `PyResult`
1. Отмечает функцию для экспорта в Python
 2. Определяет модуль, который Python сможет импортировать
 3. Определяет класс Rust, доступный в Python
 4. Тип результата функции, понятный Python

- **#[pyfunction]** — Отмечает функцию для экспорта в Python
- **#[pymodule]** — Определяет модуль, который Python сможет импортировать
- **#[pyclass]** — Определяет класс Rust, доступный в Python
- **PyResult** — Тип результата функции, понятный Python

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте термины с их назначением

Подсказка: Думайте о структуре модуля и функциях.

Ответ:

- Python
 - Rust
 - PyO3
 - Maturin
1. Язык для высокоуровневой логики
 2. Язык для высокопроизводительных компонентов
 3. Библиотека для интеграции Rust и Python
 4. Инструмент для сборки и публикации пакета
- **Python** — Язык для высокоуровневой логики
 - **Rust** — Язык для высокопроизводительных компонентов
 - **PyO3** — Библиотека для интеграции Rust и Python
 - **Maturin** — Инструмент для сборки и публикации пакета

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте атрибуты с их действием

Подсказка: Обратите внимание, какой элемент экспортируется.

Ответ:

- #[pyfunction]
 - #[pyclass]
 - #[pymodule]
 - #[text_signature]
1. Функция для Python
 2. Класс для Python
 3. Модуль для Python
 4. Устанавливает сигнатуру функции для документации
- **#[pyfunction]** — Функция для Python
 - **#[pyclass]** — Класс для Python
 - **#[pymodule]** — Модуль для Python
 - **#[text_signature]** — Устанавливает сигнатуру функции для документации

🔮 Шаг 08:

Миссия 8

Расположите шаги создания простого модуля Rust для Python:

1. Создаем проект Rust
2. Добавляем зависимость PyO3
3. Определяем `#[pymodule]`
4. Определяем `#[pyfunction]` функции
5. Собираем и тестируем модуль в Python

Подсказка: Логика: проект → зависимости → модуль → функции → тестирование.

Ответ:

- Создаем проект Rust
- Добавляем зависимость PyO3
- Определяем `#[pymodule]`
- Определяем `#[pyfunction]` функции
- Собираем и тестируем модуль в Python

Шаг 09:

Миссия 9

Расположите шаги установки модуля для Python:

1. Собираем с Maturin
2. Выбираем флаг `--install`
3. Устанавливаем пакет в текущее окружение
4. Импортируем и проверяем работу модуля

Подсказка: Сначала сборка, потом установка, затем тестирование.

Ответ:

- Собираем с Maturin
- Выбираем флаг `--install`
- Устанавливаем пакет в текущее окружение
- Импортируем и проверяем работу модуля

Шаг 10:

Миссия 10

Расположите шаги от создания функции до её вызова в Python:

1. Определяем `#[pyfunction]`
2. Добавляем функцию в `#[pymodule]`
3. Собираем проект через Maturin
4. Импортируем модуль в Python
5. Вызываем функцию в Python

Подсказка: Сначала функция, потом модуль, сборка и тест.

Ответ:

- Определяем `#[pyfunction]`
- Добавляем функцию в `#[pymodule]`
- Собираем проект через Maturin
- Импортируем модуль в Python
- Вызываем функцию в Python

Шаг 11:

Миссия 11

Расположите шаги создания класса Rust для Python:

1. Определяем `#[pyclass]`
2. Добавляем методы с `#[pymethods]`
3. Собираем проект через Maturin
4. Импортируем модуль в Python
5. Создаем объект класса и вызываем методы

Подсказка: Сначала класс и методы, затем сборка и тестирование в Python.

Ответ:

- Определяем `#[pyclass]`
- Добавляем методы с `#[pymethods]`
- Собираем проект через Maturin
- Импортируем модуль в Python
- Создаем объект класса и вызываем методы

Шаг 12:

Миссия 12

Что делает `#[pymethods]` в Rust-классе для Python?

Подсказка: Это похоже на экспорт функций класса в Python.

Ответ:

- **Отмечает методы класса, доступные из Python**
- Создает новый модуль Python
- Запускает сборку Rust
- Проверяет типы переменных

Шаг 13:

Миссия 13

 Сопоставьте типы ошибок и их источники при интеграции Rust + Python

Подсказка: Ошибки могут быть связаны с типами или сборкой.

Ответ:

- `TypeError`
 - `CompileError`
 - `ImportError`
 - `RuntimeError`
1. Неправильный тип аргумента функции Python
 2. Ошибка компиляции Rust-кода
 3. Модуль не найден в Python
 4. Ошибка при выполнении Rust-функции в Python
- **TypeError** — Неправильный тип аргумента функции Python
 - **CompileError** — Ошибка компиляции Rust-кода
 - **ImportError** — Модуль не найден в Python
 - **RuntimeError** — Ошибка при выполнении Rust-функции в Python

Шаг 14:

Миссия 14

Расположите шаги от написания функции до её тестирования в Python:

1. Определяем `#[pyfunction]`
2. Добавляем функцию в `#[pymodule]`
3. Собираем проект с Maturin
4. Импортируем модуль в Python
5. Пишем тесты и проверяем результат

Подсказка: Сначала создаем функцию, затем тестируем в Python.

Ответ:

- Определяем `#[pyfunction]`
- Добавляем функцию в `#[pymodule]`
- Собираем проект с Maturin
- Импортируем модуль в Python
- Пишем тесты и проверяем результат

Шаг 15:

Миссия 15

Что нужно сделать, чтобы Python мог использовать новый Rust-класс?

Подсказка: Подумайте о шаге после определения класса и методов.

Ответ:

- **Собрать модуль и импортировать его в Python**
- Перевести класс в Python вручную
- Создать JSON-объект с классом
- Добавить класс в Cargo.toml

Урок 5 - Практикум реализовать быструю функцию

Шаг 01:

Миссия 1

Что нужно для экспорта функции Rust в Python?

Подсказка: Подумайте о том, как Python узнает о Rust-функции.

Ответ:

- **Добавить `#[pyfunction]` и `#[pymodule]`**
- Создать C-модуль
- Написать JSON-описание функции
- Скомпилировать Rust как exe

Шаг 02:

Миссия 2

Как обозначить класс Rust, доступный из Python?

Подсказка: Смотрите на аннотации типа `#[pyclass]`.

Ответ:

- **Использовать `#[pyclass]`**
- Добавить `#[pyfunction]`
- Создать trait
- Прописать в Cargo.toml

Шаг 03:

Миссия 3

Что нужно для экспорта метода класса Rust в Python?

Подсказка: Это аннотация для методов внутри `#[pyclass]`.

Ответ:

- **Добавить `#[pymethods]`**
- Использовать `#[pyclass]`
- Добавить `#[pyfunction]`
- Создать новый crate

Шаг 04:

Миссия 4

 Сопоставьте понятия Rust-Python интеграции с их значением

Подсказка: Подумайте, какие аннотации влияют на класс, метод или модуль.

Ответ:

- `#[pyclass]`
 - `#[pymethods]`
 - `#[pyfunction]`
 - `#[pymodule]`
1. Определяет класс, доступный из Python
 2. Определяет методы класса, доступные в Python
 3. Определяет отдельную функцию Rust, доступную из Python
 4. Определяет модуль Python, включающий функции
- **`#[pyclass]`** — Определяет класс, доступный из Python
 - **`#[pymethods]`** — Определяет методы класса, доступные в Python
 - **`#[pyfunction]`** — Определяет отдельную функцию Rust, доступную из Python
 - **`#[pymodule]`** — Определяет модуль Python, включающий функции

Шаг 05:

Миссия 5

 Сопоставьте ошибки при интеграции Rust-Python с источником

Подсказка: Ошибки бывают компиляции, импорта и выполнения.

Ответ:

- `TypeError`
 - `CompileError`
 - `ImportError`
 - `RuntimeError`
1. Неверный тип аргумента Python

2. Ошибка компиляции Rust-кода
3. Модуль не найден в Python
4. Ошибка во время выполнения Rust-функции в Python

- **TypeError** — Неверный тип аргумента Python
- **CompileError** — Ошибка компиляции Rust-кода
- **ImportError** — Модуль не найден в Python
- **RuntimeError** — Ошибка во время выполнения Rust-функции в Python

Шаг 06:

Миссия 6

Расположите шаги добавления функции Rust в Python:

1. Определяем #[pyfunction]
2. Добавляем функцию в #[pymodule]
3. Собираем проект с Maturin
4. Импортируем модуль в Python
5. Пишем тесты и проверяем результат

Подсказка: Сначала определяем функцию и модуль, затем тестируем в Python.

Ответ:

- Определяем #[pyfunction]
- Добавляем функцию в #[pymodule]
- Собираем проект с Maturin
- Импортируем модуль в Python
- Пишем тесты и проверяем результат

Шаг 07:

Миссия 7

Что делает Maturin при сборке Rust-модуля для Python?

Подсказка: Подумайте, как Python получает доступ к Rust-коду.

Ответ:

- **Создает готовый для Python модуль (.pyd/.so)**
- Запускает Rust-код в Python напрямую
- Переводит Python в Rust
- Удаляет Cargo.toml

Шаг 08:

Миссия 8

Какая команда используется для установки Maturin?

Подсказка: Подумайте о менеджере пакетов Python.

Ответ:

- **pip install maturin**
- cargo install maturin
- python -m rust
- apt-get install maturin

Шаг 09:

Миссия 9

Сопоставьте команды Maturin с их назначением

Подсказка: Некоторые команды создают wheel, другие проверяют сборку.

Ответ:

- maturin build
 - maturin develop
 - maturin publish
1. Собирает модуль в wheel
 2. Устанавливает модуль в текущую среду для разработки
 3. Публикует модуль на PyPI
- **maturin build** — Собирает модуль в wheel
 - **maturin develop** — Устанавливает модуль в текущую среду для разработки
 - **maturin publish** — Публикует модуль на PyPI

Шаг 10:

Миссия 10

Расположите шаги для публикации Rust-модуля на PyPI:

1. Проверка кода и тестов
2. Создание wheel с maturin build
3. Публикация через maturin publish
4. Обновление версии в Cargo.toml

Подсказка: Сначала тестируем, потом обновляем версию, затем билд и публикация.

Ответ:

- Проверка кода и тестов
- Обновление версии в Cargo.toml
- Создание wheel с maturin build
- Публикация через maturin publish

Шаг 11:

Миссия 11

Как Python узнает о новом Rust-классе?

Подсказка: Смотрите на шаг после сборки модуля.

Ответ:

- **Импортирует собранный модуль**
- Создает JSON с описанием класса
- Переводит класс в Python вручную
- Копирует Rust-код в Python файл

Шаг 12:

Миссия 12

Сопоставьте ошибки сборки Rust-модуля и их причины

Подсказка: Ошибки могут быть в коде, зависимостях или версии Python.

Ответ:

- Missing crate
- Version mismatch
- Syntax error
- Linking error

1. Отсутствие нужного Rust crate
2. Несовместимость версий Python и Rust
3. Неправильный Rust-синтаксис
4. Ошибка при связывании библиотек

- **Missing crate** — Отсутствие нужного Rust crate
- **Version mismatch** — Несовместимость версий Python и Rust
- **Syntax error** — Неправильный Rust-синтаксис
- **Linking error** — Ошибка при связывании библиотек

 Шаг 13:

Миссия 13

Что необходимо для обновления Rust-модуля после изменения кода?

Подсказка: Подумайте о повторной сборке и установке модуля.

Ответ:

- **Собрать заново с maturin develop**
- Перезапустить Python
- Удалить .pyd/.so файл вручную
- Изменить Cargo.toml и ничего не собирать

 Шаг 14:

Миссия 14

Сопоставьте элементы Rust-модуля с их назначением в Python

Подсказка: Каждый элемент имеет конкретную роль в Python API.

Ответ:

- #[pyclass]
 - #[pymethods]
 - #[pyfunction]
 - #[pymodule]
1. Экспорт класса
 2. Экспорт методов класса
 3. Экспорт отдельной функции
 4. Определение модуля
- **#[pyclass]** — Экспорт класса
 - **#[pymethods]** — Экспорт методов класса
 - **#[pyfunction]** — Экспорт отдельной функции
 - **#[pymodule]** — Определение модуля

 Шаг 15:

Миссия 15

Расположите шаги для отладки Rust-модуля в Python:

1. Добавляем вывод для отладки в Rust
2. Собираем модуль с Maturin
3. Импортируем модуль в Python
4. Вызываем функции и проверяем вывод

Подсказка: Сначала подготавливаем код с отладкой, затем проверяем в Python.

Ответ:

- Добавляем вывод для отладки в Rust
- Собираем модуль с Maturin
- Импортируем модуль в Python
- Вызываем функции и проверяем вывод

💡 **Шаг 16:**

Миссия 16

Что важно учитывать при передаче сложных структур данных из Python в Rust?

Подсказка: Смотрите на совместимые типы и сериализацию.

Ответ:

- **Использовать совместимые типы или сериализацию**
- Передавать объекты напрямую без изменений
- Игнорировать проверку типов
- Использовать только строки

📖 **Модуль 6 - Продвинутая алхимия Rust** ✨

📖 **Урок 1 - Обработка массивов и строк в Rust.**

💡 **Шаг 01:**

Миссия 1

Как в Rust создать массив из пяти чисел?

Подсказка: Обратите внимание на синтаксис квадратных скобок и повторов.

Ответ:

- **let arr = [1, 2, 3, 4, 5];**
- let arr = (1, 2, 3, 4, 5);
- let arr = {1,2,3,4,5};
- let arr = <1,2,3,4,5>;

💡 **Шаг 02:**

Миссия 2

Как получить длину массива в Rust?

Подсказка: Методы длины обычно вызываются через точку.

Ответ:

- **arr.len()**
- arr.length
- len(arr)

- `arr.size()`

🔮 Шаг 03:

Миссия 3

Как создать строку из литерала в Rust?

Подсказка: Строки имеют тип `String` или `&str`.

Ответ:

- **`let s = String::from("hello");`**
- `let s = 'hello';`
- `let s = hello;`
- `let s = ["hello"];`

🔮 Шаг 04:

Миссия 4

Как объединить две строки в Rust?

Подсказка: Используйте методы или оператор `+`.

Ответ:

- **`let s3 = s1 + &s2;`**
- `let s3 = s1.concat(s2);`
- `let s3 = s1 & s2;`
- `let s3 = s1.append(s2);`

🔮 Шаг 05:

Миссия 5

Сопоставьте методы строк с их действием в Rust

Подсказка: Подумайте, что делает каждый метод.

Ответ:

- `to_uppercase()`
- `trim()`
- `replace("a","b")`
- `len()`

1. Преобразует все символы в верхний регистр
2. Удаляет пробелы в начале и конце строки
3. Заменяет символы 'a' на 'b'
4. Возвращает длину строки

- **`to_uppercase()`** — Преобразует все символы в верхний регистр
- **`trim()`** — Удаляет пробелы в начале и конце строки
- **`replace("a","b")`** — Заменяет символы 'a' на 'b'
- **`len()`** — Возвращает длину строки

🔮 Шаг 06:

Миссия 6

Как создать вектор в Rust с начальными значениями?

Подсказка: Используйте макрос `vec!` для быстрого создания.

Ответ:

- **let v = vec![1,2,3];**
- let v = [1,2,3];
- let v = {1,2,3};
- let v = (1,2,3);

🔮 **Шаг 07:**

Миссия 7

Как добавить элемент в конец вектора в Rust?

Подсказка: Метод push() добавляет элемент в конец.

Ответ:

- **v.push(10);**
- v.add(10);
- v.insert(10);
- v.append(10);

🔮 **Шаг 08:**

Миссия 8

Как получить доступ к элементу массива по индексу в Rust?

Подсказка: Используйте квадратные скобки.

Ответ:

- **let x = arr[2];**
- let x = arr.get(2);
- let x = arr.at(2);
- let x = arr.item(2);

🔮 **Шаг 09:**

Миссия 9

Сопоставьте методы вектора с их действием в Rust

Подсказка: Подумайте, какой метод добавляет, какой удаляет элементы и т.д.

Ответ:

- push()
- pop()
- len()
- insert(1, x)

1. Добавляет элемент в конец вектора
2. Удаляет последний элемент и возвращает его
3. Возвращает количество элементов
4. Вставляет элемент в указанную позицию

- **push()** — Добавляет элемент в конец вектора
- **pop()** — Удаляет последний элемент и возвращает его
- **len()** — Возвращает количество элементов
- **insert(1, x)** — Вставляет элемент в указанную позицию

🔗 Шаг 10:

Миссия 10

Сопоставьте методы строк с их результатом

Подсказка: Обратите внимание на отличие методов, которые возвращают новую строку, и тех, что изменяют оригинал.

Ответ:

- `to_lowercase()`
 - `starts_with("abc")`
 - `ends_with("xyz")`
 - `contains("rust")`
1. Проверяет, начинается ли строка с подстроки
 2. Проверяет, заканчивается ли строка на подстроку
 3. Проверяет, содержит ли строка указанную подстроку
 4. Преобразует строку в нижний регистр
- **`to_lowercase()`** — Преобразует строку в нижний регистр
 - **`starts_with("abc")`** — Проверяет, начинается ли строка с подстроки
 - **`ends_with("xyz")`** — Проверяет, заканчивается ли строка на подстроку
 - **`contains("rust")`** — Проверяет, содержит ли строка указанную подстроку

🔗 Шаг 11:

Миссия 11

Расположите шаги объединения двух векторов в Rust:

1. Создать новый вектор для результата
2. Добавить все элементы первого вектора
3. Добавить все элементы второго вектора
4. Использовать результат

Подсказка: Сначала нужно подготовить место для элементов, затем последовательно добавлять.

Ответ:

- Создать новый вектор для результата
- Добавить все элементы первого вектора
- Добавить все элементы второго вектора
- Использовать результат

🔗 Шаг 12:

Миссия 12

Расположите шаги удаления пробелов из строки:

1. Создать строку
2. Вызвать метод `trim()`
3. Сохранить результат
4. Использовать строку

Подсказка: `trim()` не изменяет оригинал, поэтому нужно сохранить результат.

Ответ:

- Создать строку
- Вызвать метод trim()
- Сохранить результат
- Использовать строку

🔮 **Шаг 13:**

Миссия 13

Как в Rust объединить массивы в один вектор?

Подсказка: Используйте iter() и collect() для преобразования.

Ответ:

- **let v: Vec = arr1.iter().chain(arr2.iter()).cloned().collect();**
- let v = arr1 + arr2;
- let v = [arr1, arr2];
- let v = merge(arr1, arr2);

🔮 **Шаг 14:**

Миссия 14

Как вектор в Rust можно отсортировать?

Подсказка: Используйте встроенный метод sort().

Ответ:

- **v.sort();**
- v.order();
- v.sorted();
- v.sort_by_value();

🔮 **Шаг 15:**

Миссия 15

Сопоставьте типы с их применением:

Подсказка: Векторы для динамических массивов, String для изменяемых строк.

Ответ:

- String
 - &str
 - Vec
 - [T; N]
1. Изменяемая строка
 2. Срез строки
 3. Динамический массив
 4. Фиксированный массив
- **String** — Изменяемая строка
 - **&str** — Срез строки
 - **Vec** — Динамический массив
 - **[T; N]** — Фиксированный массив

Урок 2 - Асинхронная магия `async Rust` + `Python`.

Шаг 01:

Миссия 1

Программистка Аня хочет одновременно отправлять HTTP-запросы к нескольким API. Какой подход наиболее подходящий?

Подсказка: Подумайте, что позволит не блокировать основной поток.

Ответ:

- **Использовать `async/await`**
- Вызывать функции синхронно в цикле
- Создать отдельный поток для каждого запроса
- Запускать запросы через `print`

Шаг 02:

Миссия 2

Боб пишет Rust-программу с `async`. Как ему лучше обрабатывать ошибки в `Future`?

Подсказка: Рассмотрите встроенные методы работы с `Result` и `.await`.

Ответ:

- **Использовать `.await?` и `propagate` ошибки**
- Игнорировать ошибки с `unwrap()`
- Просто печатать ошибки через `println!`
- Конвертировать все ошибки в `panic`

Шаг 03:

Миссия 3

Когда лучше использовать `tokio::spawn` в `async Rust`?

Подсказка: Подумайте, когда задача должна выполняться параллельно и не блокировать текущий `Future`.

Ответ:

- **Для параллельных задач, которые могут выполняться независимо**
- Для синхронного кода
- Чтобы заменить блокирующие `sleep`
- Чтобы автоматически завершить `main` без `await`

Шаг 04:

Миссия 4

 Задача: Сопоставьте `async`-концепции Rust с их описаниями

Понятия:

Ответ:

- `Future`
- `await`
- `Executor`
- `tokio::spawn`

Определения:

1. Объект, который представляет собой значение, которое появится позже
2. Приостанавливает выполнение, пока Future не завершится
3. Среда, которая управляет выполнением Future
4. Запускает Future параллельно на Executor

Подсказка: Обратите внимание, что одни термины описывают сущности, а другие — действия.

- **Future** — Объект, который представляет собой значение, которое появится позже
- **await** — Приостанавливает выполнение, пока Future не завершится
- **Executor** — Среда, которая управляет выполнением Future
- **tokio::spawn** — Запускает Future параллельно на Executor

Шаг 05:

Миссия 5

 Сопоставьте Python async-функции с их поведением

Понятия:

Ответ:

- `async def`
- `await`
- `asyncio.run`
- `asyncio.gather`

Определения:

1. Создает корутину
2. Ожидает завершения корутины
3. Запускает корутину как основную программу
4. Выполняет несколько корутин параллельно

Подсказка: Обратите внимание на различие между определением корутины и её запуском.

- **`async def`** — Создает корутину
- **`await`** — Ожидает завершения корутины
- **`asyncio.run`** — Запускает корутину как основную программу
- **`asyncio.gather`** — Выполняет несколько корутин параллельно

Шаг 06:

Миссия 6

 Сопоставьте типичные ошибки при работе с async

Понятия:

Ответ:

- Блокировка event loop
- Не `await Future`
- Использование блокирующих функций
- Гонка данных

Определения:

1. Программа виснет, потому что event loop занят синхронной работой

2. Корутину создали, но не дождалась её завершения
3. Вызывается синхронная функция внутри аsync-кода
4. Несогласованное состояние из-за параллельного доступа к данным

Подсказка: Подумайте, как асинхронные задачи могут мешать друг другу.

- **Блокировка event loop** — Программа виснет, потому что event loop занят синхронной работой
- **Не await Future** — Корутину создали, но не дождалась её завершения
- **Использование блокирующих функций** — Вызывается синхронная функция внутри аsync-кода
- **Гонка данных** — Несогласованное состояние из-за параллельного доступа к данным

🔮 Шаг 07:

Миссия 7

Расположите шаги запуска аsync-программы на Python в правильном порядке:

1. Определение аsync-функций
2. Создание event loop или вызов asyncio.run
3. Вызов аsync-функций через await или gather
4. Обработка результатов и ошибок

Подсказка: Сначала создаем корутины, потом запускаем их и обрабатываем результаты.

Ответ:

- Определение аsync-функций
- Создание event loop или вызов asyncio.run
- Вызов аsync-функций через await или gather
- Обработка результатов и ошибок

🔮 Шаг 08:

Миссия 8

Расположите шаги параллельной обработки HTTP-запросов в аsync Rust:

1. Создание Future для каждого запроса
2. Запуск всех Future через tokio::join! или spawn
3. Обработка результатов каждого Future
4. Обработка ошибок

Подсказка: Сначала формируем задачи, потом запускаем их и обрабатываем результат.

Ответ:

- Создание Future для каждого запроса
- Запуск всех Future через tokio::join! или spawn
- Обработка результатов каждого Future
- Обработка ошибок

🔮 Шаг 09:

Миссия 9

Что из этого не является преимуществом аsync Rust?

Подсказка: Сравните асинхронный код с синхронным и многопоточным.

Ответ:

- **Мгновенное выполнение без ожидания ресурсов**
- Меньше блокировок потоков
- Параллельное выполнение задач
- Экономия памяти по сравнению с потоками

🔮 Шаг 10:

Миссия 10

Сопоставьте асинх-конструкции Rust с Python-аналогами

Понятия:

Ответ:

- Future
- await
- tokio::spawn
- Executor

Определения:

1. Корутина, которую можно ожидать
2. Приостанавливает выполнение до завершения Future
3. Запускает задачу параллельно
4. Event loop, который управляет задачами

Подсказка: Сравните с `asyncio.run` и `asyncio.gather`.

- **Future** — Корутина, которую можно ожидать
- **await** — Приостанавливает выполнение до завершения Future
- **tokio::spawn** — Запускает задачу параллельно
- **Executor** — Event loop, который управляет задачами

🔮 Шаг 11:

Миссия 11

Когда стоит использовать `async` в Python?

Подсказка: Подумайте, что является узким местом в программе.

Ответ:

- **При множественных IO-bound операциях**
- При чисто вычислительных задачах
- Для однопоточного синхронного кода
- Чтобы ускорить `print`

🔮 Шаг 12:

Миссия 12

Сопоставьте ключевые слова `async` с их эффектами в Rust

Понятия:

Ответ:

- `async`
- `await`

- Send
- Sync

Определения:

1. Объявляет функцию возвращающую Future
2. Ожидает завершение Future
3. Метка, что объект можно безопасно передавать между потоками
4. Метка, что объект можно безопасно использовать из нескольких потоков

Подсказка: Send и Sync чаще всего встречаются при работе с многопоточностью.

- **async** — Объявляет функцию возвращающую Future
- **await** — Ожидает завершение Future
- **Send** — Метка, что объект можно безопасно передавать между потоками
- **Sync** — Метка, что объект можно безопасно использовать из нескольких потоков

🔗 Шаг 13:

Миссия 13

Расположите шаги от простой корутины к параллельной обработке нескольких задач в Python:

1. Создание async def функций
2. Создание корутин
3. Запуск через asyncio.gather
4. Обработка результатов

Подсказка: Начинаем с определения функций, потом создаем задачи и параллельно их запускаем.

Ответ:

- Создание async def функций
- Создание корутин
- Запуск через asyncio.gather
- Обработка результатов

🔗 Шаг 14:

Миссия 14

Что нужно помнить при работе с mutable данными в async Rust?

Подсказка: Подумайте о параллельном доступе к одним и тем же данным.

Ответ:

- **Использовать Arc<T> или аналогичные безопасные конструкции**
- Можно изменять данные из разных задач без блокировки
- Использовать обычные глобальные переменные
- Всё mutable поведение безопасно без синхронизации

🔗 Шаг 15:

Миссия 15

Сопоставьте async-термины с их типичными сценариями использования

Понятия:

Ответ:

- `Future`
- `tokio::spawn`
- `await`
- `async_trait`

Определения:

1. Запуск независимой задачи параллельно
2. Определение асинхронного метода в трейте
3. Ожидание завершения `Future`
4. Объект, представляющий значение, которое появится позже

Подсказка: Подумайте, какие конструкции создают задачи, а какие их выполняют.

- **`Future`** — Объект, представляющий значение, которое появится позже
- **`tokio::spawn`** — Запуск независимой задачи параллельно
- **`await`** — Ожидание завершения `Future`
- **`async_trait`** — Определение асинхронного метода в трейте

 Шаг 16:

Миссия 16

Расположите шаги от запуска нескольких `async Rust` задач до обработки их результатов:

1. Создание `Future` для каждой задачи
2. Запуск `Future` через `tokio::spawn` или `join!`
3. Сбор результатов
4. Обработка ошибок

Подсказка: Сначала формируем задачи, затем выполняем их параллельно и только потом обрабатываем результаты и ошибки.

Ответ:

- Создание `Future` для каждой задачи
- Запуск `Future` через `tokio::spawn` или `join!`
- Сбор результатов
- Обработка ошибок

 Шаг 17:

Миссия 17

Что из нижеперечисленного помогает избежать гонок данных в `async Rust`?

Подсказка: Подумайте о безопасной работе с общими ресурсами.

Ответ:

- **Использование `Arc` или `RwLock`**
- Игнорирование ошибок при доступе к данным
- Использование обычного `&mut` без синхронизации
- Запуск всех задач в одном потоке без `await`

Урок 3 - Интеграция с NumPy через Rust.

Шаг 01:

Миссия 1

Программист Ваня хочет использовать Rust для ускорения вычислений NumPy массива. Что стоит сделать в первую очередь?

Подсказка: Начните с того, как связать Rust и Python.

Ответ:

- **Использовать PyO3 или rust-numpy для обертки функций Rust**
- Сразу переписать весь Python-код на Rust
- Создать глобальные переменные NumPy в Rust
- Просто импортировать Rust как обычный модуль Python

Шаг 02:

Миссия 2

Саша хочет передать NumPy массив в Rust без копирования данных. Как это лучше сделать?

Подсказка: Подумайте о безопасных ссылках на данные.

Ответ:

- **Использовать PyArray и методы unsafe view_mut/readonly в rust-numpy**
- Скопировать массив в новый Rust-вектор
- Конвертировать в список Python и передать
- Сохранять массив в файл и читать в Rust

Шаг 03:

Миссия 3

Анна пишет функцию на Rust, которая принимает NumPy массив. Как ей обеспечить совместимость типов?

Подсказка: Обратите внимание на dtype NumPy.

Ответ:

- **Использовать PyArray с правильным T, соответствующим dtype**
- Использовать i32 для всех массивов
- Приводить массив к строкам
- Игнорировать типы и конвертировать через unsafe pointer

Шаг 04:

Миссия 4

 Сопоставьте понятия интеграции Rust с NumPy с их объяснением.

Понятия:

Ответ:

- PyO3
- rust-numpy
- PyArray
- unsafe view_mut

Определения:

1. Библиотека для создания Python-модулей на Rust
2. Библиотека для работы с NumPy массивами в Rust
3. Обертка массива NumPy для Rust
4. Позволяет работать с массивом без копирования

Подсказка: Сравните, что делает каждый слой абстракции.

- **PyO3** — Библиотека для создания Python-модулей на Rust
- **rust-numpy** — Библиотека для работы с NumPy массивами в Rust
- **PyArray** — Обертка массива NumPy для Rust
- **unsafe view_mut** — Позволяет работать с массивом без копирования

Шаг 05:

Миссия 5

Сопоставьте типы NumPy с типами Rust.

Понятия:

Ответ:

- float64
- int32
- uint8
- bool

Определения:

1. f64
2. i32
3. u8
4. bool

Подсказка: Обратите внимание на размер и знак числа.

- **float64** — f64
- **int32** — i32
- **uint8** — u8
- **bool** — bool

Шаг 06:

Миссия 6

Сопоставьте ошибки при передаче NumPy массивов в Rust с их причинами.

Понятия:

Ответ:

- Несовпадение dtype
- Несоответствие размеров
- Попытка изменять immutable массив
- Неправильный lifetime

Определения:

1. Тип Rust не совпадает с dtype NumPy

2. Размер Rust массива не совпадает с размером NumPy
3. Используется `immutable view_mut`
4. Rust не может гарантировать время жизни ссылки

Подсказка: Обратите внимание на соответствие типов и правил ownership.

- **Несовпадение dtype** — Тип Rust не совпадает с dtype NumPy
- **Несоответствие размеров** — Размер Rust массива не совпадает с размером NumPy
- **Попытка изменять immutable массив** — Используется `immutable view_mut`
- **Неправильный lifetime** — Rust не может гарантировать время жизни ссылки

🔮 Шаг 07:

Миссия 7

Расположите шаги безопасного изменения NumPy массива в Rust:

1. Создание PyArray ссылки
2. Получение mutable view через `unsafe view_mut`
3. Изменение элементов массива
4. Возврат управления Python

Подсказка: Сначала создаем ссылку, затем изменяем массив и возвращаем результат.

Ответ:

- Создание PyArray ссылки
- Получение mutable view через `unsafe view_mut`
- Изменение элементов массива
- Возврат управления Python

🔮 Шаг 08:

Миссия 8

Расположите шаги вызова Rust-функции из Python:

1. Создание Python-модуля через PyO3
2. Объявление функций с `#[pyfunction]`
3. Компиляция в shared library
4. Импорт модуля в Python
5. Вызов функции с NumPy массивом

Подсказка: Сначала создаем модуль в Rust, затем импортируем и вызываем в Python.

Ответ:

- Создание Python-модуля через PyO3
- Объявление функций с `#[pyfunction]`
- Компиляция в shared library
- Импорт модуля в Python
- Вызов функции с NumPy массивом

🔮 Шаг 09:

Миссия 9

Маша хочет ускорить вычисления матричных операций. Как лучше интегрировать Rust с NumPy?

Подсказка: Подумайте о передаче массивов в Rust без лишних копий.

Ответ:

- **Использовать rust-numpy для обработки массивов на Rust**
- Скопировать массивы в Python списки и передавать в Rust
- Сразу преобразовать все массивы в C-struct
- Игнорировать NumPy и работать только с Rust-векторами

🔗 Шаг 10:

Миссия 10

Игорь хочет изменить массив NumPy в Rust параллельно. Что важно учитывать?

Подсказка: Подумайте о безопасном доступе к данным.

Ответ:

- **Использовать mutable view и синхронизацию при параллельных изменениях**
- Можно изменять массивы без защиты в нескольких потоках
- Копировать массив для каждого потока
- Неважно, в каком порядке меняются элементы

🔗 Шаг 11:

Миссия 11

Сопоставьте методы работы с NumPy массивами в Rust с их действиями.

Понятия:

Ответ:

- to_owned
- as_slice
- view
- view_mut

Определения:

1. Создает копию массива
2. Получает неизменяемую ссылку на массив
3. Получает неизменяемый view на данные без копирования
4. Получает изменяемый view на данные без копирования

Подсказка: Обратите внимание на различие между копией и view.

- **to_owned** — Создает копию массива
- **as_slice** — Получает неизменяемую ссылку на массив
- **view** — Получает неизменяемый view на данные без копирования
- **view_mut** — Получает изменяемый view на данные без копирования

🔗 Шаг 12:

Миссия 12

Сопоставьте типы NumPy с их размерностями и Rust-представлением.

Понятия:

Ответ:

- 1D float64
- 2D int32

- 3D uint8
- bool array

Определения:

1. Одномерный f64 в Rust (Vec)
2. Двумерный i32 в Rust (Array2)
3. Трёхмерный u8 в Rust (Array3)
4. Массив булевых значений (Array)

Подсказка: Смотрите на количество осей и тип данных.

- **1D float64** — Одномерный f64 в Rust (Vec)
- **2D int32** — Двумерный i32 в Rust (Array2)
- **3D uint8** — Трёхмерный u8 в Rust (Array3)
- **bool array** — Массив булевых значений (Array)

🔮 Шаг 13:

Миссия 13

Расположите шаги безопасного параллельного вычисления на массиве NumPy в Rust:

1. Создать PyArray и получить view_mut
2. Разбить массив на чанки для потоков
3. Параллельно обрабатывать чанки с безопасной синхронизацией
4. Объединить результаты и вернуть в Python

Подсказка: Сначала подготовка, затем обработка, затем возврат результата.

Ответ:

- Создать PyArray и получить view_mut
- Разбить массив на чанки для потоков
- Параллельно обрабатывать чанки с безопасной синхронизацией
- Объединить результаты и вернуть в Python

🔮 Шаг 14:

Миссия 14

Коля хочет ускорить функцию NumPy, используя SIMD в Rust. Что нужно учитывать?

Подсказка: SIMD ускоряет вычисления на векторах, но безопасный доступ к массиву важен.

Ответ:

- **Использовать view_mut и SIMD через safe abstraction**
- Можно обрабатывать любые указатели без контроля
- SIMD работает только с копиями массивов
- Не нужно учитывать выравнивание данных

🔮 Шаг 15:

Миссия 15

Сопоставьте методы оптимизации Rust-NumPy функций с их назначением:

Понятия:

Ответ:

- Parallel iterator

- SIMD vectorization
- Lazy evaluation
- Caching

Определения:

1. Вычисление элементов параллельно в нескольких потоках
2. Обработка данных с помощью инструкций векторизации
3. Отложенное вычисление до момента использования
4. Хранение промежуточных результатов для повторного использования

Подсказка: Подумайте, как ускорить вычисления без ошибок и лишних копий.

- **Parallel iterator** — Вычисление элементов параллельно в нескольких потоках
- **SIMD vectorization** — Обработка данных с помощью инструкций векторизации
- **Lazy evaluation** — Отложенное вычисление до момента использования
- **Caching** — Хранение промежуточных результатов для повторного использования

Шаг 16:

Миссия 16

Расположите шаги интеграции Rust-функции с Python-скриптом, использующим NumPy:

1. Написание функции на Rust с `#[pyfunction]`
2. Компиляция Rust в shared library
3. Создание PyArray и передача его в Rust
4. Выполнение вычислений в Rust
5. Возврат результата в Python и преобразование в NumPy

Подсказка: Сначала создаем функцию и библиотеку, затем передаем массив и получаем результат.

Ответ:

- Написание функции на Rust с `#[pyfunction]`
- Компиляция Rust в shared library
- Создание PyArray и передача его в Rust
- Выполнение вычислений в Rust
- Возврат результата в Python и преобразование в NumPy

Урок 4 - Практикум реализовать быстрый алгоритм поиска подстрок.

Шаг 01:

Миссия 1

Петя хочет реализовать быстрый поиск подстрок в тексте на Rust. С чего лучше начать?

Подсказка: Начните с выбора алгоритма, подходящего для больших строк.

Ответ:

- **Выбрать эффективный алгоритм поиска, например KMP или Boyer-Moore**
- Сразу использовать встроенные функции Python
- Перебрать все возможные подстроки вручную
- Сначала оптимизировать ввод/вывод

🔮 Шаг 02:

Миссия 2

Саша хочет, чтобы алгоритм поиска подстрок был быстрым на больших текстах. Какой подход стоит использовать?

Подсказка: Подумайте о времени работы алгоритма при увеличении длины текста.

Ответ:

- **Использовать алгоритм с линейным временем работы по тексту, например KMP**
- Сравнивать каждую позицию текста с шаблоном вручную
- Сортировать текст перед поиском
- Использовать только рекурсию без мемоизации

🔮 Шаг 03:

Миссия 3

Аня реализует поиск подстрок и хочет избежать лишних копий текста. Как лучше передавать строки в Rust?

Подсказка: Смотрите на ссылки и lifetime.

Ответ:

- **Передавать строки по ссылке (&str) без копирования**
- Всегда создавать новые String при каждом вызове функции
- Копировать текст в Vec для каждого вызова
- Сохранять текст в глобальной переменной

🔮 Шаг 04:

Миссия 4

Сопоставьте алгоритмы поиска подстрок с их особенностями:

Понятия:

Ответ:

- KMP
- Boyer-Moore
- Rabin-Karp
- Naïve

Определения:

1. Использует префикс-функцию для линейного поиска
2. Сравнивает с конца шаблона, эффективно пропуская символы
3. Хэширует подстроки для быстрого сравнения
4. Сравнивает каждый символ подряд без оптимизации

Подсказка: Смотрите на способ уменьшения количества сравнений.

- **KMP** — Использует префикс-функцию для линейного поиска
- **Boyer-Moore** — Сравнивает с конца шаблона, эффективно пропуская символы
- **Rabin-Karp** — Хэширует подстроки для быстрого сравнения
- **Naïve** — Сравнивает каждый символ подряд без оптимизации

🔗 Шаг 05:

Миссия 5

Сопоставьте типичные ошибки при реализации поиска подстрок в Rust с их причинами:

Понятия:

Ответ:

- Выход за границы строки
- Неправильный индекс префикс-функции
- Копирование строк в цикле
- Игнорирование case sensitivity

Определения:

1. Итерация за пределы &str
2. Ошибки при вычислении длины совпадений
3. Замедление работы из-за лишних копий
4. Пропуск совпадений из-за разного регистра

Подсказка: Обратите внимание на индексы и свойства строк.

- **Выход за границы строки** — Итерация за пределы &str
- **Неправильный индекс префикс-функции** — Ошибки при вычислении длины совпадений
- **Копирование строк в цикле** — Замедление работы из-за лишних копий
- **Игнорирование case sensitivity** — Пропуск совпадений из-за разного регистра

🔗 Шаг 06:

Миссия 6

Расположите шаги реализации алгоритма КМП:

1. Вычисление префикс-функции шаблона
2. Итерация по тексту и сравнение символов
3. Использование префикс-функции для пропуска совпавших символов
4. Запись найденных позиций совпадений

Подсказка: Сначала анализируем шаблон, потом ищем по тексту.

Ответ:

- Вычисление префикс-функции шаблона
- Итерация по тексту и сравнение символов
- Использование префикс-функции для пропуска совпавших символов
- Запись найденных позиций совпадений

🔗 Шаг 07:

Миссия 7

Расположите шаги реализации алгоритма КМП:

1. Вычисление префикс-функции шаблона
2. Итерация по тексту и сравнение символов
3. Использование префикс-функции для пропуска совпавших символов
4. Запись найденных позиций совпадений

Подсказка: Сначала анализируем шаблон, потом ищем по тексту.

Ответ:

- Вычисление префикс-функции шаблона
- Итерация по тексту и сравнение символов
- Использование префикс-функции для пропуска совпавших символов
- Запись найденных позиций совпадений

🔗 **Шаг 08:**

Миссия 8

Расположите шаги алгоритма Бойера-Мура при поиске подстроки:

1. Вычисление таблицы сдвигов по bad character rule
2. Итерация по тексту с конца шаблона
3. Сравнение символов с конца шаблона
4. Сдвиг шаблона согласно таблице при несовпадении

Подсказка: Сначала создаем таблицы, потом ищем с конца.

Ответ:

- Вычисление таблицы сдвигов по bad character rule
- Итерация по тексту с конца шаблона
- Сравнение символов с конца шаблона
- Сдвиг шаблона согласно таблице при несовпадении

🔗 **Шаг 09:**

Миссия 9

Ирина хочет ускорить поиск подстрок с помощью параллельной обработки. Какой подход безопасен в Rust?

Подсказка: Обратите внимание на безопасный доступ к данным в нескольких потоках.

Ответ:

- **Разделять текст на чанки и использовать безопасные ссылки (&str) для потоков**
- Разрешить потокам изменять один и тот же текст без защиты
- Копировать текст в каждый поток
- Использовать глобальные переменные для индексов

🔗 **Шаг 10:**

Миссия 10

Максим хочет реализовать хэширование подстрок для Rabin-Karp. Что важно учесть?

Подсказка: Подумайте о коллизиях и больших числах.

Ответ:

- **Использовать модульное арифметическое хэширование и обработку коллизий**
- Не проверять хэши на совпадения, сразу выводить результат
- Использовать float числа для хэширования
- Хэшировать только первые 5 символов подстроки

🔗 **Шаг 11:**

Миссия 11

Сопоставьте методы оптимизации поиска подстрок с их назначением:

Понятия:

Ответ:

- Использование префикс-функции
- Хэширование подстрок
- Сравнение с конца (Boyer-Moore)
- Разделение на чанки для параллельного поиска

Определения:

1. Пропуск уже совпавших символов для линейного поиска
2. Быстрое сравнение подстрок через числовое представление
3. Эффективный сдвиг шаблона при несовпадении
4. Параллельная обработка больших текстов

Подсказка: Смотрите на тип ускорения: линейный, хэш, сдвиг или многопоток.

- **Использование префикс-функции** — Пропуск уже совпавших символов для линейного поиска
- **Хэширование подстрок** — Быстрое сравнение подстрок через числовое представление
- **Сравнение с конца (Boyer-Moore)** — Эффективный сдвиг шаблона при несовпадении
- **Разделение на чанки для параллельного поиска** — Параллельная обработка больших текстов

Шаг 12:

Миссия 12

Сопоставьте структуры данных с их ролью в быстром поиске подстрок:

Понятия:

Ответ:

- Префикс-функция
- Таблица bad character
- Хэш-подстроки
- Вес для хранения результатов

Определения:

1. Позволяет пропускать уже совпавшие символы
2. Сдвиг шаблона при несовпадении символа
3. Представление подстрок для быстрого сравнения
4. Сохранение позиций найденных совпадений

Подсказка: Смотрите на связь структуры данных и алгоритма.

- **Префикс-функция** — Позволяет пропускать уже совпавшие символы
- **Таблица bad character** — Сдвиг шаблона при несовпадении символа
- **Хэш-подстроки** — Представление подстрок для быстрого сравнения
- **Вес для хранения результатов** — Сохранение позиций найденных совпадений

Шаг 13:

Миссия 13

Расположите шаги алгоритма Rabin-Karp:

1. Вычисление хэшей подстрок текста
2. Вычисление хэша шаблона
3. Сравнение хэшей и проверка совпадений при совпадении
4. Возврат позиций найденных совпадений

Подсказка: Сначала считаем хэши, потом сравниваем и проверяем совпадения.

Ответ:

- Вычисление хэшей подстрок текста
- Вычисление хэша шаблона
- Сравнение хэшей и проверка совпадений при совпадении
- Возврат позиций найденных совпадений

🔔 Шаг 14:

Миссия 14

Сопоставьте типы коллизий в алгоритме Rabin-Karp с их последствиями:

Понятия:

Ответ:

- Хэш совпадает, но строки разные
- Хэш не совпадает, строки одинаковые
- Несоответствие регистра
- Проверка подстрок без хэшей

Определения:

1. Ложно положительное срабатывание, нужно дополнительное сравнение
2. Случай крайне редкий, обычно игнорируется
3. Пропуск совпадений при различном регистре
4. Медленный поиск без ускорения

Подсказка: Обратите внимание на то, что хэши — это только предварительная проверка.

- **Хэш совпадает, но строки разные** — Ложно положительное срабатывание, нужно дополнительное сравнение
- **Хэш не совпадает, строки одинаковые** — Случай крайне редкий, обычно игнорируется
- **Несоответствие регистра** — Пропуск совпадений при различном регистре
- **Проверка подстрок без хэшей** — Медленный поиск без ускорения

🔔 Шаг 15:

Миссия 15

Никита хочет протестировать алгоритм поиска подстрок на больших данных. Что лучше проверить в первую очередь?

Подсказка: Начните с корректности, потом оценивайте производительность.

Ответ:

- **Сначала проверить, что алгоритм корректно находит все совпадения**
- Сразу оптимизировать скорость без проверки корректности
- Только проверить скорость на случайных данных
- Использовать маленькие тесты для оценки производительности

Шаг 16:

Миссия 16

Расположите шаги отладки алгоритма поиска подстрок:

1. Проверка на маленьких тестовых строках
2. Использование инструментов отладки и логирования
3. Проверка на больших текстах
4. Оптимизация после подтверждения корректности

Подсказка: Сначала убедитесь в правильности работы, потом оптимизируйте.

Ответ:

- Проверка на маленьких тестовых строках
- Использование инструментов отладки и логирования
- Проверка на больших текстах
- Оптимизация после подтверждения корректности

Шаг 17:

Миссия 17

Лена хочет, чтобы алгоритм поиска подстрок был готов к реальным данным с разным регистром и спецсимволами. Что следует сделать?

Подсказка: Подумайте о предварительной обработке текста и шаблона.

Ответ:

- Привести текст и шаблон к одному регистру и обрабатывать спецсимволы
- Игнорировать регистр и спецсимволы, чтобы ускорить поиск
- Заменить все символы на числа без учета регистра
- Удалять все спецсимволы без сохранения их позиций

Модуль 7 - Сравнительная алхимия

Урок 1 - Таблица выбора инструментов где Rust лучше, где C.

Шаг 01:

Миссия 1

Аня хочет выбрать инструмент для высокопроизводительных вычислений с безопасной памятью. Что лучше использовать?

Подсказка: Подумайте о сочетании скорости и контроля за безопасностью памяти.

Ответ:

- Rust
- C
- Python
- JavaScript

Шаг 02:

Миссия 2

Игорь пишет низкоуровневый драйвер с ограниченным ресурсом памяти. Какой язык предпочтительнее?

Подсказка: Подумайте о максимальном контроле над памятью и минимальных накладных расходах.

Ответ:

- **C**
- Rust
- Python
- Go

Шаг 03:

Миссия 3

Сопоставьте задачи с языком, который чаще выбирают:

Понятия:

Ответ:

- Высокопроизводительные системы с безопасной памятью
- Низкоуровневое программирование, драйверы
- Быстрая прототипная разработка
- Веб-разработка и скрипты

Определения:

1. Rust
2. C
3. Python
4. JavaScript

Подсказка: Смотрите на баланс между безопасностью, скоростью и простотой.

- **Высокопроизводительные системы с безопасной памятью** — Rust
- **Низкоуровневое программирование, драйверы** — C
- **Быстрая прототипная разработка** — Python
- **Веб-разработка и скрипты** — JavaScript

Шаг 04:

Миссия 4

Лена хочет писать многопоточные приложения без опасности data race. Что выбрать?

Подсказка: Подумайте о встроенной системе владения и проверках компилятора.

Ответ:

- **Rust**
- C
- Python
- Java

Шаг 05:

Миссия 5

Сопоставьте особенности языков с преимуществами:

Понятия:

Ответ:

- Rust ownership и borrow checker
- С управление памятью вручную
- Python динамическая типизация
- JavaScript интерпретируемый язык

Определения:

1. Безопасность памяти и предотвращение гонок
2. Полный контроль над ресурсами и эффективная работа на низком уровне
3. Быстрая разработка, гибкость типов
4. Простота и кроссплатформенность в веб

Подсказка: Обратите внимание на связи между механизмами языка и задачами.

- **Rust ownership и borrow checker** — Безопасность памяти и предотвращение гонок
- **С управление памятью вручную** — Полный контроль над ресурсами и эффективная работа на низком уровне
- **Python динамическая типизация** — Быстрая разработка, гибкость типов
- **JavaScript интерпретируемый язык** — Простота и кроссплатформенность в веб

 Шаг 06:

Миссия 6

Дима хочет обрабатывать большие массивы данных максимально быстро. Что выбрать?

Подсказка: Смотрите на скорость выполнения без лишних проверок на каждом шаге.

Ответ:

- **C**
- Rust
- Python
- JavaScript

 Шаг 07:

Миссия 7

Расположите ситуации по приоритету использования Rust:

1. Высокопроизводительные многопоточные системы
2. Безопасность памяти критична
3. Быстрое прототипирование без контроля памяти
4. Низкоуровневый драйвер

Подсказка: Rust лучше там, где важна скорость + безопасность.

Ответ:

- Высокопроизводительные многопоточные системы
- Безопасность памяти критична
- Быстрое прототипирование без контроля памяти
- Низкоуровневый драйвер

 Шаг 08:

Миссия 8

Саша хочет писать микроконтроллерный код с точным контролем времени выполнения. Что лучше?

Подсказка: Подумайте о прямом управлении ресурсами и минимальных накладных расходах компилятора.

Ответ:

- **C**
- Rust
- Python
- Go

Шаг 09:

Миссия 9

Сопоставьте ситуации с подходящим языком:

Понятия:

Ответ:

- Разработка безопасного сетевого сервиса
- Создание операционной системы
- Прототипирование алгоритмов анализа данных
- Скрипты для автоматизации задач

Определения:

1. Rust
2. C
3. Python
4. JavaScript

Подсказка: Смотрите на безопасность и производительность для системного уровня.

- **Разработка безопасного сетевого сервиса** — Rust
- **Создание операционной системы** — C
- **Прототипирование алгоритмов анализа данных** — Python
- **Скрипты для автоматизации задач** — JavaScript

Шаг 10:

Миссия 10

Максим хочет писать безопасные многопоточные приложения без явного контроля за памятью. Что выбрать?

Подсказка: Подумайте о механизмах владения и проверок на этапе компиляции.

Ответ:

- **Rust**
- C
- Python
- Go

Шаг 11:

Миссия 11

Сопоставьте преимущества языка с задачей:

Понятия:

Ответ:

- Контроль над памятью
- Безопасность потоков
- Быстрая разработка и гибкость
- Интерпретируемость и кроссплатформенность

Определения:

1. C
2. Rust
3. Python
4. JavaScript

Подсказка: Обратите внимание, что некоторые языки минимизируют ошибки компиляции, а другие ускоряют прототипирование.

- **Контроль над памятью** — C
- **Безопасность потоков** — Rust
- **Быстрая разработка и гибкость** — Python
- **Интерпретируемость и кроссплатформенность** — JavaScript

Шаг 12:

Миссия 12

Оля хочет писать код для IoT-устройств с минимальной задержкой. Какой язык выбрать?

Подсказка: Минимальная задержка и прямой доступ к ресурсам важнее безопасности компилятора.

Ответ:

- C
- Rust
- Python
- JavaScript

Шаг 13:

Миссия 13

Расположите языки по скорости выполнения в типичных задачах:

1. C
2. Rust
3. Python
4. JavaScript

Подсказка: Смотрите на компилируемые vs интерпретируемые языки.

Ответ:

- C
- Rust
- Python
- JavaScript

Шаг 14:

Миссия 14

Павел хочет писать высоконагруженные серверы с безопасной памятью. Какой язык выбрать?

Подсказка: Безопасность памяти критична, но нужна высокая скорость.

Ответ:

- **Rust**
- C
- Python
- JavaScript

Шаг 15:

Миссия 15

Сопоставьте ситуации с языком, который подходит лучше:

Понятия:

Ответ:

- Системное программирование с ограниченными ресурсами
- Разработка web-приложений
- Научные расчёты и анализ данных
- Реализация многопоточных безопасных сервисов

Определения:

1. C
2. JavaScript
3. Python
4. Rust

Подсказка: Оцените приоритеты: ресурсы, безопасность, скорость разработки.

- **Системное программирование с ограниченными ресурсами** — C
- **Разработка web-приложений** — JavaScript
- **Научные расчёты и анализ данных** — Python
- **Реализация многопоточных безопасных сервисов** — Rust

Шаг 16:

Миссия 16

Мария хочет писать прототип алгоритма с минимальными затратами времени. Какой язык выбрать?

Подсказка: Смотрите на гибкость и скорость написания кода, а не на скорость выполнения.

Ответ:

- **Python**
- C
- Rust
- Go

Урок 2 - Совместное использование C и Rust.

Шаг 01:

Миссия 1

Анна хочет ускорить работу Python-кода, используя безопасный язык с контролем памяти. Какой язык выбрать?

Подсказка: Подумайте о безопасности памяти и отсутствии необходимости ручного управления.

Ответ:

- Rust
- C
- Python
- JavaScript

Шаг 02:

Миссия 2

Иван работает с legacy C-кодом и хочет постепенно использовать Rust для новых модулей. Что важнее учитывать?

Подсказка: Обратите внимание на совместимость с существующим C-кодом.

Ответ:

- **FFI (Foreign Function Interface) и совместимость типов**
- Игнорировать предупреждения компилятора
- Полностью переписать C-код сразу
- Использовать только unsafe блоки Rust

Шаг 03:

Миссия 3

Сопоставьте сценарии интеграции с подходящим языком:

Понятия:

Ответ:

- Модуль для обработки больших массивов данных
- Системное ядро с управлением памятью
- Прототип быстрой функции
- Скрипт для автоматизации тестов

Определения:

1. Rust
2. C
3. Python
4. JavaScript

Подсказка: Оцените приоритет: безопасность, скорость, легкость написания.

- **Модуль для обработки больших массивов данных** — Rust
- **Системное ядро с управлением памятью** — C
- **Прототип быстрой функции** — Python
- **Скрипт для автоматизации тестов** — JavaScript

🔗 Шаг 04:

Миссия 4

Наталья хочет использовать Rust для части проекта на C и обмениваться данными между модулями. Что важнее всего?

Подсказка: Смотрите на безопасный способ вызова функций и передачи данных между языками.

Ответ:

- **Использование FFI и согласованных типов данных**
- Использование глобальных переменных для всех данных
- Игнорирование проверки границ массивов
- Прямое копирование памяти без контроля

🔗 Шаг 05:

Миссия 5

Сопоставьте преимущества совместного использования C и Rust:

Понятия:

Ответ:

- Скорость выполнения критичных участков
- Безопасность памяти для новых модулей
- Простота прототипирования
- Модернизация старого кода постепенно

Определения:

1. C
2. Rust
3. Python
4. Rust + C

Подсказка: Оцените возможности для новых модулей и существующего кода.

- **Скорость выполнения критичных участков** — C
- **Безопасность памяти для новых модулей** — Rust
- **Простота прототипирования** — Python
- **Модернизация старого кода постепенно** — Rust + C

🔗 Шаг 06:

Миссия 6

Дмитрий хочет использовать Rust для замены части C-кода, не ломая существующую архитектуру. Что ключевое?

Подсказка: Обратите внимание на взаимодействие через FFI и сохранение интерфейсов.

Ответ:

- **Согласованная работа интерфейсов через FFI**
- Полная замена всех модулей сразу
- Игнорирование типов данных между языками
- Использование unsafe без проверки

🔮 Шаг 07:

Миссия 7

Сергей хочет, чтобы новый модуль был безопасен и при этом взаимодействовал с существующим С-кодом. Что выбрать?

Подсказка: Безопасность памяти для новых модулей и совместимость — ключевые моменты.

Ответ:

- **Rust с FFI для взаимодействия с С**
- Полный отказ от С-кода
- Использовать только unsafe блоки Rust
- Игнорировать типы данных при передаче

🔮 Шаг 08:

Миссия 8

Сопоставьте подходы к совместному использованию С и Rust с их преимуществами:

Понятия:

Ответ:

- Использование Rust для новых модулей
- Оптимизация критических функций в С
- Постепенная интеграция Rust в существующий проект
- Прототипирование новых функций

Определения:

1. Безопасность памяти
2. Скорость выполнения
3. Минимизация риска ошибок
4. Быстрое тестирование идей

Подсказка: Обратите внимание, что разные языки подходят для разных целей.

- **Использование Rust для новых модулей** — Безопасность памяти
- **Оптимизация критических функций в С** — Скорость выполнения
- **Постепенная интеграция Rust в существующий проект** — Минимизация риска ошибок
- **Прототипирование новых функций** — Быстрое тестирование идей

🔮 Шаг 09:

Миссия 9

Алексей хочет ускорить работу библиотеки обработки чисел. Какой вариант оптимален?

Подсказка: Выбирайте язык с высокой скоростью выполнения и контролем над памятью.

Ответ:

- **Реализовать критические функции на С или Rust**
- Оставить полностью на Python
- Использовать только скрипты на JavaScript
- Смешивать Python и JavaScript без оптимизации

🔮 Шаг 10:

Миссия 10

Сопоставьте инструменты интеграции Rust и C с реальными задачами:

Понятия:

Ответ:

- FFI
- Bindgen
- Cargo
- Компиляция статической библиотеки

Определения:

1. Вызов C-функций из Rust
2. Генерация оберток для C-кода
3. Сборка проекта Rust с зависимостями
4. Использование Rust-кода в C через линковку

Подсказка: Каждый инструмент имеет конкретную роль в совместном проекте.

- **FFI** — Вызов C-функций из Rust
- **Bindgen** — Генерация оберток для C-кода
- **Cargo** — Сборка проекта Rust с зависимостями
- **Компиляция статической библиотеки** — Использование Rust-кода в C через линковку

Шаг 11:

Миссия 11

Михаил хочет добавить безопасный модуль логирования в старый C-проект. Как лучше поступить?

Подсказка: Ищите язык с проверкой безопасности памяти и совместимостью с C.

Ответ:

- **Реализовать модуль на Rust с FFI**
- Использовать Python напрямую
- Прямо переписать весь проект на Rust
- Добавить unsafe C-функции без проверки

Шаг 12:

Миссия 12

Сопоставьте возможные риски совместного использования C и Rust с их решениями:

Понятия:

Ответ:

- Проблемы с управлением памятью
- Несоответствие типов
- Потеря производительности из-за интерфейсов
- Ошибки компоновки при линковке

Определения:

1. Использование безопасных типов Rust и проверки границ
2. Согласование типов через FFI
3. Минимизация вызовов через FFI, пакетная обработка данных
4. Корректная настройка линковки и сборки

Подсказка: Подумайте, как Rust и C могут безопасно взаимодействовать.

- **Проблемы с управлением памятью** — Использование безопасных типов Rust и проверки границ
- **Несоответствие типов** — Согласование типов через FFI
- **Потеря производительности из-за интерфейсов** — Минимизация вызовов через FFI, пакетная обработка данных
- **Ошибки компоновки при линковке** — Корректная настройка линковки и сборки

🔮 Шаг 13:

Миссия 13

Екатерина планирует использовать Rust для нового алгоритма в C-проекте. Какой подход правильный?

Подсказка: Сосредоточьтесь на безопасной интеграции и минимальных изменениях существующего кода.

Ответ:

- **Создать Rust-библиотеку и подключить через FFI**
- Переписать весь C-проект на Rust
- Игнорировать типы данных при передаче
- Использовать `unsafe` без проверки границ

🔮 Шаг 14:

Миссия 14

Сопоставьте этапы безопасной интеграции Rust в C-проект:

Понятия:

Ответ:

- Определение интерфейсов функций
- Создание оберток для передачи данных
- Компиляция Rust-кода в статическую библиотеку
- Тестирование взаимодействия с C

Определения:

1. Согласование FFI
2. Защита памяти и типов
3. Линковка с C-проектом
4. Проверка корректности работы

Подсказка: Этапы нужно выполнять по порядку для безопасного результата.

- **Определение интерфейсов функций** — Согласование FFI
- **Создание оберток для передачи данных** — Защита памяти и типов
- **Компиляция Rust-кода в статическую библиотеку** — Линковка с C-проектом
- **Тестирование взаимодействия с C** — Проверка корректности работы

🔮 Шаг 15:

Миссия 15

Павел хочет, чтобы новый модуль Rust работал с C, но не мог вызвать ошибки памяти. Что сделать?

Подсказка: Rust обеспечивает защиту памяти, если правильно настроить FFI и типы данных.

Ответ:

- **Использовать безопасные типы и границы в Rust и согласовать через FFI**
- Игнорировать проверку типов и границ
- Добавить unsafe везде без контроля
- Вызывать Rust-функции напрямую из Python без C

Урок 3 - Сравнение производительности на примерах.

Шаг 01:

Миссия 1

Иван измеряет производительность алгоритма на Python и Rust. Что обычно быстрее для числовых вычислений?

Подсказка: Rust компилируется в машинный код и управляет памятью безопасно.

Ответ:

- **Rust**
- Python
- JavaScript
- Ruby

Шаг 02:

Миссия 2

Анастасия хочет ускорить обработку больших массивов чисел. Что лучше использовать?

Подсказка: Смотрите на возможность эффективного обращения с памятью и SIMD.

Ответ:

- **C или Rust**
- Python без оптимизаций
- PHP
- Bash скрипт

Шаг 03:

Миссия 3

Сопоставьте языки с их типичным преимуществом по скорости выполнения:

Понятия:

Ответ:

- Python
- Rust
- C
- NumPy на Python

Определения:

1. Высокая скорость благодаря компиляции в машинный код
2. Интерпретируемый, медленнее чистого C
3. Оптимизированные операции с массивами через C-библиотеки
4. Низкоуровневая максимальная производительность

Подсказка: Смотрите, где компиляция и где высокоуровневая обертка.

- **Python** — Интерпретируемый, медленнее чистого C
- **Rust** — Высокая скорость благодаря компиляции в машинный код
- **C** — Низкоуровневая максимальная производительность
- **NumPy на Python** — Оптимизированные операции с массивами через C-библиотеки

🔮 Шаг 04:

Миссия 4

Сергей хочет сравнить производительность сортировки больших массивов. Что важно измерять?
Подсказка: Смотрите не только на скорость, но и на использование памяти.

Ответ:

- **Время выполнения и использование памяти**
- Только синтаксис кода
- Количество строк кода
- Количество комментариев

🔮 Шаг 05:

Миссия 5

Сопоставьте типы задач с оптимальным языком:

Ответ:

- Обработка больших матриц
- Системное программирование
- Быстрая проверка прототипа
- Написание веб-сервиса

1. Python с NumPy
2. Rust или C
3. Python
4. Python, Rust или C

Подсказка: Каждое решение зависит от сочетания скорости и удобства.

- **Обработка больших матриц** — Python с NumPy
- **Системное программирование** — Rust или C
- **Быстрая проверка прототипа** — Python
- **Написание веб-сервиса** — Python, Rust или C

🔮 Шаг 06:

Миссия 6

Екатерина хочет ускорить Python-код с помощью Rust. Что наиболее прямой путь?
Подсказка: Не нужно переписывать весь проект.

Ответ:

- **Переписать критические функции на Rust и вызвать через FFI**
- Переписать весь проект на Rust
- Добавить больше циклов в Python
- Использовать только стандартный Python

Шаг 07:

Миссия 7

Сопоставьте методы ускорения Python-кода с их эффектами:

Ответ:

- Использование NumPy
 - Встроенные C-расширения
 - Rust через FFI
 - Cython
1. Векторизация и работа с массивами
 2. Ускорение критических функций на C
 3. Безопасная и быстрая реализация алгоритмов
 4. Компиляция Python-кода в C-подобный

Подсказка: Каждый инструмент улучшает производительность по-своему.

- **Использование NumPy** — Векторизация и работа с массивами
- **Встроенные C-расширения** — Ускорение критических функций на C
- **Rust через FFI** — Безопасная и быстрая реализация алгоритмов
- **Cython** — Компиляция Python-кода в C-подобный

Шаг 08:

Миссия 8

Сопоставьте методы ускорения Python-кода с их областью применения:

Ответ:

- NumPy
 - Cython
 - Rust через FFI
 - Встроенные C-библиотеки
1. Массивы и векторизация
 2. Компиляция критических циклов
 3. Безопасные и быстрые алгоритмы
 4. Оптимизация отдельных функций

Подсказка: Каждый инструмент ускоряет Python в своей нише.

- **NumPy** — Массивы и векторизация
- **Cython** — Компиляция критических циклов
- **Rust через FFI** — Безопасные и быстрые алгоритмы
- **Встроенные C-библиотеки** — Оптимизация отдельных функций

Шаг 09:

Миссия 9

Александр хочет ускорить Python-скрипт для обработки логов. Что самый быстрый способ?

Подсказка: Используйте уже готовые библиотеки, если хотите избежать переписывания всего кода.

Ответ:

- **Использовать C-библиотеки или Rust через FFI для критических частей**

- Переписать весь проект на C
- Добавить больше print()
- Использовать чистый Python без оптимизации

🔧 Шаг 10:

Миссия 10

Мария хочет ускорить Python-код для больших CSV-файлов. Какой подход лучше?

Подсказка: Часто быстрее использовать готовые оптимизированные библиотеки.

Ответ:

- **Использовать Pandas и NumPy или вызвать Rust/C функции**
- Читать файлы построчно в Python циклом
- Сохранять CSV в текстовом формате и обрабатывать строками
- Конвертировать CSV в Excel и обрабатывать вручную

🔧 Шаг 11:

Миссия 11

Сопоставьте инструменты с преимуществами:

Ответ:

- NumPy
 - Cython
 - Rust через FFI
 - Параллельные вычисления в Python
1. Векторизация и оптимизация массивов
 2. Компиляция циклов и функций
 3. Безопасное ускорение алгоритмов
 4. Использование многопоточности в Python

Подсказка: Каждое решение усиливает производительность по-разному.

- **NumPy** — Векторизация и оптимизация массивов
- **Cython** — Компиляция циклов и функций
- **Rust через FFI** — Безопасное ускорение алгоритмов
- **Параллельные вычисления в Python** — Использование многопоточности в Python

🔧 Шаг 12:

Миссия 12

Игорь хочет ускорить числовые расчёты на Python. Что наиболее эффективное решение?

Подсказка: Смотрите на готовые высокопроизводительные библиотеки и компилируемые расширения.

Ответ:

- **Использовать NumPy и Rust через FFI для критических функций**
- Оставить чистый Python
- Переписать проект на JavaScript
- Добавлять больше циклов Python

Шаг 13:

Миссия 13

Сопоставьте сценарии ускорения Python с подходящими инструментами:

Ответ:

- Массивные матричные вычисления
- Числовые алгоритмы с критическим циклом
- Параллельная обработка данных
- Создание безопасного низкоуровневого модуля

1. NumPy
2. Cython
3. Модуль multiprocessing в Python
4. Rust через FFI

Подсказка: Смотрите на оптимизацию для скорости, параллельности и безопасности.

- **Массивные матричные вычисления** — NumPy
- **Числовые алгоритмы с критическим циклом** — Cython
- **Параллельная обработка данных** — Модуль multiprocessing в Python
- **Создание безопасного низкоуровневого модуля** — Rust через FFI

Шаг 14:

Миссия 14

Расположите шаги сравнения производительности Python и Rust правильно:

1. Выбрать тестовую задачу
2. Написать Python-реализацию
3. Написать Rust-реализацию
4. Измерить время выполнения и использование памяти
5. Сравнить результаты и сделать вывод

Подсказка: Сначала пишем оба варианта, потом измеряем и анализируем.

Ответ:

- Выбрать тестовую задачу
- Написать Python-реализацию
- Написать Rust-реализацию
- Измерить время выполнения и использование памяти
- Сравнить результаты и сделать вывод

Урок 4 - Практикум решить одну задачу тремя способами

Шаг 01:

Миссия 1

Какой способ ускорения Python лучше использовать для числового массива с миллионами элементов?

Подсказка: Подумайте, где Python работает медленно и нужны оптимизированные библиотеки.

Ответ:

- **Использовать NumPy с Rust или C расширением**

- Использовать чистый Python без изменений
- Сохранять данные в списке Python
- Переписать алгоритм на JavaScript

💡 Шаг 02:

Миссия 2

Сопоставьте подходы ускорения Python с их характеристиками:

Ответ:

- NumPy
 - Cython
 - Rust через FFI
 - Модуль multiprocessing
1. Оптимизация массивных вычислений
 2. Ускорение циклов Python
 3. Безопасный низкоуровневый код
 4. Параллельная обработка задач

Подсказка: Смотрите на назначение каждого инструмента для конкретной задачи.

- **NumPy** — Оптимизация массивных вычислений
- **Cython** — Ускорение циклов Python
- **Rust через FFI** — Безопасный низкоуровневый код
- **Модуль multiprocessing** — Параллельная обработка задач

💡 Шаг 03:

Миссия 3

Какой подход лучше выбрать для быстрой проверки алгоритма поиска подстрок в больших строках?

Подсказка: Сначала подумайте, что важнее — скорость выполнения или удобство разработки.

Ответ:

- **Написать Rust модуль и вызвать через Python FFI**
- Использовать стандартные функции Python str
- Сделать всё в чистом Python с циклами
- Использовать JavaScript для поиска

💡 Шаг 04:

Миссия 4

Расположите шаги реализации одной задачи тремя способами в правильном порядке:

1. Выбрать задачу для оптимизации
2. Реализовать на Python
3. Реализовать ускорение через C
4. Реализовать ускорение через Rust
5. Сравнить результаты и сделать вывод

Подсказка: Сначала выбираем задачу, затем реализуем все три подхода, и только потом сравниваем.

Ответ:

- Выбрать задачу для оптимизации
- Реализовать на Python
- Реализовать ускорение через C
- Реализовать ускорение через Rust
- Сравнить результаты и сделать вывод

 Шаг 05:

Миссия 5

Какой метод лучше использовать для вычислений с многомерными массивами?

Подсказка: Смотрите на библиотеки, которые оптимизированы для работы с матрицами.

Ответ:

- **NumPy с вызовом C или Rust функций**
- Списки Python с вложенными циклами
- Использовать стандартный Python range
- Переписать на Java

 Шаг 06:

Миссия 6

Сопоставьте подходы ускорения с их преимуществами:

Ответ:

- NumPy
 - Cython
 - Rust через FFI
 - Multiprocessing
1. Быстрая работа с массивами
 2. Компиляция Python-кода для ускорения циклов
 3. Безопасная работа с памятью и параллельные вычисления
 4. Запуск нескольких процессов одновременно

Подсказка: Каждый инструмент имеет своё сильное направление.

- **NumPy** — Быстрая работа с массивами
- **Cython** — Компиляция Python-кода для ускорения циклов
- **Rust через FFI** — Безопасная работа с памятью и параллельные вычисления
- **Multiprocessing** — Запуск нескольких процессов одновременно

 Шаг 07:

Миссия 7

Какой подход выбрать для быстрого поиска шаблонов в больших текстовых файлах?

Подсказка: Скорость чтения и обработки больших строк критична.

Ответ:

- **Использовать Rust через Python FFI**
- Чистый Python с методами str
- Переписать на JavaScript
- Считывать файл построчно и искать вручную

🔮 Шаг 08:

Миссия 8

Расположите шаги реализации одной текстовой задачи в правильном порядке:

1. Выбрать задачу поиска в тексте
2. Реализовать на Python
3. Реализовать на C
4. Реализовать на Rust
5. Сравнить производительность

Подсказка: Сначала реализуем простое решение, затем ускорим его через низкоуровневые языки, и в конце измеряем результаты.

Ответ:

- Выбрать задачу поиска в тексте
- Реализовать на Python
- Реализовать на C
- Реализовать на Rust
- Сравнить производительность

🔮 Шаг 09:

Миссия 9

Для какой задачи лучше использовать C вместо Rust?

Подсказка: Подумайте о зрелых библиотеках и поддержке экосистемы.

Ответ:

- **Интеграция с устаревшими библиотеками C**
- Новые безопасные параллельные алгоритмы
- Создание Rust crate
- Использование Python list comprehension

🔮 Шаг 10:

Миссия 10

Какой способ лучше использовать для ускорения Python без изменения логики?

Подсказка: Подумайте о том, как подключить сторонний код для ускорения.

Ответ:

- **Использовать Rust или C через FFI**
- Переписать весь код на Rust
- Использовать Java
- Использовать Python без изменений

🔮 Шаг 11:

Миссия 11

Расположите действия для ускорения Python через FFI в правильном порядке:

1. Выбрать узкое место кода
2. Реализовать ускорение на C или Rust
3. Скомпилировать библиотеку
4. Вызвать через Python FFI

5. Проверить производительность

Подсказка: Логика та же: сначала выбираем проблему, потом ускоряем, и проверяем результат.

Ответ:

- Выбрать узкое место кода
- Реализовать ускорение на C или Rust
- Скомпилировать библиотеку
- Вызвать через Python FFI
- Проверить производительность

Шаг 12:

Миссия 12

Какой инструмент выбрать для анализа производительности разных реализаций?

Подсказка: Сосредоточьтесь на инструментах, которые показывают время выполнения и потребление ресурсов.

Ответ:

- **Использовать профайлеры Python и Rust/C**
- Просто догадаться, что быстрее
- Считать вручную время выполнения
- Использовать print для замеров

Шаг 13:

Миссия 13

Сопоставьте инструменты с их назначением:

Ответ:

- cProfile
- timeit
- perf
- valgrind

1. Профилирование Python-кода
2. Замеры времени выполнения фрагментов
3. Точный бенчмарк производительности
4. Отладка использования памяти в C/Rust

Подсказка: Разные инструменты подходят для разных языков и задач.

- **cProfile** — Профилирование Python-кода
- **timeit** — Замеры времени выполнения фрагментов
- **perf** — Точный бенчмарк производительности
- **valgrind** — Отладка использования памяти в C/Rust

Шаг 14:

Миссия 14

Какой подход ускорения лучше для кроссплатформенных приложений?

Подсказка: Обратите внимание на переносимость и безопасность кода.

Ответ:

- **Использовать Rust для безопасного низкоуровневого кода**
- Использовать C с низкой переносимостью
- Переписать весь Python-код на Java
- Использовать только Python

Шаг 15:

Миссия 15

Сопоставьте тип ускорения с его преимуществами:

Ответ:

- Python + C через FFI
 - Python + Rust через FFI
 - Cython
 - Multiprocessing
1. Высокая переносимость и безопасность памяти
 2. Ускорение критических участков с минимальными изменениями
 3. Компиляция Python-кода для ускорения циклов
 4. Параллельное выполнение задач

Подсказка: Смотрите на цель каждого подхода: безопасность, скорость или параллельность.

- **Python + C через FFI** — Ускорение критических участков с минимальными изменениями
- **Python + Rust через FFI** — Высокая переносимость и безопасность памяти
- **Cython** — Компиляция Python-кода для ускорения циклов
- **Multiprocessing** — Параллельное выполнение задач

Шаг 16:

Миссия 16

Что поможет определить узкие места кода перед оптимизацией?

Подсказка: Не угадываем — используем инструменты для анализа.

Ответ:

- **Профайлинг и замеры времени выполнения**
- Просто догадки и интуиция
- Сразу переписывать код на Rust
- Игнорировать производительность

Шаг 17:

Миссия 17

Расположите шаги анализа и оптимизации кода:

1. Выбрать задачу или узкое место
2. Измерить время выполнения
3. Реализовать ускорение через C или Rust
4. Сравнить результаты
5. Документировать изменения

Подсказка: Сначала анализируем, затем оптимизируем, после чего проверяем и документируем.

Ответ:

- Выбрать задачу или узкое место
- Измерить время выполнения
- Реализовать ускорение через C или Rust
- Сравнить результаты
- Документировать изменения

Шаг 18:

Миссия 18

Какой метод лучше для параллельных вычислений в Python?

Подсказка: Подумайте о GIL и распределении задач.

Ответ:

- **Multiprocessing или Rust через FFI**
- Обычные Python циклы
- Использовать только NumPy
- Переписать на JavaScript

Шаг 19:

Миссия 19

Сопоставьте задачи с подходящими инструментами ускорения:

Ответ:

- Обработка больших массивов чисел
- Поиск в текстовых файлах
- Параллельные вычисления
- Простые скрипты анализа данных

1. NumPy или Rust
2. Rust через FFI или C
3. Multiprocessing или Rust
4. Python

Подсказка: Подумайте, какой инструмент лучше подходит для типа данных и задачи.

- **Обработка больших массивов чисел** — NumPy или Rust
- **Поиск в текстовых файлах** — Rust через FFI или C
- **Параллельные вычисления** — Multiprocessing или Rust
- **Простые скрипты анализа данных** — Python

Модуль 8 - Алхимия проектов

Урок 1 - Оптимизация работы с изображениями.

Шаг 01:

Миссия 1

Алиса обрабатывает тысячи изображений и замечает, что Python слишком медленный. Какой подход ускорения будет наиболее подходящим?

Подсказка: Смотрите на возможности подключения высокопроизводительного кода.

Ответ:

- **Использовать C или Rust через FFI для обработки изображений**
- Оставить Python без изменений
- Переписать на JavaScript
- Использовать только Pillow в Python

Шаг 02:

Миссия 2

Сопоставьте инструмент ускорения обработки изображений с его преимуществом:

Ответ:

- Pillow
 - NumPy
 - OpenCV с C
 - Rust через FFI
1. Простая работа с изображениями в Python
 2. Быстрое манипулирование массивами пикселей
 3. Высокая производительность через C-библиотеки
 4. Безопасная и параллельная обработка больших изображений

Подсказка: Подумайте, что важно: простота, скорость или безопасность и параллельность.

- **Pillow** — Простая работа с изображениями в Python
- **NumPy** — Быстрое манипулирование массивами пикселей
- **OpenCV с C** — Высокая производительность через C-библиотеки
- **Rust через FFI** — Безопасная и параллельная обработка больших изображений

Шаг 03:

Миссия 3

Боб загружает сотни изображений и применяет фильтры. Что поможет ускорить обработку?

Подсказка: Оптимизация алгоритмов часто важнее простого переписывания на другом языке.

Ответ:

- **Использовать NumPy для векторизации операций с пикселями**
- Использовать обычные Python циклы
- Сохранять изображения в текстовом формате
- Обрабатывать только одно изображение за раз без оптимизации

Шаг 04:

Миссия 4

Расположите шаги оптимизации обработки изображений в правильном порядке:

1. Проанализировать производительность текущей реализации
2. Выбрать узкие места (горячие точки)
3. Реализовать ускорение через C или Rust
4. Сравнить результаты с исходной реализацией
5. Документировать изменения и рекомендации

Подсказка: Сначала анализируем, затем ускоряем, потом проверяем и документируем.

Ответ:

- Проанализировать производительность текущей реализации
- Выбрать узкие места (горячие точки)
- Реализовать ускорение через C или Rust
- Сравнить результаты с исходной реализацией
- Документировать изменения и рекомендации

🔮 Шаг 05:

Миссия 5

Клара хочет применять фильтры на видео в реальном времени. Что будет оптимальнее?

Подсказка: Обратите внимание на производительность и параллельность.

Ответ:

- **Использовать Rust через FFI с параллельной обработкой кадров**
- Python с обычными циклами и Pillow
- Сохранять видео на диск и обрабатывать позже
- Использовать JavaScript в браузере

🔮 Шаг 06:

Миссия 6

Сопоставьте методы ускорения с их применением в работе с изображениями:

Ответ:

- Векторизация через NumPy
 - Параллельные вычисления в Rust
 - Использование OpenCV с C
 - Pillow для Python
1. Обработка больших массивов пикселей быстро
 2. Реализация безопасного и параллельного фильтра видео
 3. Высокая производительность готовых функций обработки изображений
 4. Быстрая и простая работа с изображениями для небольших проектов

Подсказка: Подумайте, что важнее: скорость, безопасность, готовые функции или простота.

- **Векторизация через NumPy** — Обработка больших массивов пикселей быстро
- **Параллельные вычисления в Rust** — Реализация безопасного и параллельного фильтра видео
- **Использование OpenCV с C** — Высокая производительность готовых функций обработки изображений
- **Pillow для Python** — Быстрая и простая работа с изображениями для небольших проектов

🔮 Шаг 07:

Миссия 7

Давид оптимизирует код для наложения фильтров на тысячи изображений. Что повысит скорость без переписывания всей программы?

Подсказка: Сосредоточьтесь на ускорении только узких мест.

Ответ:

- **Использовать C или Rust только для функций фильтрации**
- Переписать весь проект на Rust
- Добавить комментарии и логирование
- Использовать Python без изменений

Шаг 08:

Миссия 8

Расположите шаги ускорения обработки изображений в проекте:

1. Проанализировать текущую производительность
2. Выбрать функции для оптимизации
3. Реализовать ускорение через C или Rust
4. Проверить корректность и скорость
5. Документировать изменения и рекомендации

Подсказка: Сначала анализируем, затем ускоряем, потом проверяем и документируем.

Ответ:

- Проанализировать текущую производительность
- Выбрать функции для оптимизации
- Реализовать ускорение через C или Rust
- Проверить корректность и скорость
- Документировать изменения и рекомендации

Шаг 09:

Миссия 9

Эмма хочет быстро изменять размер тысяч изображений. Что ускорит процесс?

Подсказка: Подумайте про пакетные операции и векторизацию.

Ответ:

- **Использовать NumPy для пакетной обработки массивов пикселей**
- Изменять размер по одному изображению вручную
- Сохранять каждое изображение в отдельный файл и обрабатывать позже
- Применять Pillow без оптимизации

Шаг 10:

Миссия 10

Сопоставьте инструменты с их преимуществом в пакетной обработке изображений:

Ответ:

- NumPy
 - Rust через FFI
 - OpenCV с C
 - Pillow
1. Быстрое выполнение массивных операций с пикселями
 2. Безопасная и многопоточная обработка больших изображений
 3. Высокая производительность готовых функций фильтров
 4. Простая работа с изображениями для небольших задач

Подсказка: Смотрите на разницу между скоростью, безопасностью и удобством.

- **NumPy** — Быстрое выполнение массивных операций с пикселями
- **Rust через FFI** — Безопасная и многопоточная обработка больших изображений
- **OpenCV с C** — Высокая производительность готовых функций фильтров
- **Pillow** — Простая работа с изображениями для небольших задач

🔮 Шаг 11:

Миссия 11

Феликс хочет накладывать водяные знаки на тысячи изображений. Что ускорит процесс без изменения Python кода полностью?

Подсказка: Сосредоточьтесь на узких местах в обработке.

Ответ:

- **Вынести функцию наложения водяного знака в Rust через FFI**
- Писать водяные знаки вручную для каждого изображения
- Переписать весь проект на C
- Использовать Pillow без оптимизации

🔮 Шаг 12:

Миссия 12

Расположите шаги ускорения пакетного наложения фильтров на изображения:

1. Определить функции, которые тормозят обработку
2. Вынести эти функции в C или Rust
3. Сравнить скорость и корректность работы
4. Документировать изменения и рекомендации

Подсказка: Сначала анализируем, потом ускоряем, проверяем и документируем.

Ответ:

- Определить функции, которые тормозят обработку
- Вынести эти функции в C или Rust
- Сравнить скорость и корректность работы
- Документировать изменения и рекомендации

🔮 Шаг 13:

Миссия 13

Люсия хочет одновременно применять несколько фильтров к видео. Какой подход оптимален?

Подсказка: Производительность зависит от параллельной обработки.

Ответ:

- **Использовать Rust через FFI с многопоточностью**
- Python с обычными циклами
- Сохранять видео и обрабатывать позже
- JavaScript в браузере

🔮 Шаг 14:

Миссия 14

Сопоставьте методы ускорения с конкретными сценариями:

Ответ:

- NumPy
 - OpenCV с C
 - Rust через FFI
 - Pillow
1. Пакетная обработка массивов пикселей
 2. Применение сложных фильтров с высокой скоростью
 3. Безопасная многопоточная обработка видео
 4. Простая обработка изображений для прототипов

Подсказка: Подумайте, где нужна скорость, где безопасность, а где простота.

- **NumPy** — Пакетная обработка массивов пикселей
- **OpenCV с C** — Применение сложных фильтров с высокой скоростью
- **Rust через FFI** — Безопасная многопоточная обработка видео
- **Pillow** — Простая обработка изображений для прототипов

 Шаг 15:

Миссия 15

Никита хочет ускорить конвертацию тысяч изображений из PNG в JPEG. Что даст наибольший прирост скорости?

Подсказка: Сосредоточьтесь на низкоуровневой обработке форматов.

Ответ:

- **Использовать Rust или C через FFI для конвертации**
- Python с обычными циклами и Pillow
- Конвертировать вручную каждое изображение
- JavaScript в браузере

 Шаг 16:

Миссия 16

Расположите шаги ускорения пакетной конвертации изображений:

1. Определить узкие места конвертации
2. Вынести конвертацию в C или Rust
3. Проверить корректность форматов и скорость
4. Документировать процесс и рекомендации

Подсказка: Сначала анализируем, потом ускоряем, проверяем и документируем.

Ответ:

- Определить узкие места конвертации
- Вынести конвертацию в C или Rust
- Проверить корректность форматов и скорость
- Документировать процесс и рекомендации

 Урок 2 - Ускорение алгоритмов шифрования.

 Шаг 01:

Миссия 1

Алиса шифрует сообщения Python-скриптом, но процесс слишком медленный. Что поможет ускорить вычисления?

Подсказка: Смотрите на возможность использовать более быстрый язык для узких мест.

Ответ:

- **Вынести узкие функции в C через CFFI или ctypes**
- Использовать только Python без оптимизаций
- Добавлять большие комментарии
- Переписывать весь проект на Bash

Шаг 02:

Миссия 2

Сопоставьте методы оптимизации с их преимуществами при ускорении шифрования:

Ответ:

- C через CFFI
 - Rust через FFI
 - NumPy
 - Многопоточность Python
1. Высокая скорость низкоуровневых операций
 2. Безопасная и многопоточная обработка данных
 3. Эффективные операции с массивами данных
 4. Параллельная обработка с ограничениями GIL

Подсказка: Думайте, где нужна безопасность, где скорость и где удобство работы с массивами.

- **C через CFFI** — Высокая скорость низкоуровневых операций
- **Rust через FFI** — Безопасная и многопоточная обработка данных
- **NumPy** — Эффективные операции с массивами данных
- **Многопоточность Python** — Параллельная обработка с ограничениями GIL

Шаг 03:

Миссия 3

Боб хочет шифровать файлы быстрее, используя Python. Какой способ оптимизации лучше всего?

Подсказка: Подумайте про узкие места в коде.

Ответ:

- **Переписать вычислительные функции на Rust через FFI**
- Оставить Python и запускать последовательные циклы
- Сохранять данные в текстовых файлах между операциями
- Использовать только стандартные библиотеки без оптимизаций

Шаг 04:

Миссия 4

Расположите шаги оптимизации алгоритма шифрования:

1. Проанализировать узкие места в коде
2. Вынести критические функции в C или Rust
3. Протестировать корректность и скорость

4. Документировать рекомендации по ускорению

Подсказка: Сначала анализируем, потом оптимизируем, проверяем и документируем.

Ответ:

- Проанализировать узкие места в коде
- Вынести критические функции в C или Rust
- Протестировать корректность и скорость
- Документировать рекомендации по ускорению

Шаг 05:

Миссия 5

Клара работает с шифрованием больших файлов. Какой подход снизит время обработки?

Подсказка: Думайте про многопоточность и низкоуровневую оптимизацию.

Ответ:

- **Использовать Rust с многопоточными вычислениями через FFI**
- Запускать шифрование в одном потоке Python
- Сохранять промежуточные данные на диск
- Увеличить количество комментариев в коде

Шаг 06:

Миссия 6

Сопоставьте типы шифрования с оптимальными стратегиями ускорения:

Ответ:

- Симметричное шифрование
 - Асимметричное шифрование
 - Хэширование
 - Подписи цифровые
1. Использовать Rust или C для блоков шифрования
 2. Оптимизировать ключевые функции на низком уровне
 3. Векторизовать операции над данными
 4. Применять безопасную многопоточность для вычислений

Подсказка: Думайте, где важна скорость операций над массивами, а где безопасность и многопоточность.

- **Симметричное шифрование** — Использовать Rust или C для блоков шифрования
- **Асимметричное шифрование** — Оптимизировать ключевые функции на низком уровне
- **Хэширование** — Векторизовать операции над данными
- **Подписи цифровые** — Применять безопасную многопоточность для вычислений

Шаг 07:

Миссия 7

Давид хочет ускорить генерацию ключей RSA. Что поможет на практике?

Подсказка: Сосредоточьтесь на математических вычислениях.

Ответ:

- **Переписать генерацию ключей на Rust с использованием FFI**

- Оставить чистый Python с циклом for
- Генерировать ключи вручную через калькулятор
- Добавить много print() для отладки

🔧 Шаг 08:

Миссия 8

Расположите шаги ускорения алгоритмов шифрования для большого объема данных:

1. Проанализировать профиль производительности
2. Определить узкие функции для переноса на Rust/C
3. Реализовать ускоренные функции через FFI
4. Проверить корректность и сравнить скорость

Подсказка: Сначала анализ, потом оптимизация, затем тестирование.

Ответ:

- Проанализировать профиль производительности
- Определить узкие функции для переноса на Rust/C
- Реализовать ускоренные функции через FFI
- Проверить корректность и сравнить скорость

🔧 Шаг 09:

Миссия 9

Сопоставьте типы данных в шифровании с методами их ускорения:

Ответ:

- Большие строки
 - Массивы байт
 - Списки ключей
 - Словари параметров
1. Векторизация с NumPy
 2. Использование C/Rust для массивов
 3. Многопоточность Python
 4. Оптимизация доступа через CFFI

Подсказка: Обратите внимание, где важна скорость операций с массивами и где многопоточность.

- **Большие строки** — Векторизация с NumPy
- **Массивы байт** — Использование C/Rust для массивов
- **Списки ключей** — Многопоточность Python
- **Словари параметров** — Оптимизация доступа через CFFI

🔧 Шаг 10:

Миссия 10

Элла хочет ускорить шифрование потоков данных. Какой подход эффективнее всего?

Подсказка: Смотрите на параллельные вычисления и низкоуровневые оптимизации.

Ответ:

- **Использовать Rust с многопоточными функциями через FFI**
- Последовательно обрабатывать данные в Python

- Сохранять промежуточные результаты на диск
- Увеличить количество логов и print()

🔮 Шаг 11:

Миссия 11

Расположите этапы оптимизации алгоритма шифрования с большими файлами:

1. Проанализировать профиль производительности
2. Определить узкие места
3. Вынести критические функции в Rust/C
4. Протестировать корректность и сравнить скорость

Подсказка: Сначала анализируем, затем оптимизируем и проверяем.

Ответ:

- Проанализировать профиль производительности
- Определить узкие места
- Вынести критические функции в Rust/C
- Протестировать корректность и сравнить скорость

🔮 Шаг 12:

Миссия 12

Франк обрабатывает шифрование в реальном времени. Какой метод ускорения даст наибольший выигрыш?

Подсказка: Ищите решение для критических вычислений.

Ответ:

- **Переписать узкие функции на Rust и использовать FFI**
- Добавить больше циклов for в Python
- Обработать данные вручную через калькулятор
- Записывать все промежуточные данные на диск

🔮 Шаг 13:

Миссия 13

Сопоставьте операции с оптимальными инструментами ускорения:

Ответ:

- Шифрование блоков данных
 - Хэширование файлов
 - Генерация ключей
 - Цифровые подписи
1. Rust через FFI
 2. Векторизация NumPy
 3. C через CFFI
 4. Многопоточность безопасная

Подсказка: Определите, где важна скорость, а где — безопасная многопоточность.

- **Шифрование блоков данных** — Rust через FFI
- **Хэширование файлов** — Векторизация NumPy
- **Генерация ключей** — C через CFFI

- **Цифровые подписи** — Многопоточность безопасная

🔮 Шаг 14:

Миссия 14

Габриэль хочет ускорить проверку подписи цифрового документа. Что будет эффективнее?

Подсказка: Обратите внимание на вычислительно сложные операции.

Ответ:

- **Вынести функции проверки в Rust через FFI**
- Запускать проверку в чистом Python
- Выполнять проверку вручную
- Добавлять много логов и `print()`

🔮 Шаг 15:

Миссия 15

Расположите шаги оптимизации генерации ключей RSA:

1. Проанализировать вычислительные узкие места
2. Переписать функции генерации на Rust
3. Протестировать корректность и скорость
4. Внедрить в основной проект через FFI

Подсказка: Сначала анализируем, затем оптимизируем и интегрируем.

Ответ:

- Проанализировать вычислительные узкие места
- Переписать функции генерации на Rust
- Протестировать корректность и скорость
- Внедрить в основной проект через FFI

🔮 Шаг 16:

Миссия 16

Изабель хочет ускорить многопоточное шифрование потоков сообщений. Какой подход выбрать?

Подсказка: Ищите безопасное многопоточное решение.

Ответ:

- **Использовать Rust с безопасной многопоточностью через FFI**
- Создавать потоки в Python без ограничений
- Обрабатывать потоки по одному
- Сохранять данные на диск перед каждым шагом

📖 Урок 3 - Ускорение симуляций (например, клеточный автомат).

🔮 Шаг 01:

Миссия 1

Лена запускает клеточный автомат на большом поле. Что поможет ускорить его работу?

Подсказка: Подумайте о способах уменьшить время вычислений без изменения логики.

Ответ:

- **Использовать массивы NumPy вместо списков Python**
- Писать весь код на Python без библиотек
- Добавлять print() в каждый цикл
- Сохранять каждый шаг в файл

Шаг 02:

Миссия 2

Сопоставьте элементы симуляции с методами ускорения:

Ответ:

- Клетки
 - Соседи
 - Правила обновления
 - Отображение состояния
1. Векторизация через NumPy
 2. Использование C/Rust через FFI
 3. Применение Cython для циклов
 4. Отрисовка раз в N шагов

Подсказка: Определите, где критична скорость вычислений, а где можно оптимизировать визуализацию.

- **Клетки** — Векторизация через NumPy
- **Соседи** — Использование C/Rust через FFI
- **Правила обновления** — Применение Cython для циклов
- **Отображение состояния** — Отрисовка раз в N шагов

Шаг 03:

Миссия 3

Максим хочет ускорить симуляцию с миллионами клеток. Что эффективнее всего?

Подсказка: Смотрите на инструменты низкого уровня для массивов.

Ответ:

- **Переписать критические функции на Rust и подключить через FFI**
- Увеличить количество print()
- Сохранять каждый шаг на диск
- Использовать обычные списки Python

Шаг 04:

Миссия 4

Расположите шаги оптимизации клеточного автомата:

1. Проанализировать узкие места
2. Вынести критические функции в Rust/C
3. Использовать векторизацию для массивов
4. Протестировать корректность и скорость

Подсказка: Сначала анализируем, затем ускоряем, потом проверяем.

Ответ:

- Проанализировать узкие места
- Вынести критические функции в Rust/C
- Использовать векторизацию для массивов
- Протестировать корректность и скорость

🔗 Шаг 05:

Миссия 5

Анна использует многопоточность для обновления клеток. Как улучшить производительность?
Подсказка: Смотрите на безопасные методы параллельного выполнения.

Ответ:

- **Использовать Rust с безопасной многопоточностью через FFI**
- Создавать потоки Python без ограничений
- Обработать клетки последовательно
- Добавлять print() в каждый поток

🔗 Шаг 06:

Миссия 6

Сопоставьте типы операций с методами ускорения:

Ответ:

- Обновление соседей
 - Расчет правил жизни
 - Отрисовка экрана
 - Сбор статистики
1. Rust через FFI
 2. Cython для циклов
 3. Отрисовка раз в N шагов
 4. Сбор вектором и подсчет через NumPy

Подсказка: Определите, где критична скорость, а где можно оптимизировать визуализацию.

- **Обновление соседей** — Rust через FFI
- **Расчет правил жизни** — Cython для циклов
- **Отрисовка экрана** — Отрисовка раз в N шагов
- **Сбор статистики** — Сбор вектором и подсчет через NumPy

🔗 Шаг 07:

Миссия 7

Дмитрий хочет ускорить симуляцию на огромной сетке. Какой подход даст максимальный прирост скорости?

Подсказка: Смотрите на низкоуровневые методы работы с массивами.

Ответ:

- **Переписать критические вычисления на Rust и подключить через FFI**
- Добавить больше print()
- Использовать обычные списки Python
- Сохранять каждый шаг на диск

🔧 Шаг 08:

Миссия 8

Расположите шаги ускорения клеточного автомата:

1. Проанализировать узкие места
2. Векторизовать массивы через NumPy
3. Переписать критические функции на Rust/C
4. Тестирование корректности и сравнение скорости

Подсказка: Сначала анализируем, затем ускоряем, затем проверяем.

Ответ:

- Проанализировать узкие места
- Векторизовать массивы через NumPy
- Переписать критические функции на Rust/C
- Тестирование корректности и сравнение скорости

🔧 Шаг 09:

Миссия 9

Ирина хочет ускорить симуляцию с большим количеством клеток, которые почти не меняются каждый шаг. Что лучше использовать?

Подсказка: Подумайте о том, что не все клетки нужно обновлять.

Ответ:

- **Обновлять только изменившиеся клетки**
- Пересчитывать все клетки каждый шаг
- Сохранять каждый шаг в файл
- Добавлять задержки между шагами

🔧 Шаг 10:

Миссия 10

Сопоставьте подходы ускорения с примерами:

Ответ:

- Мемоизация правил
 - Использование массивов NumPy
 - Переписывание на Rust
 - Пакетная отрисовка
1. Сохраняем вычисленные результаты для повторного использования
 2. Работа с целыми блоками данных в памяти
 3. Критические вычисления на низком уровне
 4. Отрисовка не каждый шаг, а блоками

Подсказка: Смотрите, где можно использовать память для ускорения и где важна низкоуровневая оптимизация.

- **Мемоизация правил** — Сохраняем вычисленные результаты для повторного использования
- **Использование массивов NumPy** — Работа с целыми блоками данных в памяти
- **Переписывание на Rust** — Критические вычисления на низком уровне
- **Пакетная отрисовка** — Отрисовка не каждый шаг, а блоками

🔮 Шаг 11:

Миссия 11

Вячеслав хочет уменьшить время симуляции клеточного автомата на GPU. Что ему поможет?

Подсказка: Смотрите на библиотеки для массивов и массивные вычисления.

Ответ:

- **Использовать CuPy или Numba для массивных вычислений на GPU**
- Сохранять каждый шаг на диск
- Добавлять print() в каждый цикл
- Использовать обычные списки Python

🔮 Шаг 12:

Миссия 12

Расположите шаги оптимизации клеточного автомата на GPU:

1. Переписать функции на CUDA/Numba
2. Оптимизировать массивы для GPU
3. Векторизовать вычисления
4. Тестирование и измерение скорости

Подсказка: Сначала готовим данные, затем переписываем функции, потом тестируем.

Ответ:

- Переписать функции на CUDA/Numba
- Оптимизировать массивы для GPU
- Векторизовать вычисления
- Тестирование и измерение скорости

🔮 Шаг 13:

Миссия 13

Екатерина хочет ускорить симуляцию с миллионами клеток и сложными правилами. Какой подход выбрать?

Подсказка: Подумайте о комбинации низкоуровневого кода и векторизации.

Ответ:

- **Векторизовать массивы и критические функции переписать на Rust/C**
- Добавлять print() для отладки на каждом шаге
- Обрабатывать клетки последовательно
- Сохранять каждый шаг на диск

🔮 Шаг 14:

Миссия 14

Сопоставьте тип данных с методом ускорения:

Ответ:

- Списки Python
- NumPy массивы
- Массивы на GPU

- Структуры в Rust
1. Медленно для больших объемов
 2. Быстро для CPU с векторизацией
 3. Высокая скорость параллельных вычислений
 4. Критические вычисления на низком уровне

Подсказка: Подумайте, где узкое место — CPU, где GPU, где низкоуровневый код.

- **Списки Python** — Медленно для больших объемов
- **NumPy массивы** — Быстро для CPU с векторизацией
- **Массивы на GPU** — Высокая скорость параллельных вычислений
- **Структуры в Rust** — Критические вычисления на низком уровне

Шаг 15:

Миссия 15

Игорь хочет ускорить симуляцию клеточного автомата на многопроцессорной машине. Что выбрать?

Подсказка: Используйте безопасные способы параллелизации.

Ответ:

- **Использовать многопоточность через Rust или multiprocessing с Cython**
- Создавать произвольные потоки Python без контроля
- Обновлять все клетки последовательно
- Добавлять print() в каждый поток

Шаг 16:

Миссия 16

Расположите шаги ускорения симуляции на многопроцессорной машине:

1. Выделить критические функции
2. Распараллелить на потоки или процессы
3. Векторизовать массивы
4. Тестирование и замеры времени

Подсказка: Сначала выбираем, что ускорять, потом распараллеливаем и проверяем.

Ответ:

- Выделить критические функции
- Распараллелить на потоки или процессы
- Векторизовать массивы
- Тестирование и замеры времени

Урок 4 - Финальный проект ускорить настоящий Python-проект в 10+ раз.

Шаг 01:

Миссия 1

Анна начала финальный проект. Она хочет измерить, какие части кода работают медленно. Что стоит использовать?

Подсказка: Сначала нужно понять, где узкие места, прежде чем ускорять код.

Ответ:

- **Использовать профилировщик Python (например, cProfile)**
- Просто угадывать, что медленно
- Добавлять print() в случайные места
- Удалять код без анализа

🔮 **Шаг 02:**

Миссия 2

После профилирования Иван заметил, что функция сортировки обрабатывает миллионы элементов. Как ускорить?

Подсказка: Рассмотрите использование готовых высокопроизводительных библиотек или C/Rust.

Ответ:

- **Переписать критическую функцию на Rust или использовать C-расширение**
- Добавить больше print()
- Сортировать вручную через Python-циклы
- Уменьшить данные, удаляя элементы случайно

🔮 **Шаг 03:**

Миссия 3

Сопоставьте подходы ускорения с их описанием:

Ответ:

- Использование C-расширений
 - Векторизация с NumPy
 - Многопоточность через Rust
 - Кэширование результатов
1. Критические вычисления на низком уровне
 2. Обработка массивов блоками в памяти
 3. Распараллеливание работы на ядрах CPU
 4. Повторные вызовы функции используют заранее вычисленные результаты

Подсказка: Смотрите, где важна скорость CPU, где память и где параллелизм.

- **Использование C-расширений** — Критические вычисления на низком уровне
- **Векторизация с NumPy** — Обработка массивов блоками в памяти
- **Многопоточность через Rust** — Распараллеливание работы на ядрах CPU
- **Кэширование результатов** — Повторные вызовы функции используют заранее вычисленные результаты

🔮 **Шаг 04:**

Миссия 4

Ольга работает с большим JSON и хочет ускорить разбор. Что выбрать?

Подсказка: Стандартный Python JSON может быть медленным для огромных файлов.

Ответ:

- **Использовать быстрые C-библиотеки для JSON, например, orjson**
- Использовать стандартный json без изменений
- Разбивать JSON вручную на строки

- Игнорировать проверку данных

Шаг 05:

Миссия 5

Сопоставьте методы оптимизации с типами данных:

Ответ:

- Списки Python
 - NumPy массивы
 - Массивы на GPU
 - Структуры в Rust
1. Медленно для больших объёмов
 2. Быстро для CPU с векторизацией
 3. Высокая скорость параллельных вычислений
 4. Критические вычисления на низком уровне

Подсказка: Где узкое место — CPU, где GPU, где низкоуровневый код.

- **Списки Python** — Медленно для больших объёмов
- **NumPy массивы** — Быстро для CPU с векторизацией
- **Массивы на GPU** — Высокая скорость параллельных вычислений
- **Структуры в Rust** — Критические вычисления на низком уровне

Шаг 06:

Миссия 6

Сергей хочет ускорить алгоритм поиска в огромной базе данных. Что поможет?

Подсказка: Индексация и структуры данных часто решают проблему поиска.

Ответ:

- **Использовать индексированные структуры и C/Rust ускорение**
- Последовательно проверять все записи Python-циклом
- Сортировать данные случайным образом
- Удалять половину данных

Шаг 07:

Миссия 7

Расположите шаги оптимизации Python-проекта для ускорения в 10+ раз:

1. Профилирование кода
2. Выделение критических функций
3. Использование C/Rust для критических функций
4. Векторизация и оптимизация работы с данными
5. Параллелизация при необходимости
6. Тестирование и контроль корректности

Подсказка: Сначала анализируем, потом оптимизируем, потом тестируем.

Ответ:

- Профилирование кода
- Выделение критических функций

- Использование C/Rust для критических функций
- Векторизация и оптимизация работы с данными
- Параллелизация при необходимости
- Тестирование и контроль корректности

🔮 Шаг 08:

Миссия 8

Мария хочет уменьшить время загрузки данных в проекте. Что ускорит процесс?

Подсказка: Используйте бинарные форматы и быстрые библиотеки для I/O.

Ответ:

- **Использовать бинарные форматы (например, Parquet, Feather) и быстрые библиотеки**
- Хранить данные как большие CSV
- Загружать данные по одному элементу
- Игнорировать формат данных

🔮 Шаг 09:

Миссия 9

Сопоставьте типы ускорений с их применением:

Ответ:

- Векторизация
- Многопоточность
- Кэширование
- Использование C/Rust

1. Обработка больших массивов данных за один проход
2. Параллельная работа на нескольких ядрах CPU
3. Сохранение часто используемых результатов для повторного доступа
4. Критические функции на низком уровне для максимальной скорости

Подсказка: Векторизация подходит для массивов, многопоточность для CPU, кэширование для повторных вызовов, а C/Rust для критического кода.

- **Векторизация** — Обработка больших массивов данных за один проход
- **Многопоточность** — Параллельная работа на нескольких ядрах CPU
- **Кэширование** — Сохранение часто используемых результатов для повторного доступа
- **Использование C/Rust** — Критические функции на низком уровне для максимальной скорости

🔮 Шаг 10:

Миссия 10

Дмитрий заметил, что при работе с большими CSV проект тормозит. Как ускорить загрузку данных?

Подсказка: Бинарные форматы и сторонние библиотеки часто быстрее, чем стандартный Python I/O.

Ответ:

- **Использовать бинарные форматы и быстрые библиотеки, например, Parquet или Feather**

- Читать CSV по строке с помощью стандартного Python
- Игнорировать обработку ошибок
- Делить файл на части вручную

🔮 Шаг 11:

Миссия 11

Сопоставьте типы данных с оптимальными подходами ускорения:

Ответ:

- Списки Python
 - NumPy массивы
 - Данные на GPU
 - Структуры в Rust
1. Медленно для больших объёмов
 2. Быстро с векторизацией CPU
 3. Высокая скорость параллельных вычислений
 4. Критические вычисления на низком уровне

Подсказка: Python-списки медленные, NumPy быстро векторизуется, GPU для параллельных массивов, Rust для низкоуровневых функций.

- **Списки Python** — Медленно для больших объёмов
- **NumPy массивы** — Быстро с векторизацией CPU
- **Данные на GPU** — Высокая скорость параллельных вычислений
- **Структуры в Rust** — Критические вычисления на низком уровне

🔮 Шаг 12:

Миссия 12

Ирина хочет ускорить обработку изображений в проекте. Что выбрать?

Подсказка: Векторизация и C/Rust-расширения часто дают существенный прирост.

Ответ:

- **Использовать NumPy для массивов и C/Rust для критических функций**
- Использовать чистый Python циклом
- Игнорировать оптимизацию памяти
- Писать вручную каждый пиксель на Python

🔮 Шаг 13:

Миссия 13

Павел хочет ускорить сетевые запросы в Python-проекте. Как лучше поступить?

Подсказка: Асинхронность и библиотеки с C/Rust ускорением помогут одновременно обрабатывать множество запросов.

Ответ:

- **Использовать асинхронные библиотеки с C/Rust ускорением**
- Делать запросы последовательно через requests
- Игнорировать обработку ошибок
- Добавлять sleep между запросами

🔧 Шаг 14:

Миссия 14

Расположите шаги интеграции C/Rust ускорений в проект:

1. Выделение критических функций
2. Переписывание на C/Rust
3. Компиляция и подключение расширения
4. Тестирование корректности и производительности
5. Профилирование всего проекта

Подсказка: Сначала выделяем узкие места, потом переписываем, подключаем и проверяем.

Ответ:

- Выделение критических функций
- Переписывание на C/Rust
- Компиляция и подключение расширения
- Тестирование корректности и производительности
- Профилирование всего проекта

🔧 Шаг 15:

Миссия 15

Сопоставьте способы ускорения с типами узких мест:

Ответ:

- CPU-бутылочное горлышко
 - I/O операции
 - Большие данные в памяти
 - Повторные вычисления
1. Переписать на C/Rust или оптимизировать алгоритм
 2. Использовать бинарные форматы и быстрые библиотеки
 3. Векторизация и NumPy
 4. Кэширование результатов

Подсказка: Подумайте, где нужен CPU, где память, а где I/O.

- **CPU-бутылочное горлышко** — Переписать на C/Rust или оптимизировать алгоритм
- **I/O операции** — Использовать бинарные форматы и быстрые библиотеки
- **Большие данные в памяти** — Векторизация и NumPy
- **Повторные вычисления** — Кэширование результатов

🔧 Шаг 16:

Миссия 16

Екатерина хочет ускорить сложные математические расчеты. Что оптимальнее всего?

Подсказка: Используйте C/Rust для критических функций и векторизацию массивов.

Ответ:

- **Комбинировать C/Rust функции с NumPy векторизацией**
- Выполнять всё в чистом Python циклом
- Игнорировать массивы и использовать скаляры
- Переписывать всё на Bash скрипты